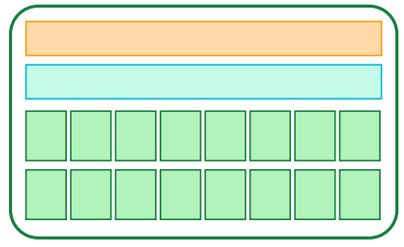
A course in 40 boards

GPU Architecture



From basic computer science to modern hardware, one diagram at a time.

What is inside

- **Eight lessons.** Each one is a single board that covers one topic from end to end, from why GPUs exist to what the newest ones add.
- **Thirty-two sub-lessons.** Lessons 1.1 to 8.4 go deeper into each topic, four per lesson, with worked numbers and short code examples where they help.
- **A notes page after every board.** It adds what does not fit on the board, the key terms, and one question to check your understanding.
- **Answers at the back.** Every check question is answered, with the working shown.

How to read the boards

- **The map stays the same.** The left half of every board is the whole GPU: the host, the card, the chip with its grid of SMs, the caches and VRAM.
- **Orange marks the lesson.** Parts outlined in orange are what the lesson covers. Numbered badges on them match the numbered steps on the right.
- **The zoom reads top to bottom.** Each step has a small picture, a heading that states the point, and three lines of detail. Follow the arrows down.

Figures such as cache sizes, latencies and bandwidth are orders of magnitude. They vary by chip and generation.

Contents

Lessons and sub-lessons

1 **Why GPUs exist** 3
- 1.1 Where a CPU's transistors go 5
- 1.2 Why single-thread speed stopped growing 7
- 1.3 Amdahl's law 9
- 1.4 What makes a workload parallel 11

2 **One instruction, many threads** 13
- 2.1 SIMD up close 15
- 2.2 Inside a warp 17
- 2.3 Divergence, step by step 19
- 2.4 Latency hiding with numbers 21

3 **Inside a GPU core** 23
- 3.1 Warp scheduling 25
- 3.2 The register file and occupancy 27
- 3.3 Execution units and throughput 29
- 3.4 One instruction's timeline 31

4 **The memory hierarchy** 33
- 4.1 How a cache works 35
- 4.2 Shared memory and bank conflicts 37
- 4.3 VRAM and bandwidth math 39
- 4.4 Moving data between host and GPU 41

5 **The graphics pipeline** 43
- 5.1 Coordinate spaces and the vertex stage 45
- 5.2 Rasterization 47
- 5.3 Texturing and mipmaps 49
- 5.4 Depth, early depth testing and blending 51

6 **Programming the GPU directly** 53
- 6.1 Thread indexing in 1D, 2D and 3D 55
- 6.2 Cooperation inside a block 57
- 6.3 From source code to machine instructions 59
- 6.4 Launches and asynchronous execution 61

7 **Why GPU code runs fast or slow** 63
- 7.1 Coalescing and data layout 65
- 7.2 Tuning occupancy 67
- 7.3 Compute-bound or memory-bound? 69
- 7.4 Reading a profile 71

8 **What modern GPUs add** 73
- 8.1 Tensor cores and mixed precision 75
- 8.2 How a ray finds its triangle 77
- 8.3 Tile-based rendering 79
- 8.4 Stacked memory and multi-GPU links 81

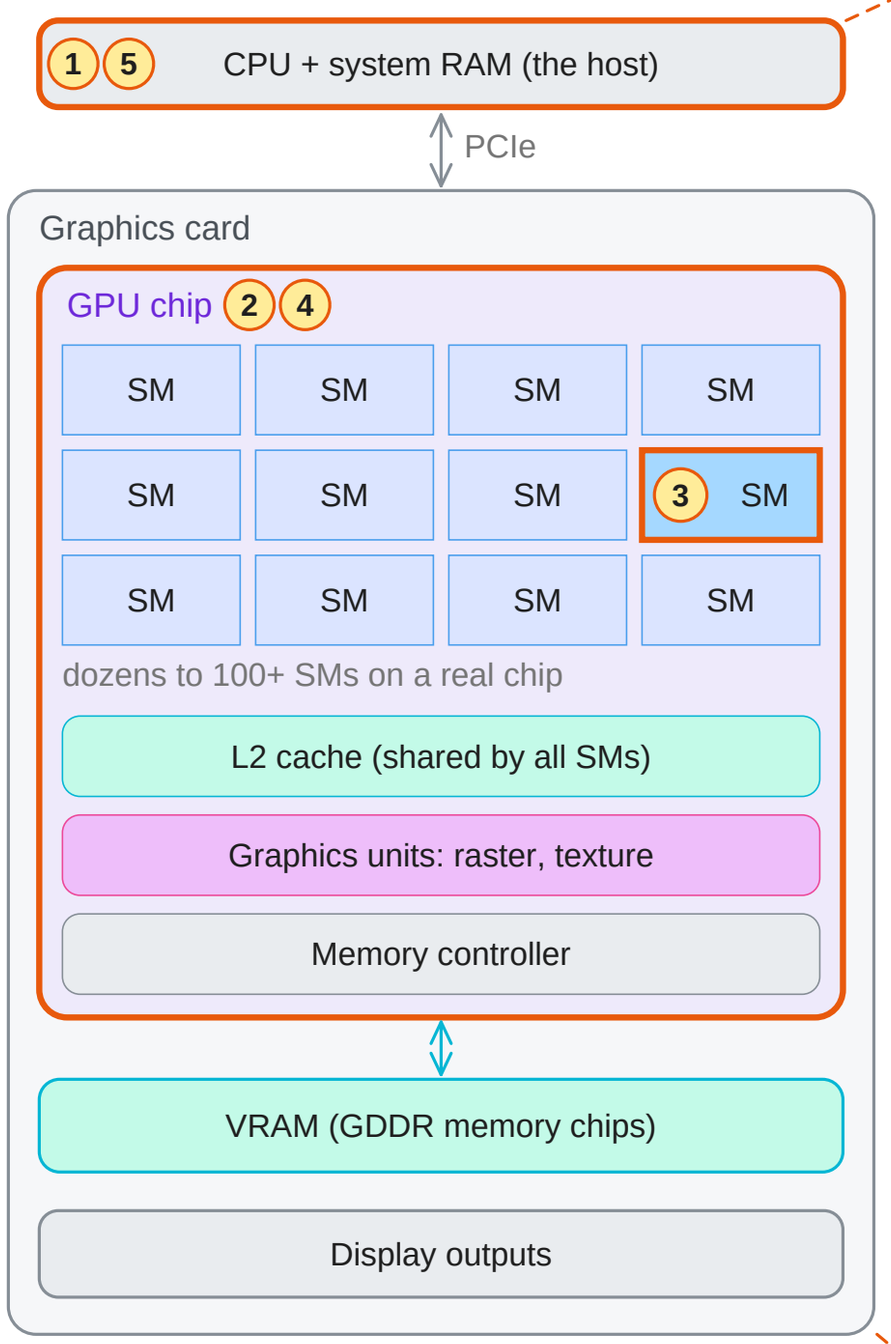
Answers to the check questions 83

Appendix: inside one SM, in detail 87

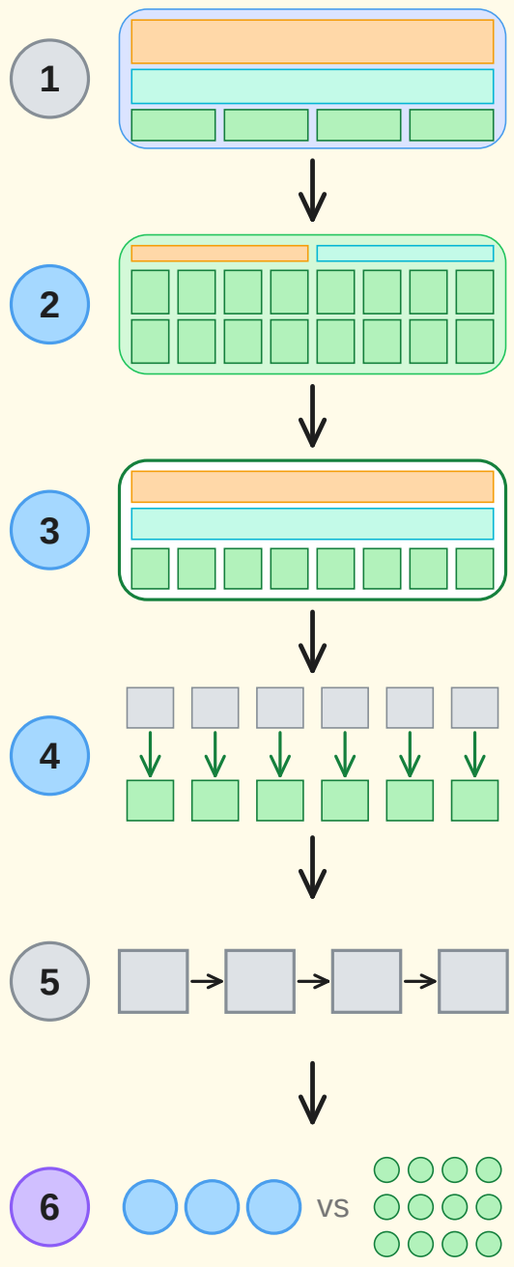
Lesson 1: Why GPUs exist

Same chip area, two opposite goals. Orange = control, teal = memory, green = arithmetic.

The whole GPU (map)



Zoomed in: two chips, two goals



CPU: finish one task fast

A few powerful cores. Most of the chip is control logic and cache, all there so a single thread never has to wait.

GPU: finish the most work per second

Thousands of simple arithmetic units and very little control. Any one item may be slow, but the total throughput is huge.

Inside the GPU: groups called SMs

The units are grouped into SMs. Each SM has a small scheduler, some local memory, and rows of ALUs that do the math.

Work that suits a GPU

One operation on millions of independent items: shading pixels, multiplying big matrices, updating particles.

Work that suits a CPU

Steps that depend on the previous step: parsing a file, running an operating system, branchy and unpredictable logic.

The analogy

CPU = a few expert chefs: any dish, fast.
GPU = a thousand line cooks: one simple recipe, all at the same time.

Key idea: a GPU trades single-thread speed for massive parallel volume

Lesson 1 notes

Why GPUs exist

In brief

A CPU and a GPU are built from the same transistors but optimize for opposite goals. A CPU minimizes latency: it finishes one arbitrary task as fast as possible. A GPU maximizes throughput: it finishes the most total work per second and does not care how long any single item takes.

Beyond the board

- **Where the CPU spends its area.** Mostly on things that are not arithmetic: branch predictors, out-of-order execution machinery and large caches. All of it exists to keep a handful of powerful cores from ever waiting.
- **Where the GPU spends its area.** It strips each core down to something simple and spends the savings on having thousands of them, with only a small scheduler and a little local memory per group.
- **The workload that pays off.** The same operation applied to a huge number of independent items. If each step depends on the last one, the extra lanes have nothing to do.
- **ALUs live inside SMs.** "ALU" is the generic name for a unit that does arithmetic and logic. In a real SM that job is split across integer units, floating-point units, SFUs and tensor cores (Lesson 3).
- **The picture is schematic.** Block sizes are not to scale, and a real SM has on the order of a hundred arithmetic lanes rather than the handful drawn.

Check yourself

If the GPU has so little control logic and cache per core, what happens when one of its threads has to wait a long time for data from memory?

Answer on page 83.

Key terms

Latency: time to finish one task.

Throughput: total work finished per second.

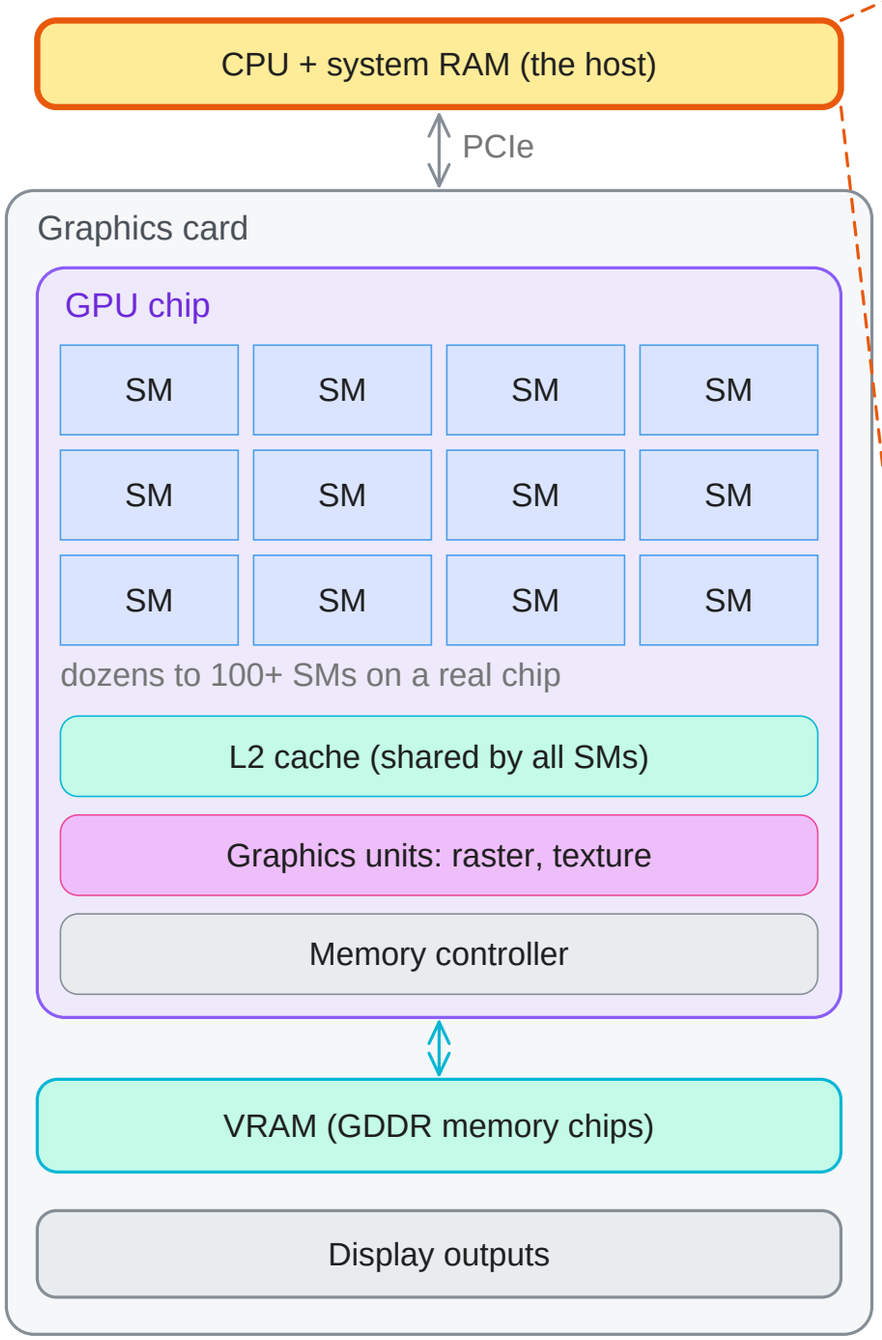
ALU: arithmetic logic unit, the part that does the math.

SM: streaming multiprocessor, one group of GPU execution units.

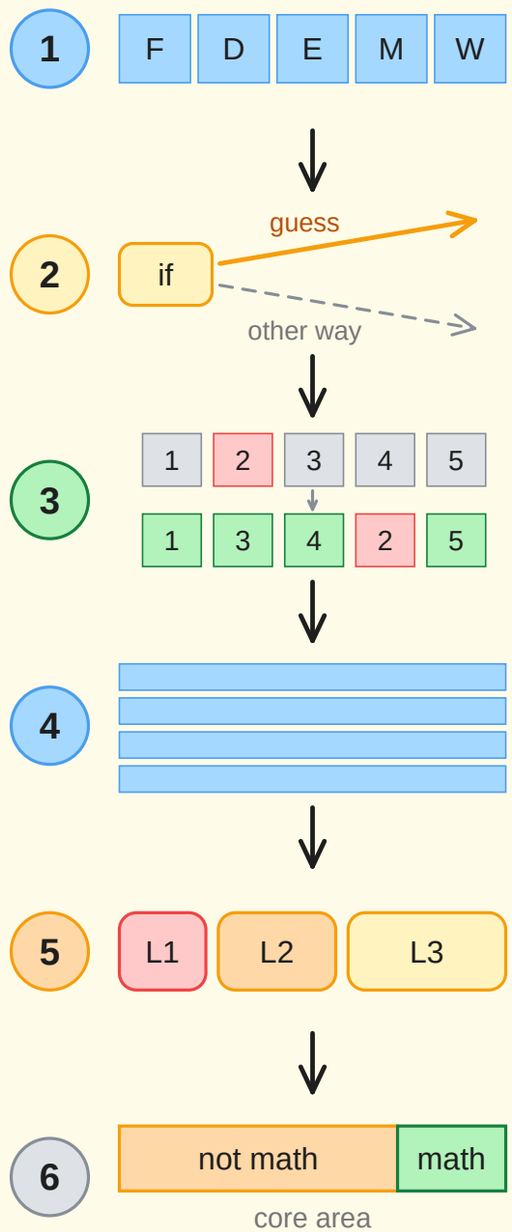
Lesson 1.1: Where a CPU's transistors go

A closer look at the CPU side of Lesson 1: five tricks a core uses so that one thread never waits.

The whole GPU (map)



Zoomed in: inside one CPU core



Pipelining: overlap the steps

Each instruction is split into stages: fetch, decode, execute, memory, write. Several instructions are in flight at once.

Branch prediction: guess, don't wait

The CPU guesses which way an if will go and runs ahead. Right over 95% of the time. A wrong guess wastes 15-20 cycles.

Out-of-order: skip past the stall

If instruction 2 is waiting on memory, the CPU runs 3 and 4 first, then puts the results back in program order.

Superscalar: several per cycle

Wide cores fetch, run and finish 4 to 8 instructions every cycle, so the parts above are all built several times over.

Big caches: keep data close

L1 answers in about 4 cycles. Main memory takes a few hundred. Three cache levels exist to make the slow trip rare.

The bill: area that does no math

All of this serves ONE thread. The units that do arithmetic are a small part of the core. A GPU makes the opposite bet.

Key idea: a CPU core is mostly machinery for not waiting, and only a small part of it does arithmetic

Lesson 1.1 notes

Where a CPU's transistors go

In brief

A modern CPU core spends most of its transistors on avoiding delay rather than on arithmetic. It overlaps instructions, guesses the outcome of branches, runs instructions out of order, does several per cycle, and keeps data in large caches. All five tricks serve a single thread.

Beyond the board

- **Every trick has a GPU counterpart.** Where the CPU predicts branches, a GPU masks lanes off (Lesson 2.3). Where the CPU reorders instructions around a stall, a GPU switches to another warp (Lesson 2.4). Where the CPU leans on big caches, a GPU leans on bandwidth and on hiding the wait.
- **Returns shrink as cores grow.** A rule of thumb called Pollack's rule says a core's speed grows roughly with the square root of its complexity. Doubling the area buys about 40% more speed, while two simple cores in the same area could do twice the work.
- **These features multiply each other's cost.** A core that handles 6 instructions per cycle needs 6 decoders, a larger reorder window and a better predictor to keep them all fed.
- **The numbers are typical, not exact.** Predictor accuracy, pipeline depth and cache latency differ from chip to chip, but the proportions are representative.

Check yourself

A branch predictor is right 95% of the time, and a wrong guess costs 20 cycles. If 1 instruction in 5 is a branch, how many cycles do wrong guesses add per 100 instructions?

Answer on page 83.

Key terms

Pipeline: the stages an instruction passes through.

Branch prediction: guessing which way an if will go.

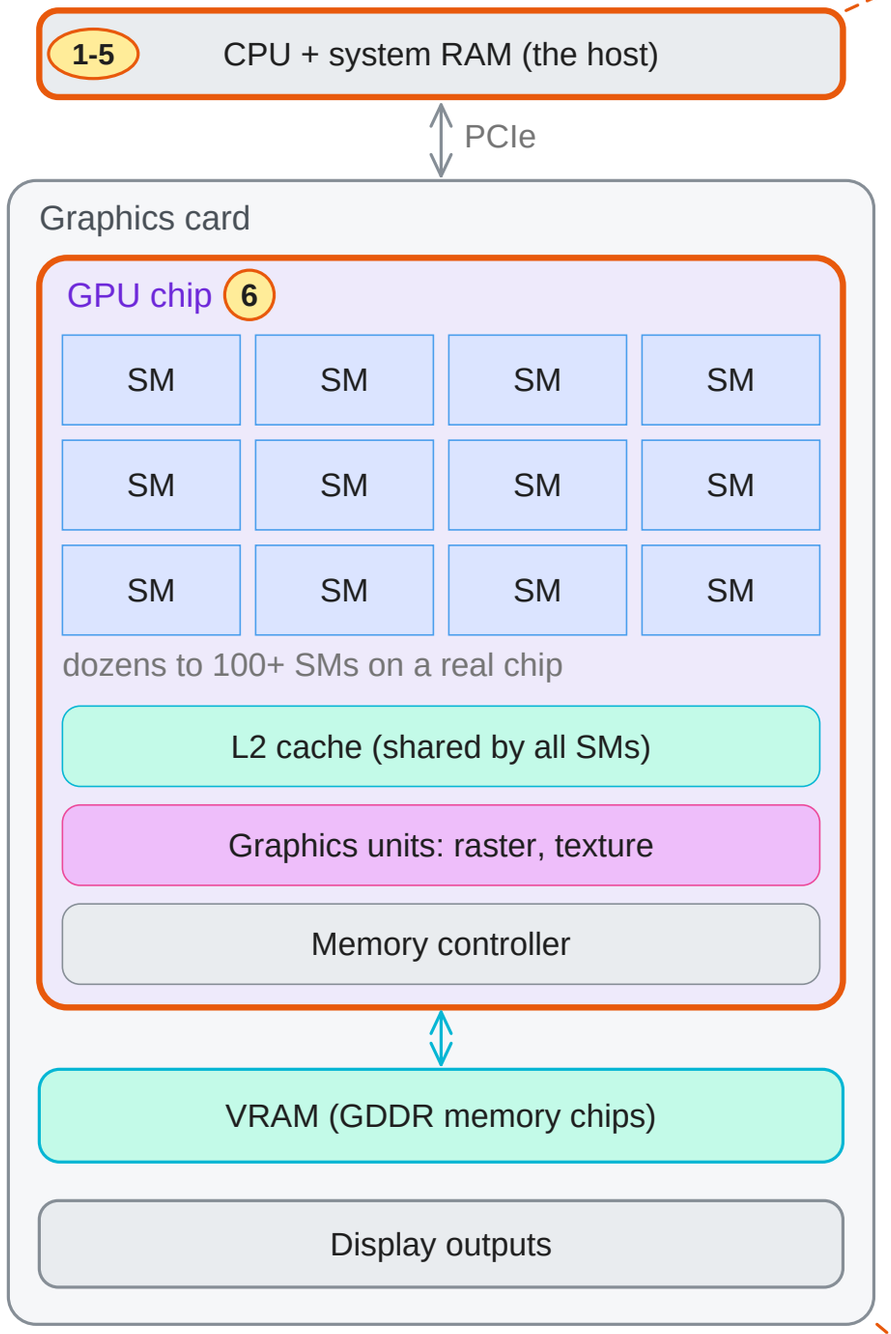
Out-of-order execution: running later instructions while an earlier one waits.

Superscalar: handling several instructions per cycle.

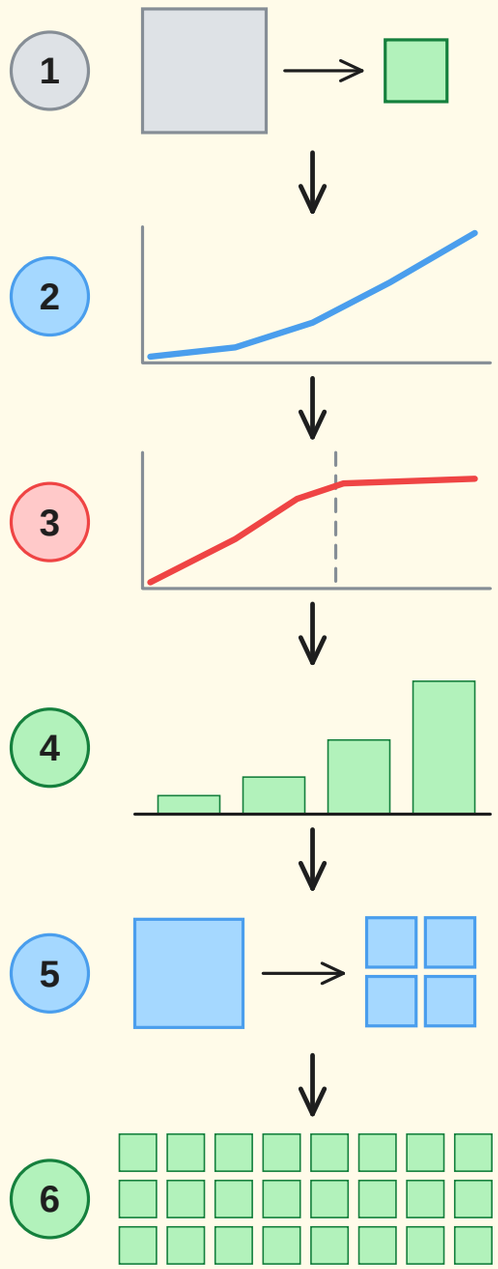
Lesson 1.2: Why single-thread speed stopped growing

For decades chips got faster for free. Around 2005 that ended, and parallel hardware became the way forward.

The whole GPU (map)



Zoomed in: from faster clocks to more cores



The old deal: smaller meant faster

Each new generation shrank transistors. Smaller ones switched faster and used less power, so chips sped up for free.

1990 to 2004: clock speed soars

Desktop chips went from about 30 MHz to over 3 GHz, roughly 100 times faster, without programs having to change.

About 2005: the power wall

Voltage could not drop any further, so faster clocks meant more heat than a chip can shed. Speeds stalled at 3-5 GHz.

Transistor counts kept growing

Chips still fit more transistors each generation. They just could not all be switched any faster than before.

The fix: more cores, not faster ones

From 2005, the extra transistors became extra cores: 2, then 4, then dozens. Speed now comes from doing work in parallel.

The GPU takes this to the limit

If speed must come from parallelism, build thousands of simple lanes rather than a few clever cores. That is Lesson 1.

Key idea: when clocks stopped getting faster, extra transistors could only help by working in parallel

Lesson 1.2 notes

Why single-thread speed stopped growing

In brief

Until about 2005, each generation of chips ran the same programs faster with no effort from programmers. Then clock speeds hit a power limit. Transistor counts kept rising, so the extra transistors were spent on more cores, and speed began to depend on parallel work.

Beyond the board

- **Two trends that are often confused.** Moore's law is about how many transistors fit on a chip. Dennard scaling said that smaller transistors could also run faster at the same power density. Moore's law continued; Dennard scaling is what ended.
- **Why voltage was the wall.** A chip's switching power grows with voltage squared times clock frequency. For decades voltage fell with each generation, which paid for the faster clock. Below roughly 1 volt, transistors leak too much current, so voltage stopped falling.
- **Single-thread speed still improves, slowly.** It used to grow by about 50% per year. Since the mid-2000s the gain has been a small fraction of that, mostly from the tricks in Lesson 1.1.
- **Dates and figures are approximate.** The first mainstream dual-core desktop chips arrived in 2005. Top clock speeds have crept from under 4 GHz to a little over 5 GHz in the years since.

Check yourself

Switching power grows with voltage squared times frequency. With voltage fixed, how much more power does it take to go from 4 GHz to 5 GHz? How far would voltage have to fall to make that step free?

Answer on page 83.

Key terms

Clock speed: how many cycles a chip runs per second.

Moore's law: transistor counts double about every two years.

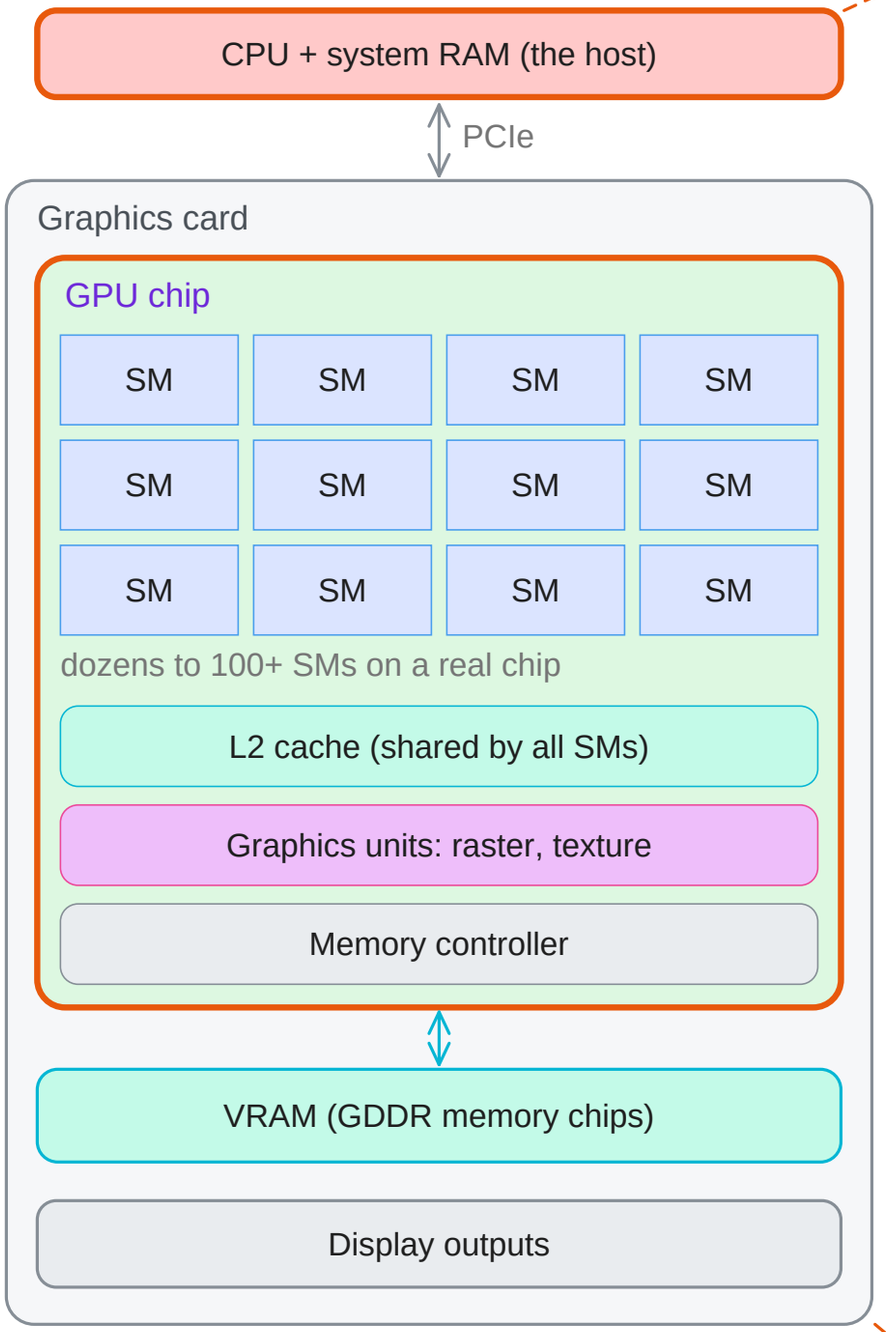
Dennard scaling: smaller transistors using proportionally less power.

Power wall: the heat limit that stopped clock speeds rising.

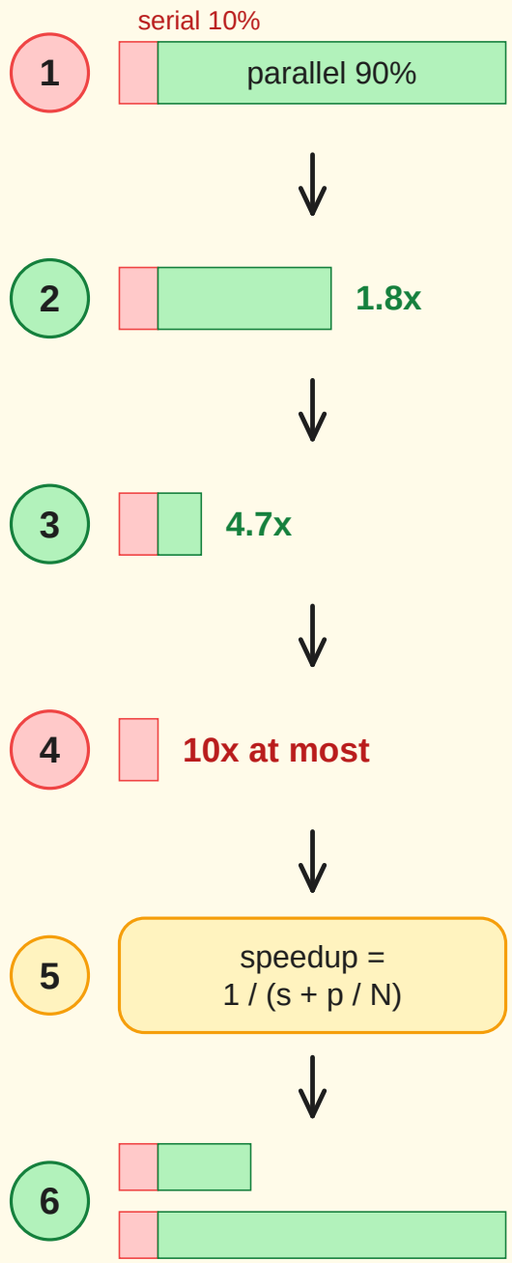
Lesson 1.3: Amdahl's law

How much can parallel hardware speed a program up? The part that cannot be split sets the limit.

The whole GPU (map)



Zoomed in: where the time goes as cores are added



One program, two kinds of work

Red must run in order: setup, file reads, steps that depend on each other. Green can be split up. Here: 10% red, 90% green.

2 cores: green takes half as long

Time falls from 100 to 10 + 45 = 55. That is 1.8 times faster, not 2, because the red part did not shrink at all.

8 cores: red starts to dominate

Time is 10 + 11 = 21, so 4.7 times faster from 8 times the hardware. Red is now almost half of the whole run.

Unlimited cores: stuck at 10x

Green drops to nothing, but red still takes 10. The speedup can never pass 1 / serial fraction = 1 / 0.10 = 10.

Amdahl's law

s = serial fraction, p = parallel fraction, N = number of cores. Check: 1 / (0.10 + 0.90 / 8) = 4.7, matching step 3.

Why GPUs still win: bigger problems

More pixels or bigger matrices grow the green part while red stays the same, so the serial fraction shrinks toward zero.

Key idea: the part of a program that cannot run in parallel sets a hard ceiling on the speedup

Lesson 1.3 notes

Amdahl's law

In brief

Every program has a part that must run in order and a part that can be split across cores. Adding cores shrinks only the second part, so the first part comes to dominate. Amdahl's law puts a number on it: speedup = 1 / (s + p / N).

Parallel share	8 cores	1,024 cores	Ceiling
50%	1.8x	2.0x	2x
90%	4.7x	9.9x	10x
95%	5.9x	19.6x	20x
99%	7.5x	91x	100x
99.9%	7.9x	506x	1,000x

Beyond the board

- **Reading the table.** Thousands of lanes only pay off when the parallel share is very close to 100%. At 90% parallel, 1,024 cores are only about twice as fast as 8.
- **Gustafson's law is the other half.** Step 6 has a name. In practice people use bigger machines to solve bigger problems, and the parallel part usually grows with the problem while the serial part does not.
- **The law is optimistic.** It assumes the parallel part splits perfectly. Real programs also pay for coordination and for moving data, which is what Lessons 4 and 7 are about.
- **N does not have to be a core count.** It can be any factor by which one part gets faster, such as moving a loop onto a GPU.

Check yourself

A program is 95% parallel. You move the parallel part to a GPU that runs it 100 times faster. What is the overall speedup, and what should you optimize next?

Answer on page 83.

Key terms

Serial fraction: the share of run time that must happen in order.

Speedup: old run time divided by new run time.

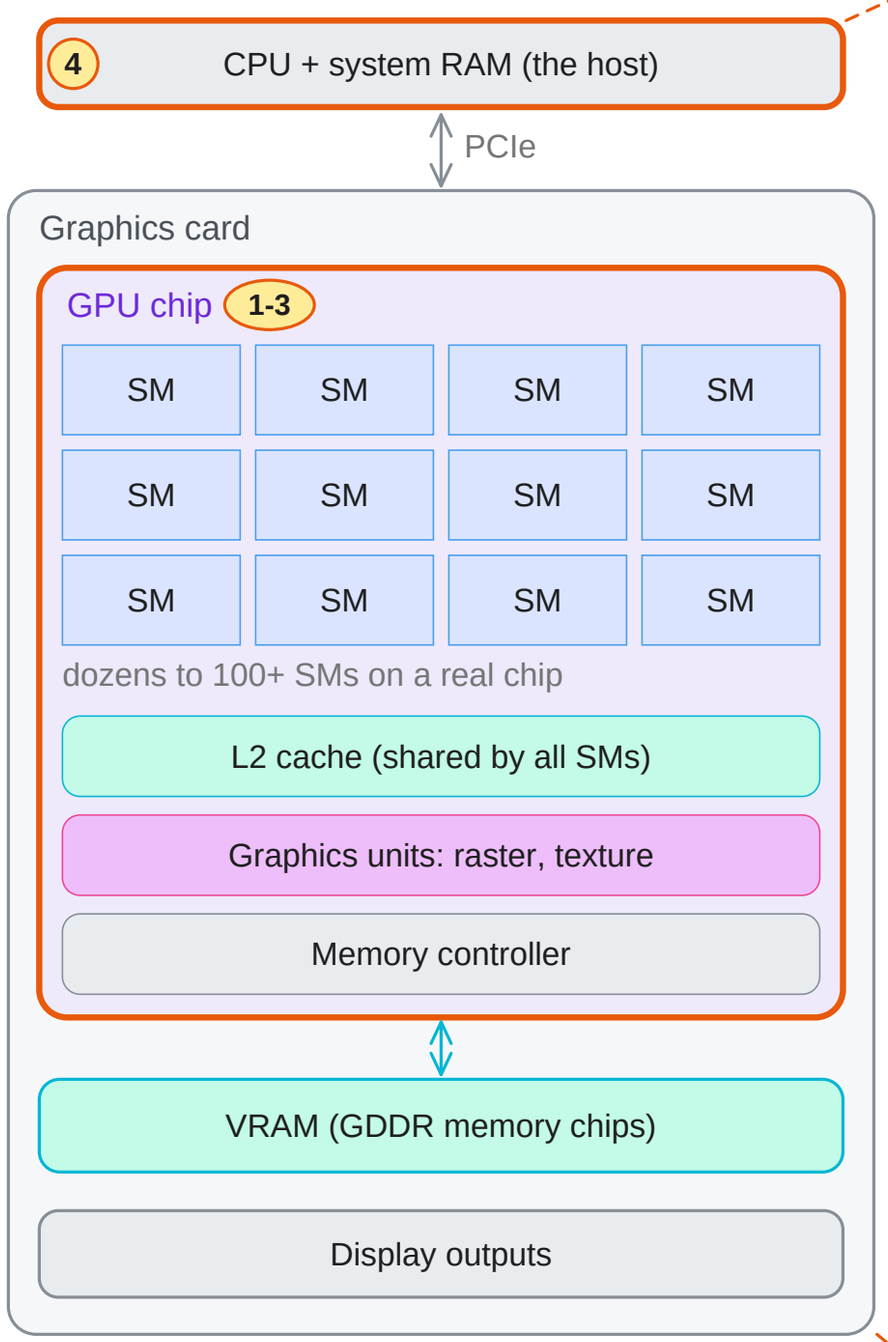
Amdahl's law: speedup = 1 / (s + p / N).

Gustafson's law: bigger problems have smaller serial fractions.

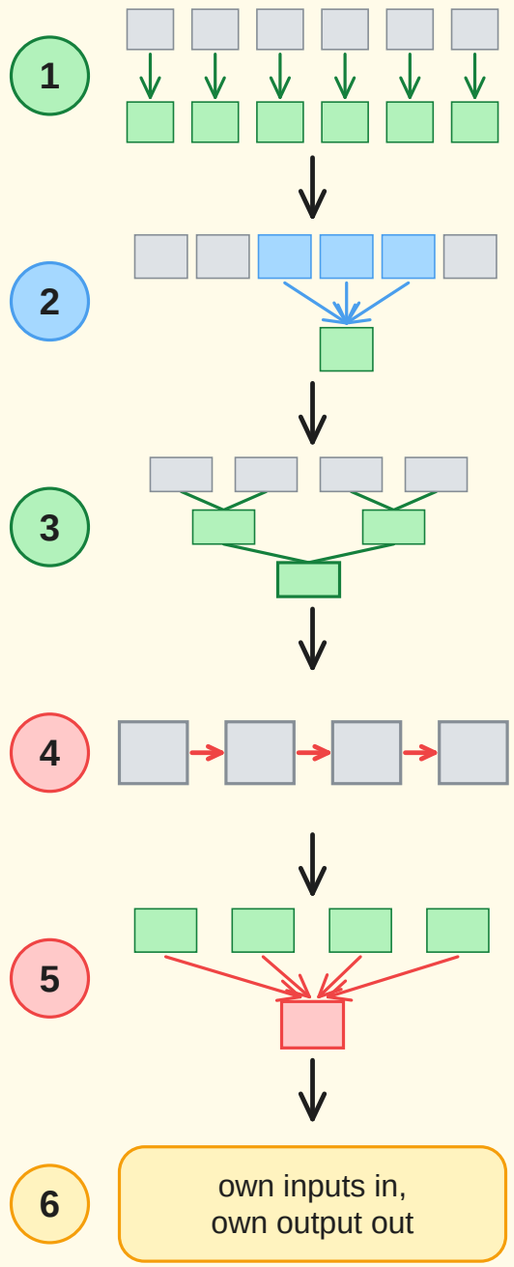
Lesson 1.4: What makes a workload parallel

It comes down to what each piece of work depends on. Five patterns, from GPU-friendly to stuck in order.

The whole GPU (map)



Zoomed in: five dependency patterns



Independent items: the ideal case

Each output needs only its own input: brighten a pixel, add two array elements. Any order works, so all can run at once.

Neighbour reads: still parallel

A blur reads several nearby pixels but writes only its own. Reading never clashes, so every output can still run at once.

Combining: parallel as a tree

A sum looks serial, but pairs can be added side by side, then pairs of pairs. A million values take about 20 rounds, not a million.

Chains: stuck in order

Each step needs the result of the one before: parsing a file, or moving a simulation forward one moment at a time.

Shared writes: threads collide

Many threads updating one counter or one histogram bin overwrite each other. It needs special care, and that costs speed.

The test for GPU-friendly work

Can each item be computed from its own inputs and written to its own place? If yes for millions of items, use a GPU.

Key idea: work is parallel when each item reads what it needs and writes only to its own place

Lesson 1.4 notes

What makes a workload parallel

In brief

Whether work can be spread over thousands of lanes depends on its dependencies: what each item has to read and where it writes. Independent items, neighbour reads and tree-shaped combining all suit a GPU. Chains and shared writes do not, at least not without extra work.

Beyond the board

- **Real programs mix the patterns.** A physics simulation is a chain over time, because each step needs the last one. Inside a single step, though, every particle can be updated independently. The GPU takes the inside, and the chain stays serial.
- **Neighbour reads need two copies.** A blur must read the old image and write a new one. If it wrote into the image it was reading, some threads would see pixels that were already blurred.
- **The tree trick needs order not to matter.** Adding in pairs gives the same answer only if grouping does not change the result. For floating-point numbers it changes it very slightly, so a parallel sum can differ from a serial one in its last digits.
- **Shared writes have standard fixes.** Either use an update the hardware performs as one indivisible step, or give each thread a private tally and combine the tallies with a tree.

Check yourself

Classify each task: (a) turning a color image to grayscale, (b) finding the largest value in an array, (c) computing the 1,000th Fibonacci number, (d) building a brightness histogram of an image.

Answer on page 83.

Key terms

Dependency: a result that another piece of work needs first.

Map: the same operation on every item independently.

Stencil: an output computed from a neighbourhood of inputs.

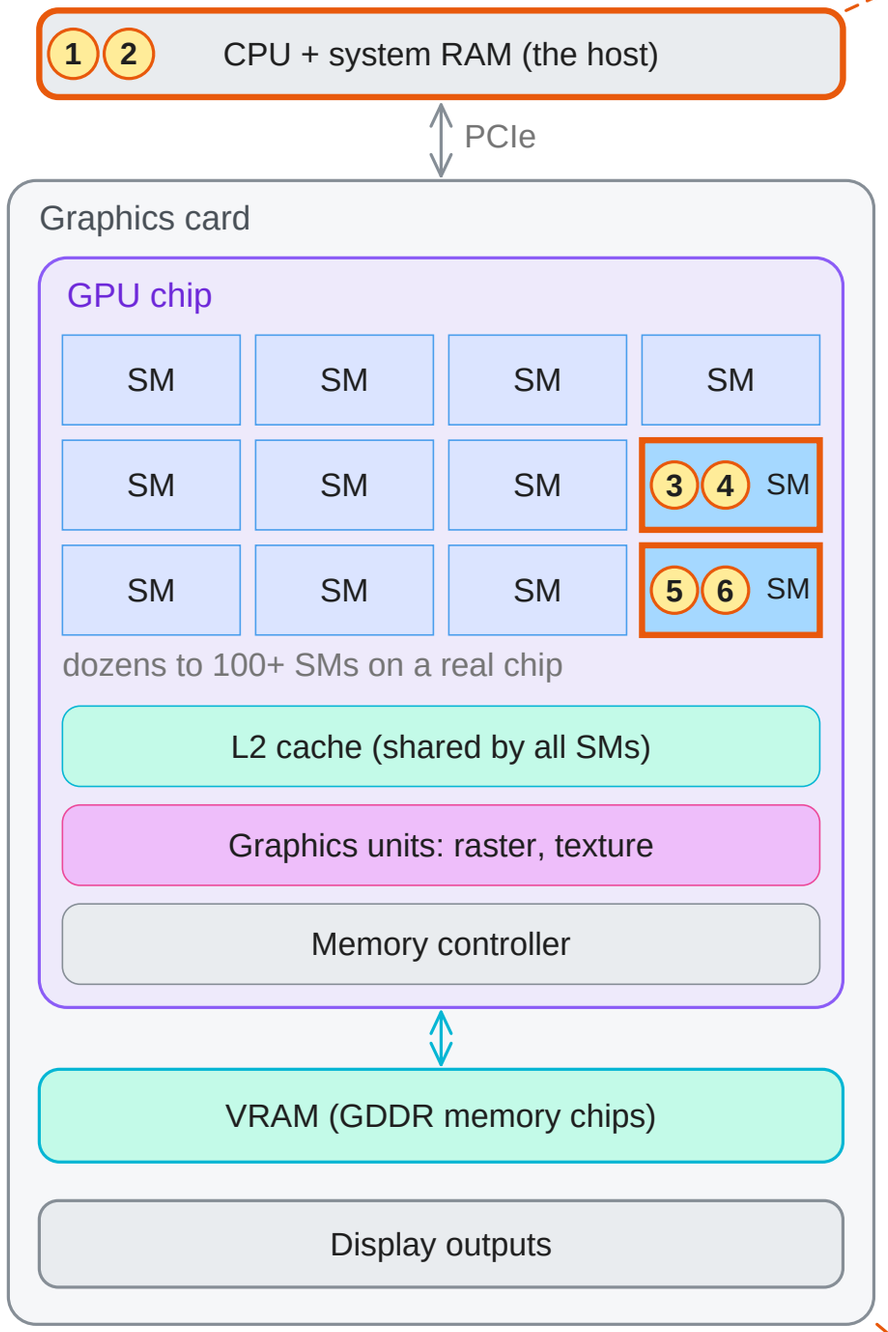
Reduction: combining many values into one, such as a sum.

Race: threads writing the same place with no fixed order.

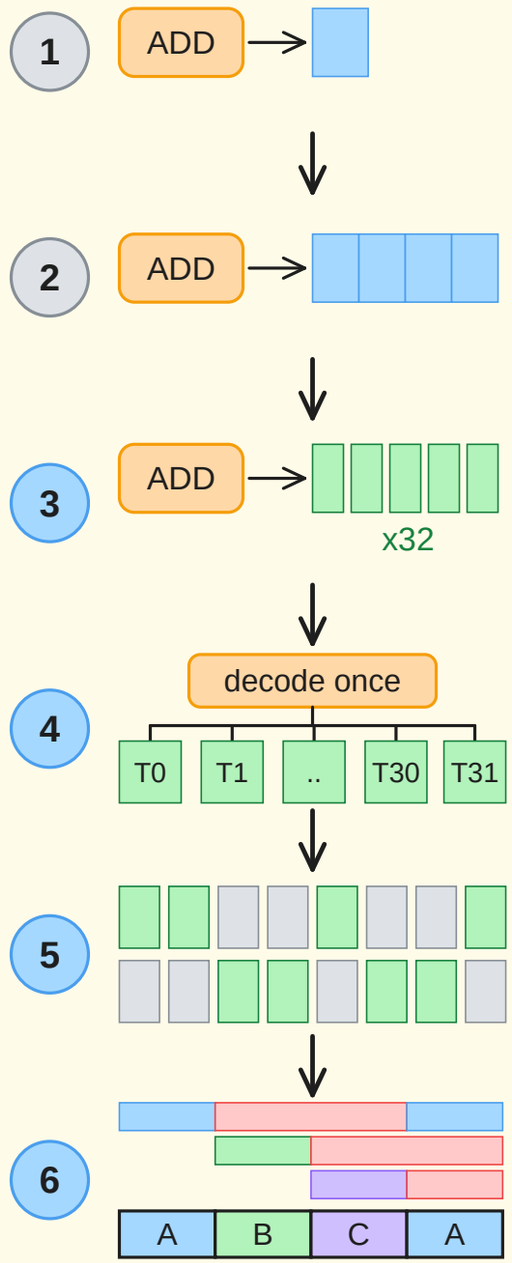
Lesson 2: One instruction, many threads

From a CPU running one value at a time to a GPU running 32 threads per instruction, and what that costs.

The whole GPU (map)



Zoomed in: from one value to a warp



Scalar: one instruction, one value

A classic CPU core runs one instruction on one piece of data at a time. Simple, flexible, and the baseline for the rest.

SIMD: one instruction, a few values

CPU vector units apply one instruction to 4-16 values packed in a wide register. You or the compiler must arrange the data.

SIMT: one instruction, many threads

A GPU applies one instruction to 32 full threads, each with its own registers. You just write ordinary code for one thread.

The warp: 32 threads, one counter

Those 32 threads form a warp. It shares one program counter, so each instruction is fetched and decoded once for all 32.

Branch divergence

At an if/else the warp cannot split. It runs the if-lanes, then the else-lanes, with the others masked off. Both cost time.

Latency hiding

When a warp waits on memory, the SM runs another warp that is ready. The core stays busy instead of stalling.

Key idea: threads are grouped into warps that share one instruction stream

Lesson 2 notes

One instruction, many threads

In brief

A GPU does not run its thousands of threads one by one. It groups them into warps of 32 that execute the same instruction at the same moment, each thread on its own data. That sharing is what lets the control logic stay small.

Beyond the board

- **SIMT is SIMD hardware with a threading illusion on top.** Underneath, a warp executes much like a 32-lane vector instruction. The difference is that the hardware tracks which lanes are active and handles branches for you, so you write ordinary per-thread code instead of vector code.
- **Divergence is a performance problem, not a correctness problem.** Your code always produces the right result. It just runs slower when threads in the same warp disagree about which way to go.
- **Latency hiding replaces prediction.** A CPU stalls or guesses when data is slow to arrive. A GPU simply runs a different warp, which is why it can do without heavy prediction and reordering machinery.
- **Vendor names.** NVIDIA calls the group a warp (32 threads). AMD calls it a wavefront (32 or 64).

Check yourself

A warp hits an if/else where 31 threads take the if and only 1 takes the else. Roughly how much of the warp's compute capacity is wasted while that single thread runs its else branch?

Answer on page 83.

Key terms

SIMD: one instruction applied to several values in a wide register.

SIMT: one instruction applied to many threads at once.

Warp: 32 threads that share one program counter.

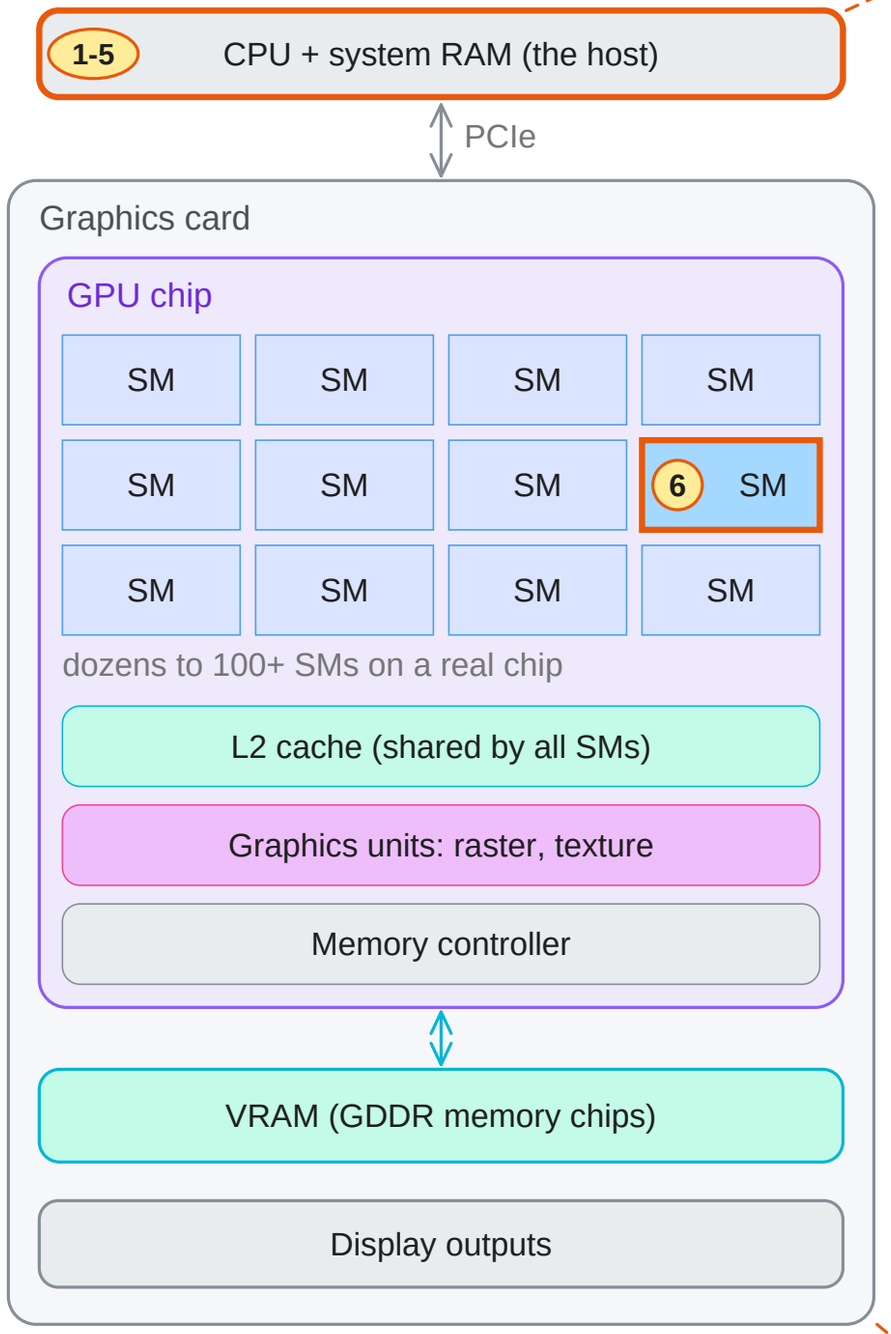
Divergence: threads of a warp taking different branches.

Latency hiding: running another warp while one waits.

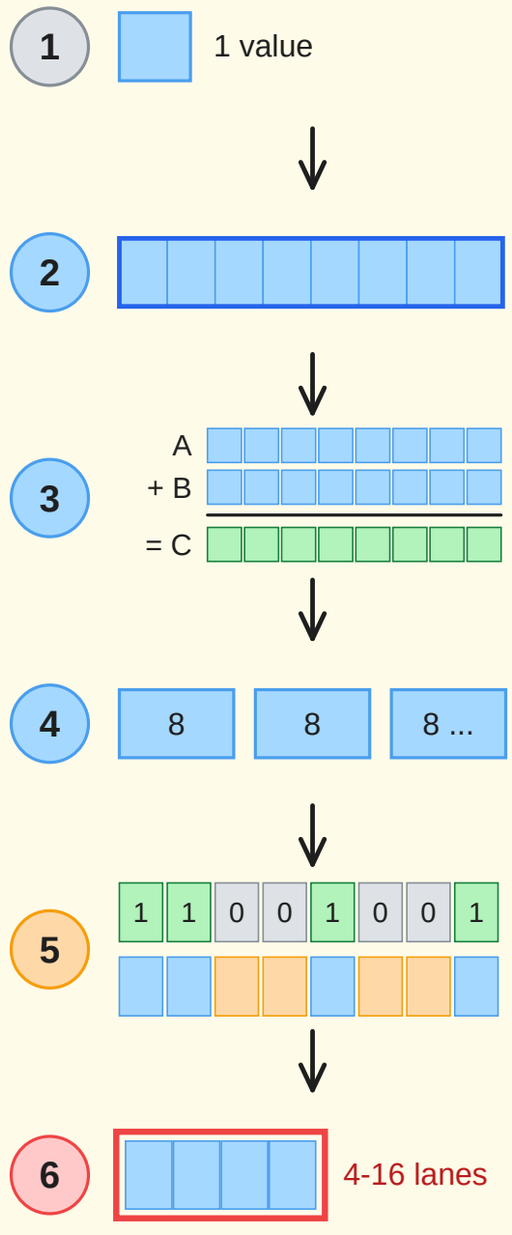
Lesson 2.1: SIMD up close

How a CPU applies one instruction to several values at once, and why that idea leads to the GPU's warp.

The whole GPU (map)



Zoomed in: from one value to a vector



A normal register holds one value

Scalar code works on one number at a time: load one, add one, store one. A loop of 1,000 items takes 1,000 adds.

A vector register holds several

Same idea, but wider. A 256-bit register holds 8 numbers of 32 bits each, side by side. Each slot is called a lane.

One instruction works on every lane

A vector add takes lane 0 of A plus lane 0 of B, lane 1 plus lane 1, and so on, all in the same cycle. Lanes do not interact.

Loops turn into chunks

1,000 items become 125 steps of 8. The compiler can often do this rewrite for you. It is called auto-vectorization.

Branches turn into masks

Lanes cannot jump separately. So both sides of an if are computed, and a mask of 1s and 0s picks a result per lane.

Where SIMD runs out

The lane count is fixed by the chip, data must sit side by side in memory, and complex code resists it. Next: SIMT.

Key idea: SIMD packs several values into one wide register and applies each instruction to all of them

Lesson 2.1 notes

SIMD up close

In brief

SIMD is the CPU's version of data parallelism. A wide register holds several values in side-by-side lanes, and one instruction operates on every lane in the same cycle. It is fast but rigid, and its limits are the reason GPUs went a step further.

Beyond the board

- **The widths you will see.** On x86 chips, SSE registers are 128 bits (4 lanes of 32-bit numbers), AVX registers are 256 bits (8 lanes) and AVX-512 registers are 512 bits (16 lanes). Arm's NEON is 128 bits.
- **Lanes can be cut differently.** The same 256-bit register can hold 8 numbers of 32 bits, 4 of 64 bits or 32 of 8 bits. The instruction says which.
- **The speedup is usually below the lane count.** 8 lanes rarely means 8 times faster, because the loop is often waiting on memory rather than on arithmetic.
- **Step 5 is where divergence comes from.** Computing both sides of an if and selecting with a mask is exactly what a warp does at a branch. A GPU just manages the mask for you.

Check yourself

You add two arrays of 1,000 numbers using 512-bit registers and 32-bit values. How many lanes is that, how many full steps does the loop take, and what happens to the leftovers?

Answer on page 83.

Key terms

Vector register: a wide register holding several values.

Lane: one slot of a vector register.

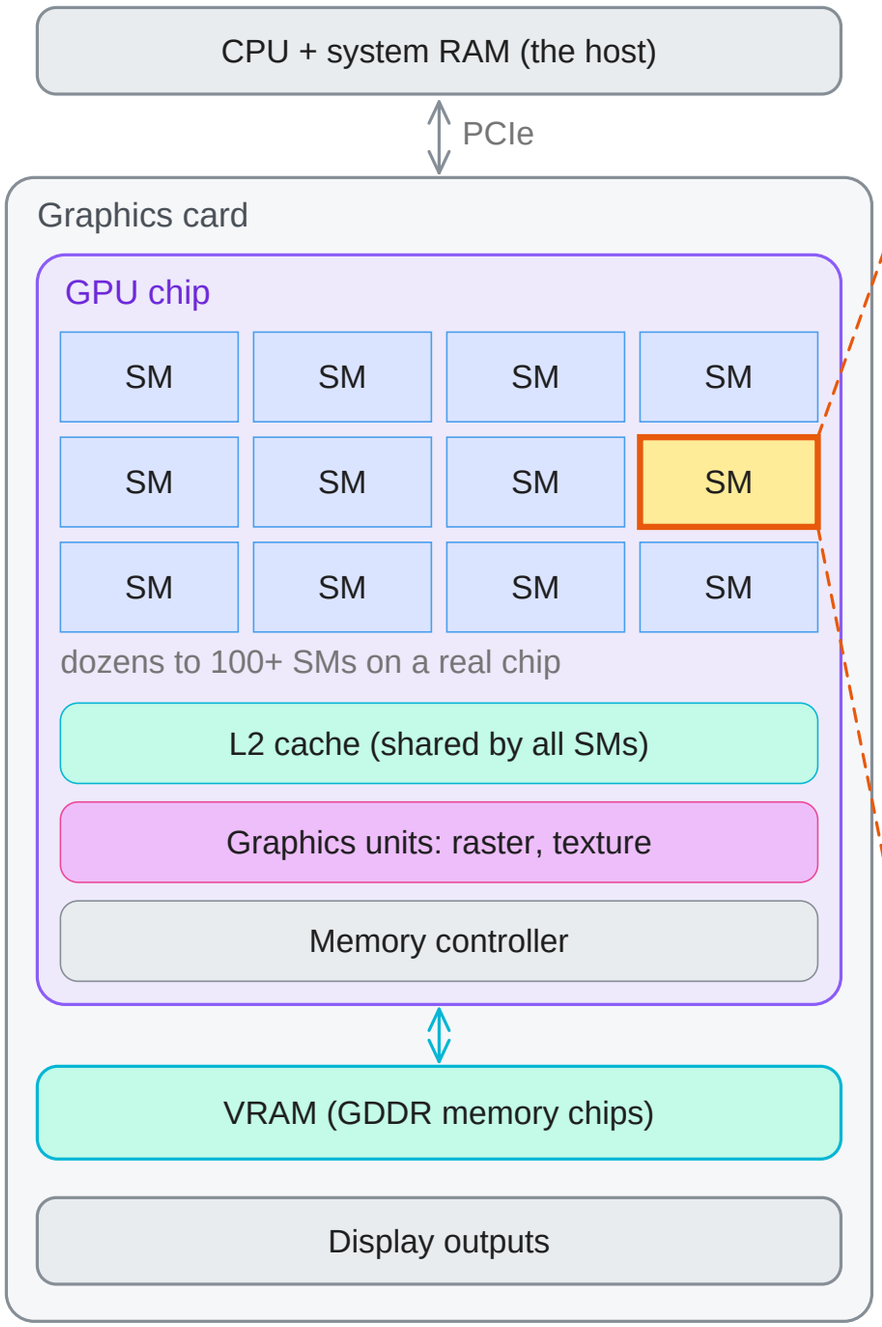
Mask: one bit per lane saying whether it takes part.

Auto-vectorization: the compiler turning a loop into SIMD code.

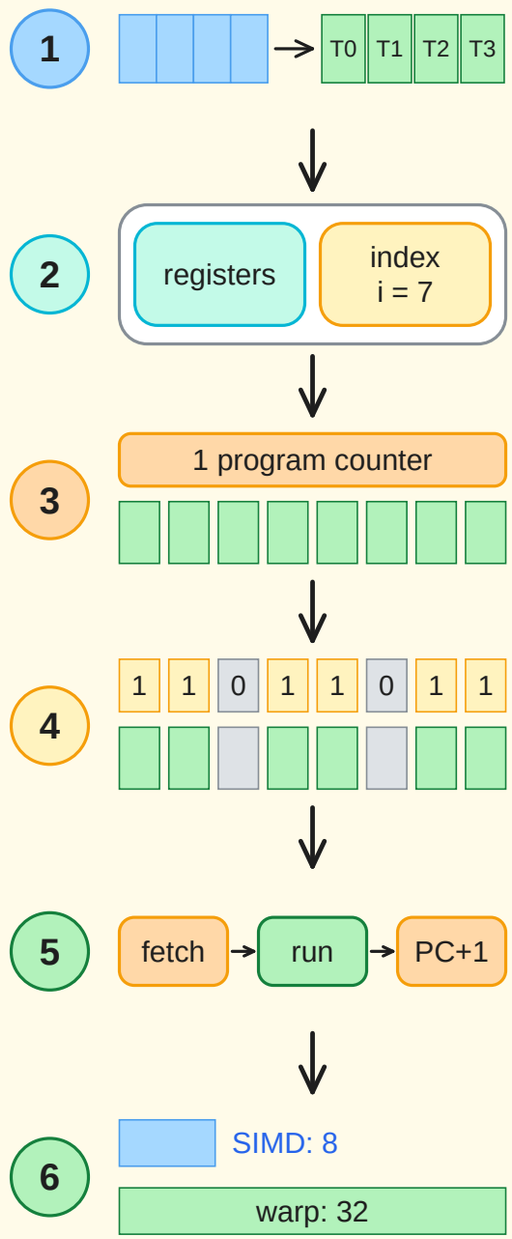
Lesson 2.2: Inside a warp

What each of the 32 threads keeps for itself, what they all share, and the mask that says who is taking part.

The whole GPU (map)



Zoomed in: one warp, 32 threads



Each lane becomes a whole thread

SIMD lanes are slots in one register. A GPU promotes each slot to a thread with an identity and state of its own.

What each thread owns

Its own registers, its own index (which element am I?), and its own addresses: each thread can read a different place.

What the warp shares

One program counter for all 32, so one fetch and one decode per instruction. The threads always march in step.

The active mask

One bit per thread. A 1 means: take part in this instruction. A 0 means: sit this one out and keep your registers as is.

One step of a warp

Fetch the instruction at the PC. Run it in every lane whose mask bit is 1. Move the PC forward. Repeat.

Why 32 and not 8 or 1,000?

Wider warps share control across more threads, but waste more when threads diverge. 32 is the usual balance.

Key idea: a warp is 32 private sets of registers driven by one shared program counter and one mask

Lesson 2.2 notes

Inside a warp

In brief

A warp turns each SIMD lane into a full thread. Every thread keeps its own registers and its own index, so it can work on different data and read different addresses. The 32 threads share one program counter and one active mask, which is what keeps the control hardware small.

Beyond the board

- **Private state lives in the register file.** The registers of all 32 threads, for every warp on the SM, are held at once. That is why the register file is so large (Lesson 3.2).
- **The mask explains two things.** It is how an if/else is handled (Lesson 2.3), and it is why a block of 100 threads wastes lanes: its fourth warp has 4 mask bits on and 28 permanently off.
- **Your code never sees any of this.** The program counter and the mask belong to the hardware. You write code for one thread, as if it ran alone.
- **Other vendors, same idea.** AMD calls the group a wavefront and uses 32 or 64 threads.

Check yourself

A kernel uses 64 registers per thread. How many register slots does one warp need? How many warps fit in a register file of 65,536 slots?

Answer on page 83.

Key terms

Thread: one copy of the kernel, with its own registers.

Program counter (PC): the address of the next instruction.

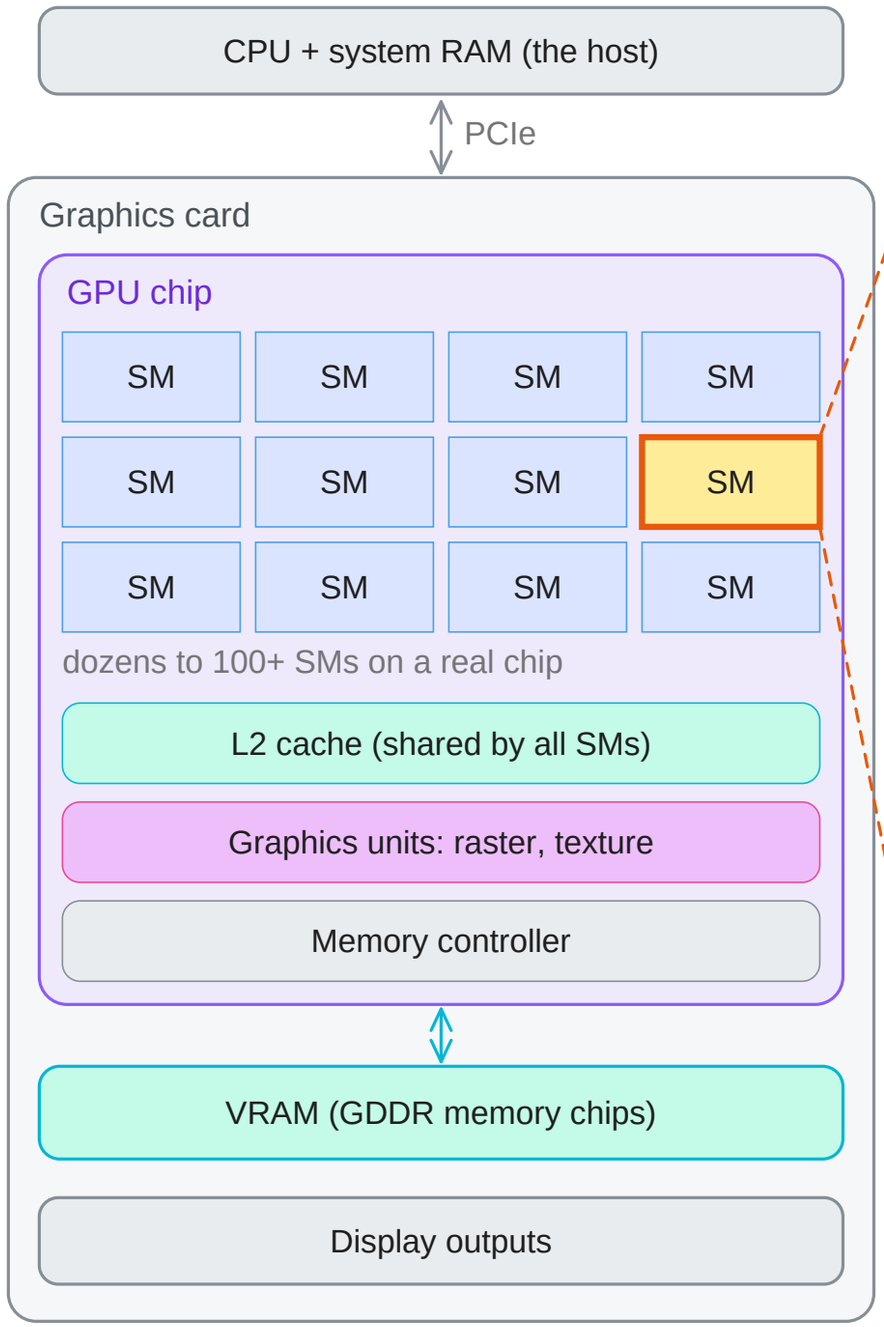
Active mask: one bit per thread: take part or sit out.

Warp: 32 threads sharing one program counter.

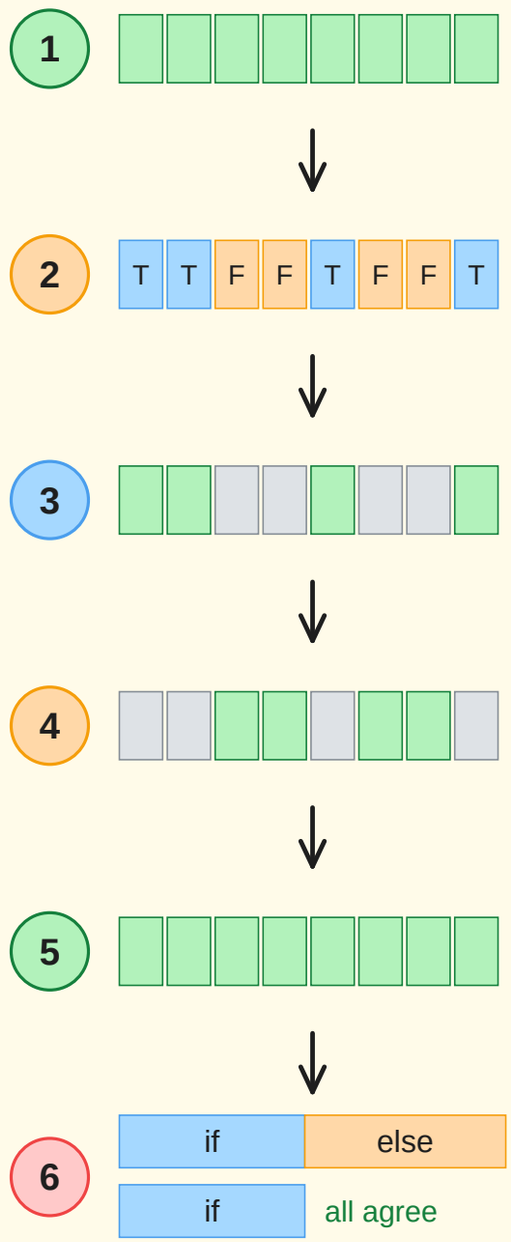
Lesson 2.3: Divergence, step by step

Follow one warp through: if (x > 0) y = sqrt(x); else y = 0; then z = y + 1;

The whole GPU (map)



Zoomed in: one warp meets an if/else



Before the branch: everyone runs

The mask is all 1s. Every thread runs the comparison x > 0 on its own value of x, in the same cycle.

The comparison splits the warp

Each thread gets its own answer, true or false. The warp now holds two groups that want to go different ways.

Pass 1: the if side

The mask keeps only the true threads. They run y = sqrt(x). The others wait, and the hardware remembers to come back.

Pass 2: the else side

The mask flips. Now the false threads run y = 0 while the first group waits. Same warp, same lanes, second pass.

Reconverge: the warp is whole again

At the first instruction after the if/else, the mask returns to all 1s. Every thread runs z = y + 1 together.

What it cost

Time = if side + else side, however the threads split. If all 32 agree, the other side is skipped and costs nothing.

Key idea: a diverged warp runs each side in turn with some of its lanes idle, then rejoins

Lesson 2.3 notes

Divergence, step by step

In brief

When the threads of a warp disagree at a branch, the warp cannot split. It runs the if side with the else threads masked off, then the else side with the if threads masked off, and rejoins afterwards. The result is correct; the cost is time spent with idle lanes.

Beyond the board

- **How the hardware keeps track.** Each warp has a small stack of masks and rejoin addresses. Entering a branch pushes an entry, and reaching the rejoin point pops it. Nested ifs push more.
- **Loops diverge too.** If the threads of a warp need different numbers of iterations, the warp keeps looping until the slowest thread is finished. The others sit masked off.
- **Short branches are often free.** For something as small as this example, the compiler usually computes both sides and selects one, with no branch at all.
- **The fix is grouping.** Divergence only hurts when threads in the same warp disagree. Sorting or grouping the data so similar items sit together makes each warp agree.

Check yourself

In a warp running a loop, 31 threads need 10 iterations and one needs 1,000. How many iterations does the warp run, and what share of its lane capacity is wasted?

Answer on page 83.

Key terms

Divergence: threads of one warp taking different paths.

Reconvergence: the point where the warp runs together again.

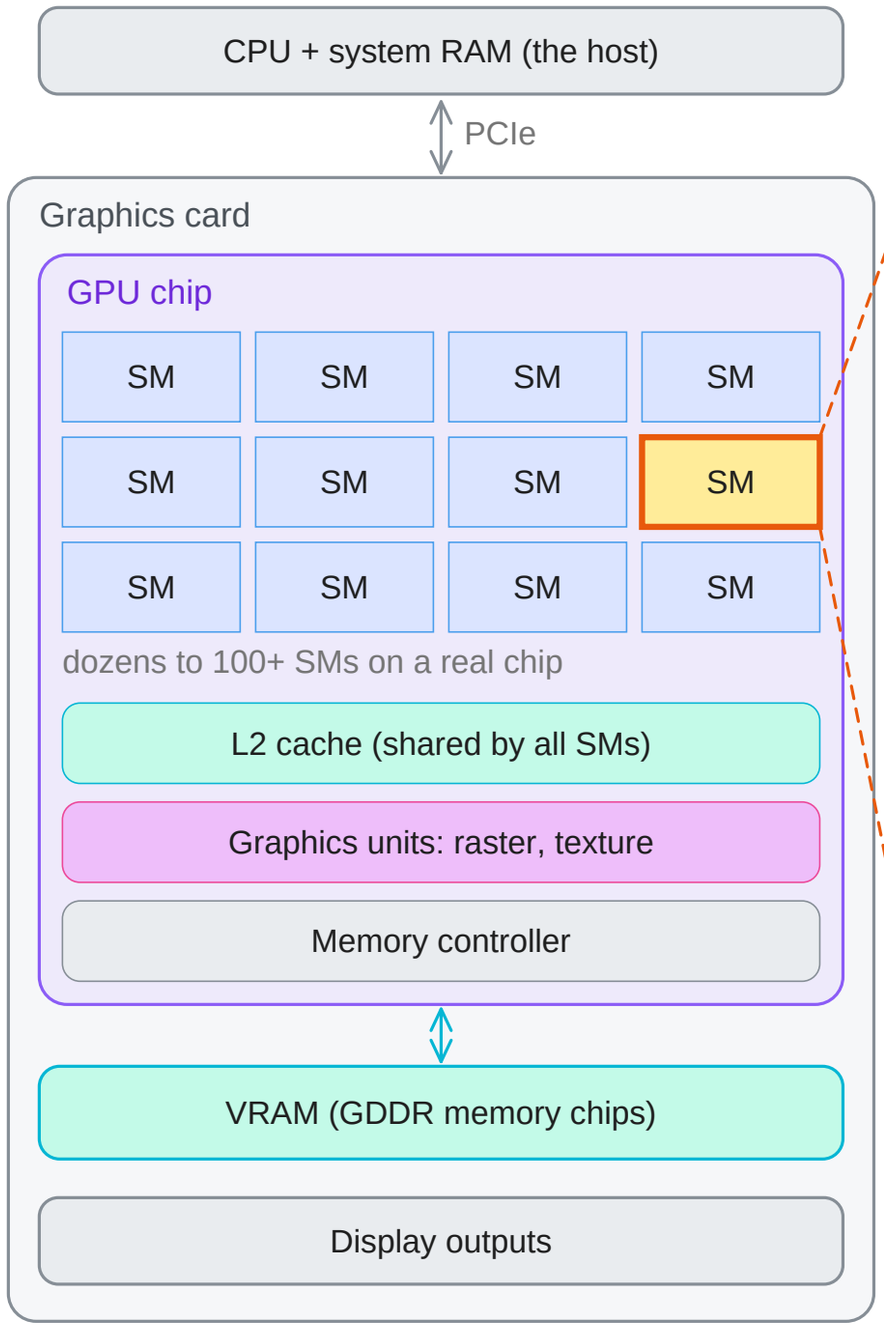
Mask stack: the warp's record of pending paths.

Select: computing both sides and picking one per lane.

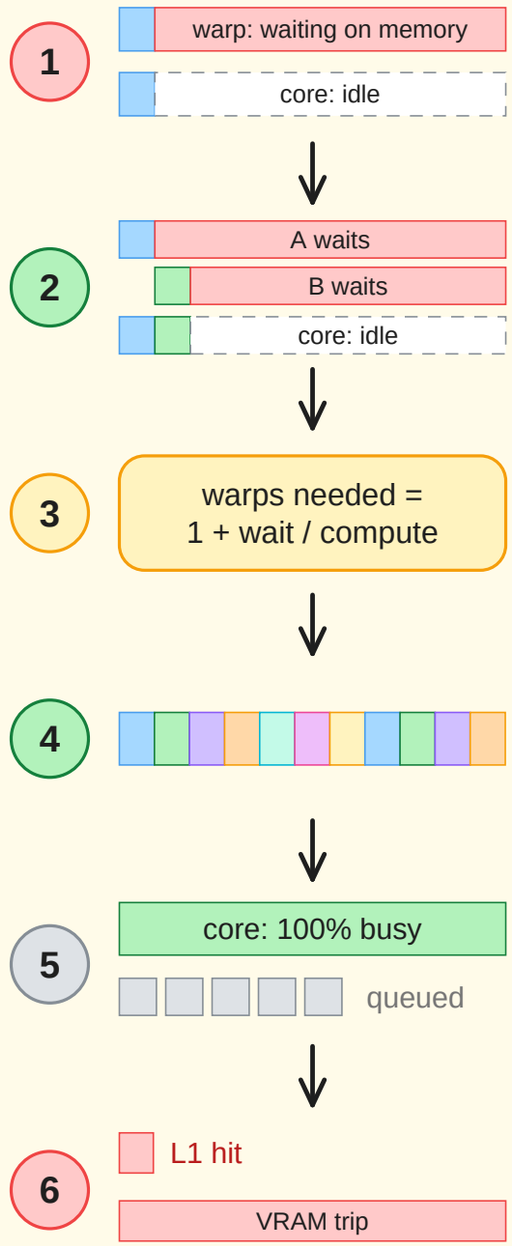
Lesson 2.4: Latency hiding with numbers

How many warps does an SM need on hand to stay busy? In the timelines, red means a warp is waiting on memory.

The whole GPU (map)



Zoomed in: one core's timeline



One warp alone: mostly waiting

Say a warp computes for 10 cycles, then waits 100 for memory. Alone on the core, it keeps the core busy 9% of the time.

A second warp fills part of the gap

While A waits, B runs its 10 cycles. The core is now busy 20 cycles in every 110, about 18%. Each warp added helps.

The rule of thumb

To cover a 100-cycle wait with 10-cycle bursts you need 1 + 100 / 10 = 11 warps: one running and ten waiting.

11 warps: the core never idles

By the time the 11th finishes its burst, the 1st has its data back. The core is busy 100% of the time. That is the goal.

More than enough: no extra speed

With 20 warps the core is still just full. The extras queue for a turn. They add safety margin but use register space.

What sets the two numbers

The wait depends on where data comes from: tens of cycles for L1, hundreds for VRAM. The burst depends on your code.

Key idea: a core stays busy when it holds about 1 + wait / compute warps, so slower memory needs more warps

Lesson 2.4 notes

Latency hiding with numbers

In brief

A warp alternates between short bursts of computing and long waits for memory. One warp alone leaves the core idle most of the time. With enough warps taking turns, the core is always busy. The number needed is about 1 + wait / compute.

Beyond the board

- **No warp got faster.** Each one still takes 110 cycles per round. What changed is throughput: the core now finishes 11 rounds in the time it used to finish one.
- **A CPU attacks the same problem differently.** It tries to make the wait itself shorter or rarer, with large caches and prediction. A GPU accepts the wait and fills it.
- **More warps cannot fix slow memory.** If memory can only deliver so many bytes per second, extra warps just queue for it. That is the memory-bound case in Lessons 4.3 and 7.3.
- **The model is simplified.** Real bursts and waits vary from one instruction to the next, and the waits depend on what the other warps are doing to the caches.

Check yourself

A kernel computes in bursts of 20 cycles and waits 300 cycles for memory. How many warps keep a core busy? If only 8 fit, how busy is the core?

Answer on page 83.

Key terms

Latency: how long one thing takes.

Throughput: how much gets done per second.

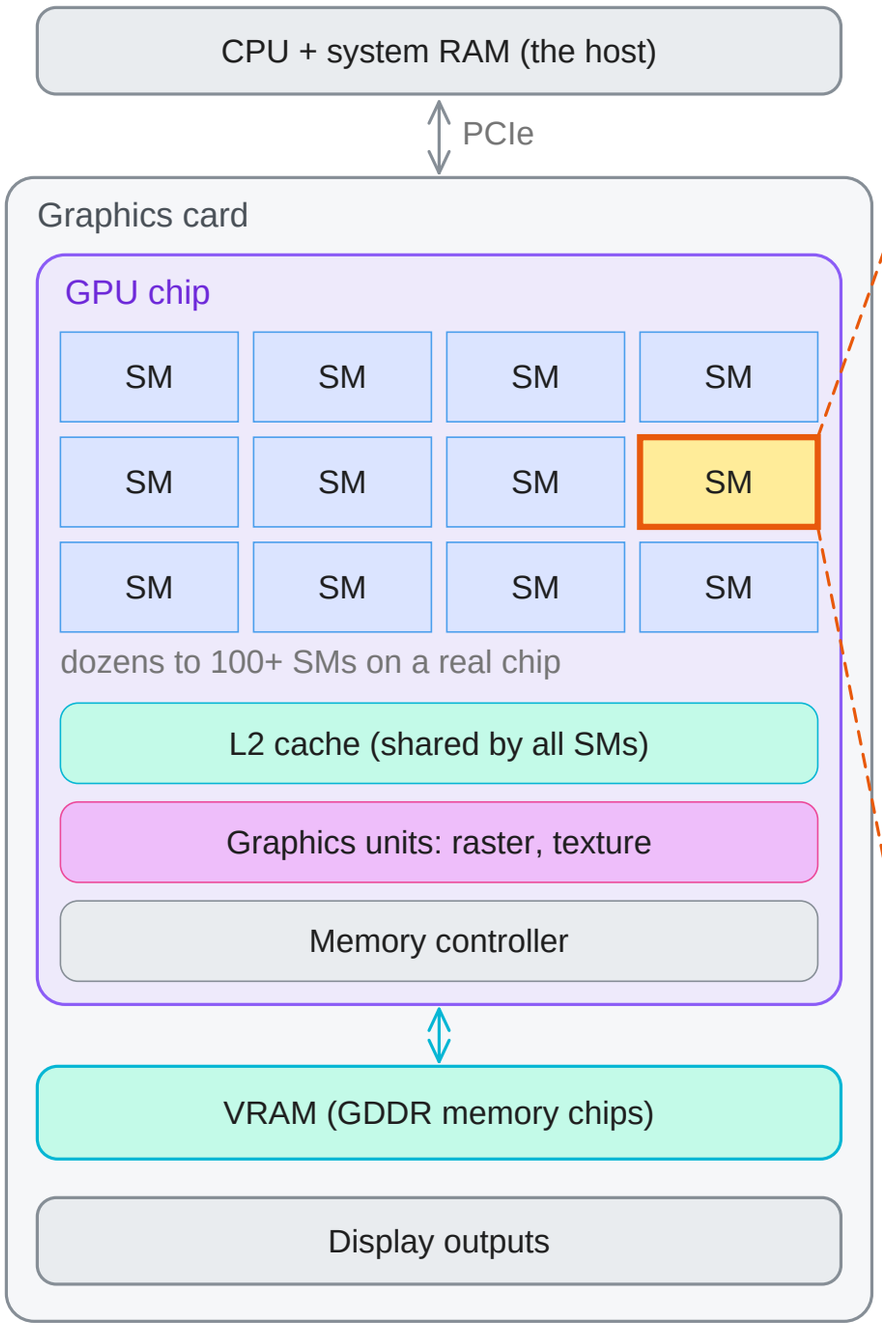
Latency hiding: running other warps during a wait.

Burst: the cycles a warp computes between two waits.

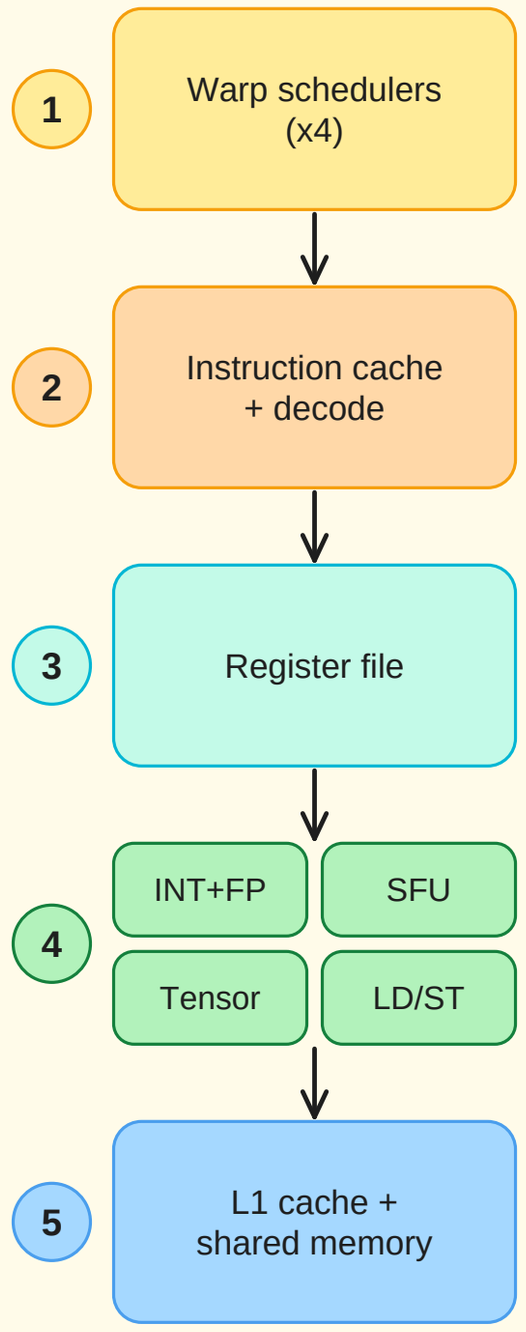
Lesson 3: Inside a GPU core

First the whole GPU, then zoom into the part this lesson covers: one streaming multiprocessor.

The whole GPU (map)



Zoomed in: one streaming multiprocessor (SM)



Pick a ready warp

Schedulers track each warp as ready or waiting. Every cycle they issue from ready ones. Switching is nearly free: no state is saved or restored.

Fetch and decode once

The warp's next instruction comes from the instruction cache and is decoded one time for all 32 threads. Shared control logic is why GPU cores are so small.

Read the registers

Each thread reads operands from its own registers. The file is a fixed pool, so more registers per thread means fewer warps fit on the SM (occupancy).

Execute in every lane

INT+FP units: everyday math, the 'cores'.
SFU: slow functions like sin and sqrt.
Tensor cores: small matrix multiplies.
LD/ST: compute addresses for memory.

Touch memory, or write back

Results return to registers. Loads and stores hit L1 / shared memory first, then L2 and VRAM (see map). The warp waits, and the scheduler goes back to step 1.

Key idea: schedulers keep simple execution units busy by always having another warp ready

Lesson 3 notes

Inside a GPU core

In brief

An SM is a few schedulers feeding simple execution units, backed by a very large register file and a small local memory. One instruction touches every part: pick a ready warp, fetch and decode once, read registers, execute in every lane, then touch memory or write back.

Beyond the board

- **Vendor naming.** "Streaming multiprocessor" is NVIDIA's term. AMD calls the equivalent block a compute unit, and Intel calls it an Xe-core. The internals differ in detail, but the same ingredients appear in all of them.
- **What a "core count" means.** When a spec sheet says a GPU has around 10,000 cores, it is counting the floating-point lanes across all SMs. Those lanes cannot run independent programs the way CPU cores can; they execute in warp-sized groups under a shared scheduler.
- **Why the register file is so large.** A CPU saves and restores registers when it switches threads. A GPU keeps every resident thread's registers live at once, which is what makes switching warps nearly free. The price is chip area and a hard limit on how many warps fit.
- **No prediction needed.** With dozens of warps resident, something is almost always ready to run, so the SM needs no branch predictor or reorder buffer.

Check yourself

Suppose you rewrite a kernel so each thread needs twice as many registers, but does the same amount of arithmetic. Why might it run slower?

Answer on page 83.

Key terms

Warp scheduler: picks a ready warp each cycle.

Register file: private registers for every resident thread.

SFU: special function unit, for sin, sqrt and similar.

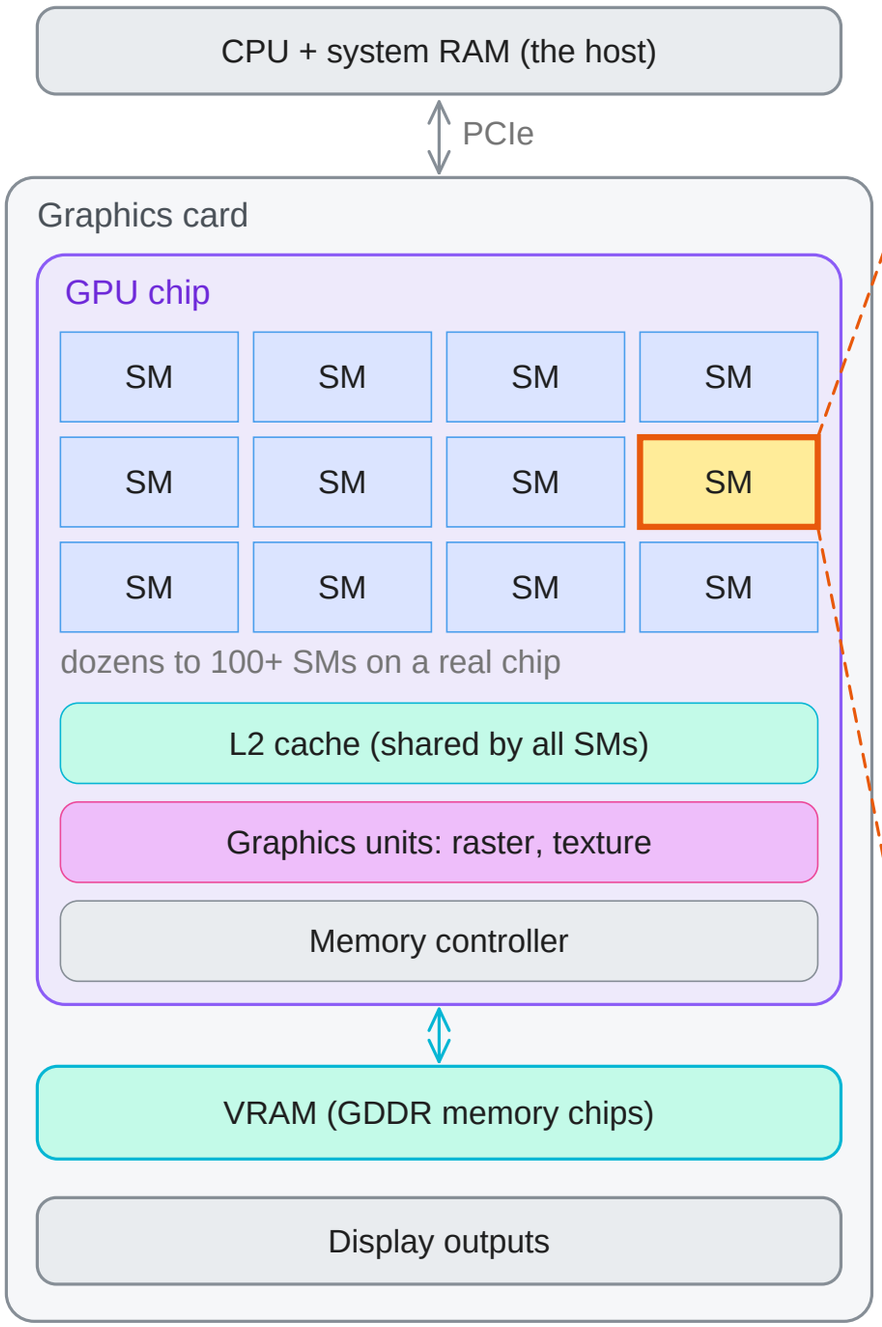
Tensor core: unit for small matrix multiply-accumulate.

Occupancy: warps resident on an SM versus the maximum.

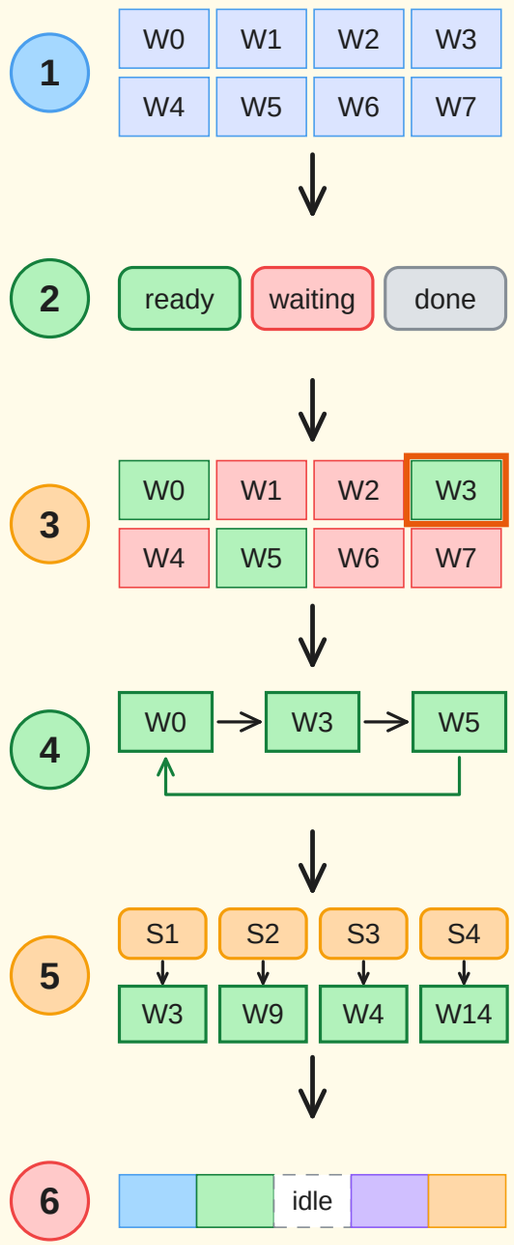
Lesson 3.1: Warp scheduling

How an SM keeps track of its warps and decides, every cycle, which one runs next.

The whole GPU (map)



Zoomed in: the warp scheduler



The SM keeps a list of its warps

Every warp living on the SM has an entry: where its program counter is, its mask, and what state it is in right now.

Three states

Ready: its next instruction can run now. Waiting: it needs data from memory or an earlier result. Done: it has finished.

Every cycle: pick one ready warp

The scheduler looks only at the green ones, picks one, and sends its next instruction to the execution units.

How it picks: simple and fair

The choice must be made in under a cycle, so the rule is simple, such as taking turns among the ready warps.

Several schedulers work side by side

Recent NVIDIA SMs have four. Each has its own share of warps and execution units, so four warps issue every cycle.

No ready warp means a lost cycle

If every warp is waiting, the units sit idle for that cycle. Keeping enough warps on hand prevents this (Lesson 2.4).

Key idea: each cycle the scheduler issues one instruction from a ready warp, and switching warps costs nothing

Lesson 3.1 notes

Warp scheduling

In brief

The scheduler is what turns many waiting warps into one busy core. It keeps a list of the warps living on the SM with a state for each. Every cycle it picks one that is ready and issues its next instruction. Recent SMs have four schedulers doing this side by side.

Beyond the board

- **Why switching is free.** A CPU that switches threads must save one set of registers and load another, which costs thousands of cycles. On an SM every warp's registers stay where they are in the register file, so the scheduler simply points at a different warp.
- **Waits come in two sizes.** A memory wait lasts tens to hundreds of cycles. A warp also waits a few cycles for its own previous result (Lesson 3.4). The scheduler fills both kinds.
- **The exact rule is not public.** Vendors do not document how a ready warp is chosen. Published models use round-robin, or keep issuing from one warp until it stalls and then take the oldest.
- **A fourth reason to wait.** A warp that reaches a block's sync point waits there until the other warps of its block arrive (Lesson 6.2).

Check yourself

An SM has 4 schedulers and 16 warps, 4 per scheduler. This cycle 3 warps are ready. At most how many instructions are issued? What if all 3 belong to the same scheduler?

Answer on page 84.

Key terms

Ready: the warp's next instruction can run now.

Issue: sending one instruction to the execution units.

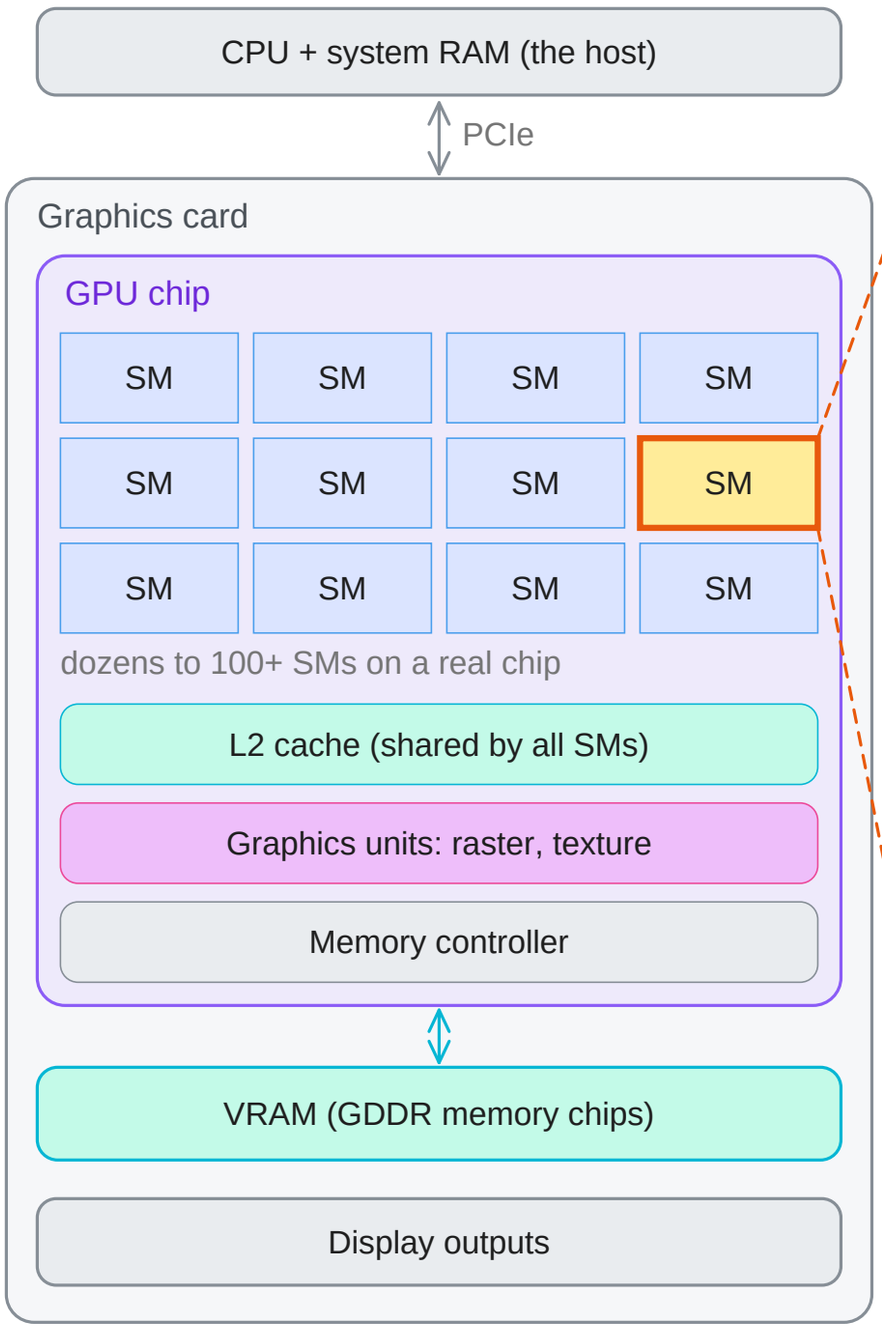
Round-robin: taking turns in a fixed order.

Context switch: changing which thread a core is running.

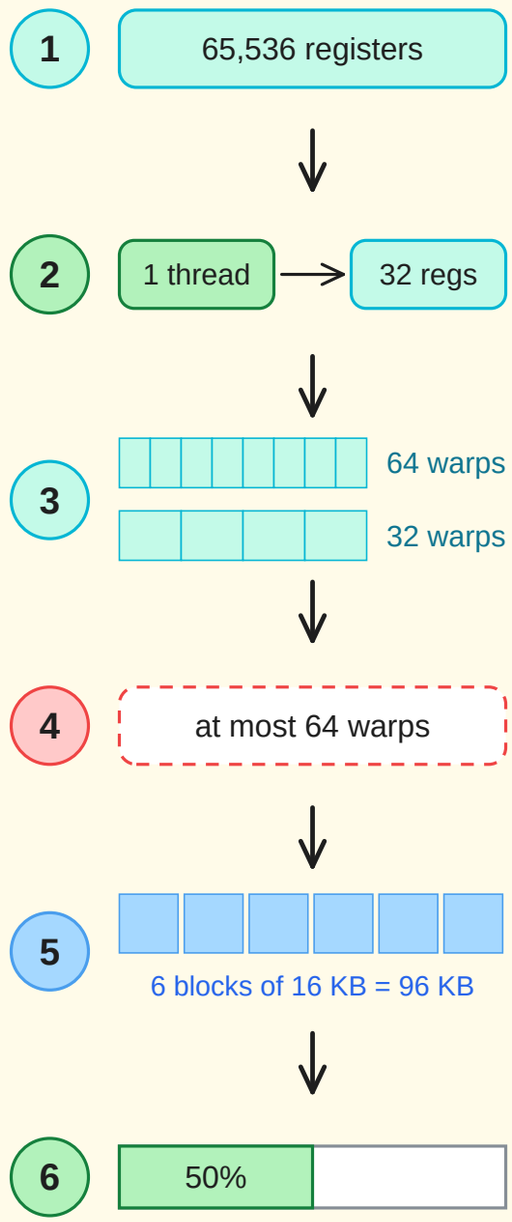
Lesson 3.2: The register file and occupancy

Working out how many warps actually fit on one SM. Three limits apply, and the smallest one wins.

The whole GPU (map)



Zoomed in: what limits the warps on an SM



One pool of registers per SM

A typical NVIDIA SM has 65,536 registers. Every thread living on the SM takes its registers from this one fixed pool.

Each thread takes a fixed share

The compiler decides how many registers a kernel needs per thread, say 32. Every thread of that kernel gets exactly that.

Limit 1: how many warps fit

A warp is 32 threads, so at 32 registers each: 65,536 / (32 x 32) = 64 warps. At 64 registers each, only 32 warps.

Limit 2: a hard cap on warps

The scheduler's list has a fixed length, for example 64 warps per SM. Using fewer registers cannot push past this cap.

Limit 3: shared memory per block

Each block reserves its scratchpad space. With 96 KB on the SM and 16 KB per block, only 6 blocks fit at the same time.

Occupancy: the tightest limit wins

Occupancy = warps resident / the cap. 32 warps out of 64 is 50%. Whichever limit is smallest sets the number.

Key idea: occupancy is set by registers per thread, the warp cap, or shared memory per block, whichever is tightest

Lesson 3.2 notes

The register file and occupancy

In brief

Latency hiding needs enough warps on the SM. How many fit is set by three limits: the fixed pool of registers, a hard cap on the scheduler's list, and the shared memory each block reserves. Occupancy is the number of warps that fit, divided by the cap.

Beyond the board

- **An SM takes whole blocks.** Blocks are never split across SMs. If the registers allow 6.4 blocks, 6 fit and the rest of the space goes unused.
- **More registers per thread is not a mistake.** Complex code keeps more values alive at once and needs them. A compiler setting can cap the count, at the price of extra memory traffic (Lesson 7.2).
- **100% is not the target.** The goal is enough warps to keep the schedulers busy, which is often well below full occupancy.
- **The figures vary by generation.** The warp cap has been 32 to 64, and shared memory per SM has ranged from about 48 KB to over 100 KB.

Check yourself

A kernel uses 40 registers per thread, blocks of 256 threads and 12 KB of shared memory per block. The SM has 65,536 registers, a cap of 64 warps and 96 KB of shared memory. How many blocks fit, which limit decides, and what is the occupancy?

Answer on page 84.

Key terms

Register file: the SM's fixed pool of registers.

Resident: currently living on the SM.

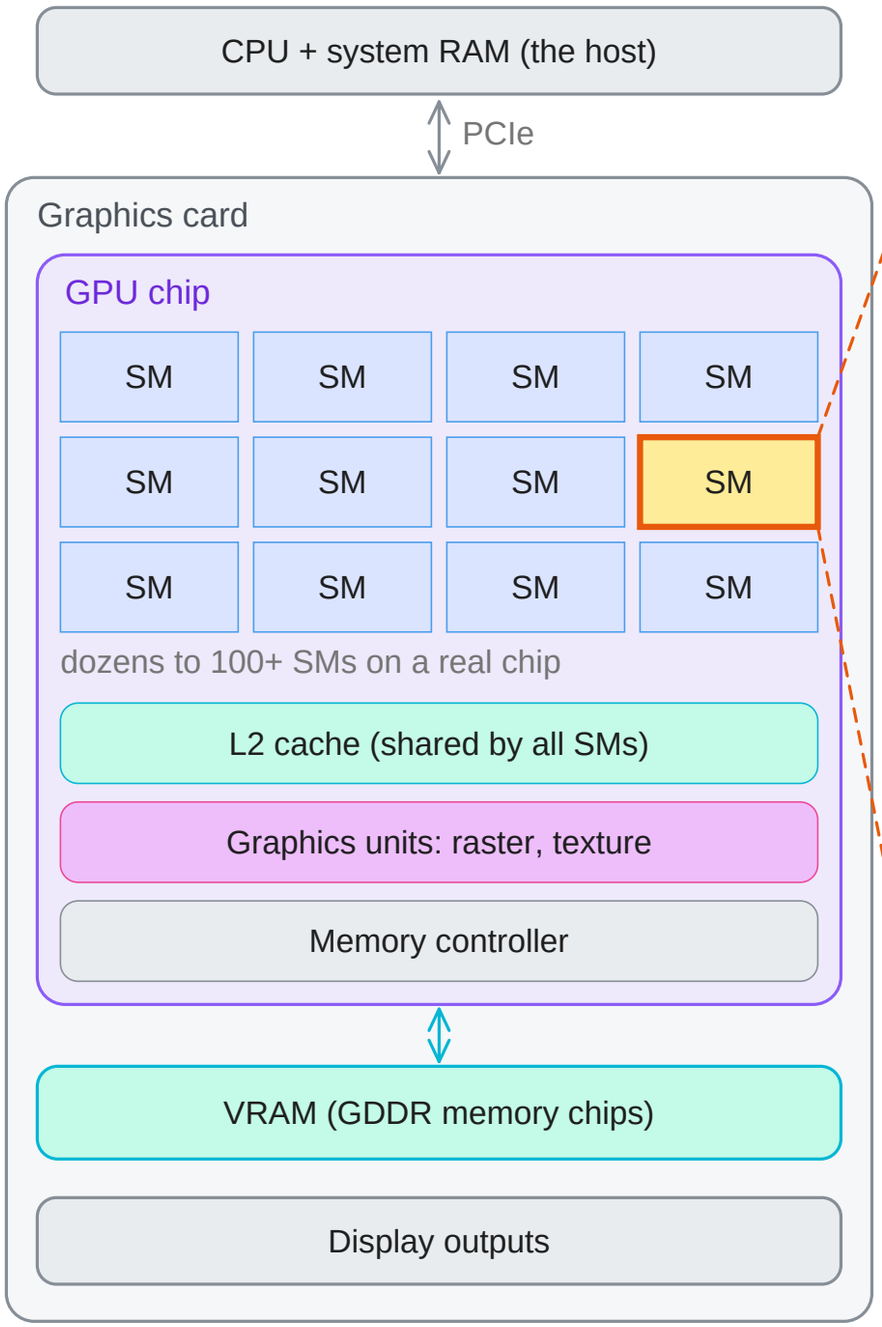
Occupancy: resident warps divided by the warp cap.

Shared memory: per-block scratchpad inside the SM.

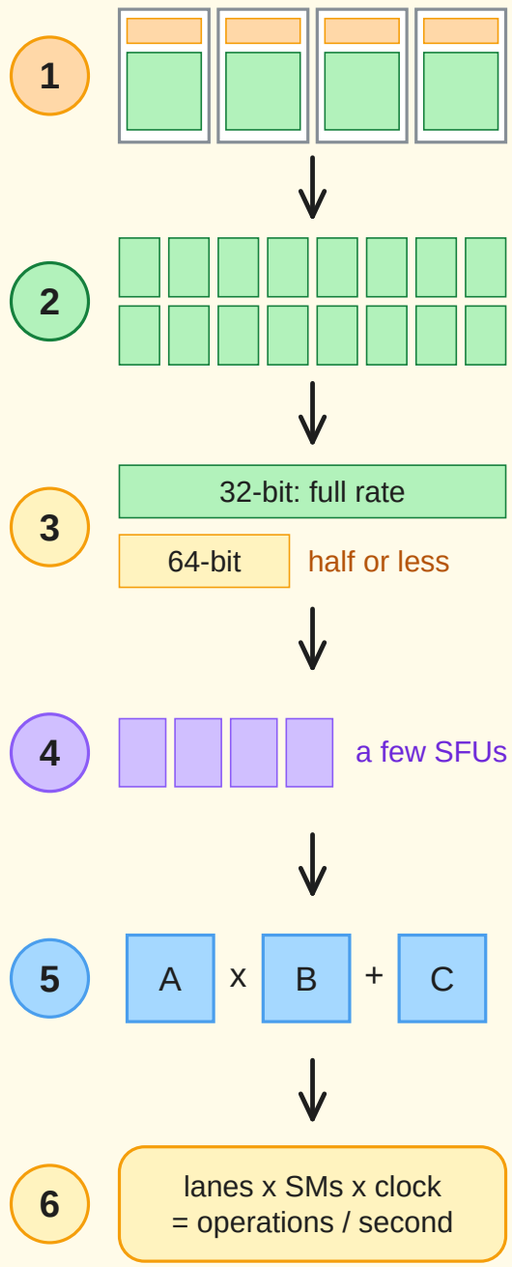
Lesson 3.3: Execution units and throughput

The units that do the actual math: what each kind is for, how many there are, and how fast they go.

The whole GPU (map)



Zoomed in: the execution units of one SM



An SM is four small cores side by side

Recent NVIDIA SMs have four partitions. Each has its own scheduler (orange) and its own set of execution units (green).

Everyday math: FP32 and integer lanes

These add, multiply and compare. A partition may have 16 lanes, so a warp of 32 goes through as two halves.

Double precision costs more

64-bit math needs bigger units, so chips have fewer of them. It runs at half speed on some GPUs and far less on others.

Special functions are slower still

sin, cos, square root and 1/x use a few special function units (SFUs) and take several cycles per result.

Tensor cores: matrix math in bulk

One unit multiplies and adds a small block of numbers per step. Far faster than the lanes, but only for matrices.

Adding up the throughput

Peak rate = lanes per SM x number of SMs x clock speed. The mix of operations in your code decides how close you get.

Key idea: an SM has many cheap lanes for everyday math and a few special units, so the mix of operations sets the speed

Lesson 3.3 notes

Execution units and throughput

In brief

An SM's execution units are not all alike. Most are simple lanes for 32-bit arithmetic. There are fewer units for double precision, a handful for functions like sine and square root, and tensor cores for matrix work. How fast a kernel runs depends on which it uses.

Beyond the board

- **A worked example.** NVIDIA's V100 has 80 SMs with 64 FP32 lanes each. That is the 5,120 "cores" on its spec sheet. At about 1.5 GHz, with each lane doing a multiply and an add per cycle, the peak is roughly 15 trillion operations per second.
- **Why a multiply and an add count as two.** A fused multiply-add computes a x b + c as one instruction. It is the most common step in numerical code, so peak rates assume it.
- **Double precision depends on the product.** Data-center GPUs run it at half the single-precision rate. Many consumer cards run it at a small fraction of that.
- **Lane counts vary.** The number of lanes per partition differs between generations, and some designs process a warp in one cycle rather than two.

Check yourself

A GPU has 100 SMs with 128 lanes each and runs at 2 GHz, with every lane doing a multiply and an add per cycle. What is the peak rate? Give two reasons a real program falls short of it.

Answer on page 84.

Key terms

Lane: one arithmetic unit, handling one thread's value.

FP32 / FP64: 32-bit and 64-bit floating-point numbers.

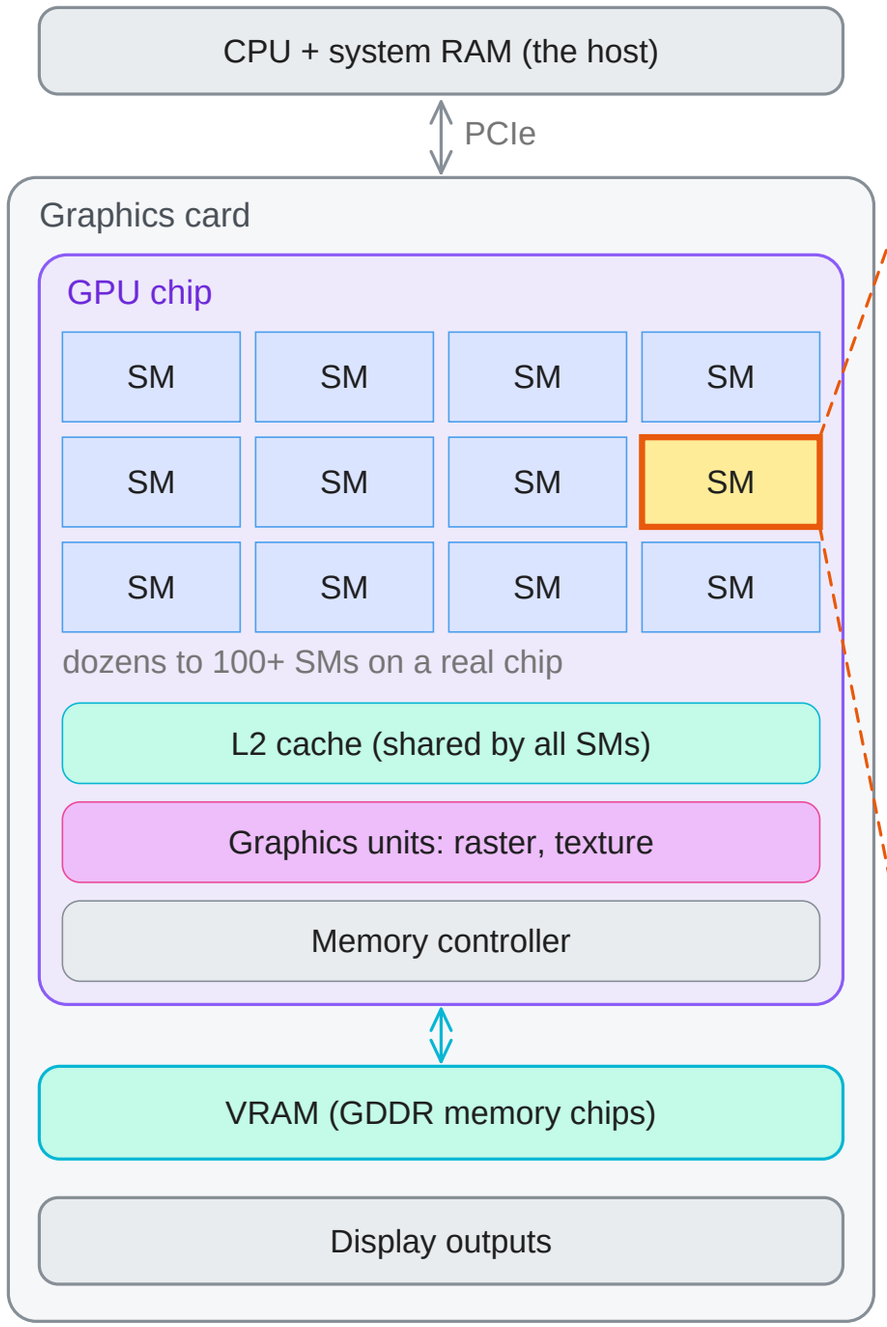
SFU: special function unit, for sin, sqrt and similar.

Fused multiply-add: a x b + c as a single instruction.

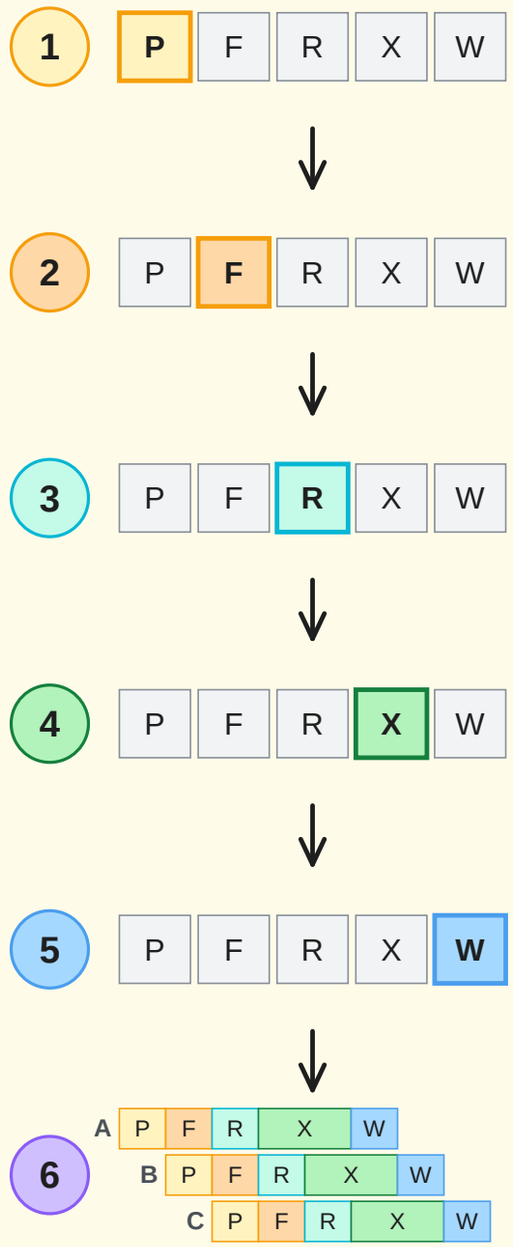
Lesson 3.4: One instruction's timeline

Follow a single instruction (multiply r1 by r2, then add r3) through the SM, cycle by cycle.

The whole GPU (map)



Zoomed in: six cycles in the life of an instruction



Cycle 1: pick (P)

The scheduler checks its list, finds warp A ready, and selects it. Nothing has been read or computed yet.

Cycle 2: fetch and decode (F)

The instruction at A's program counter is read from the instruction cache and decoded once: multiply r1 by r2, add r3.

Cycle 3: read registers (R)

Each active thread's r1, r2 and r3 are read from the register file: 32 threads, 3 values each, 96 values at once.

Cycles 4-5: execute (X)

The lanes compute. With 16 lanes in the partition, threads 0-15 go first and 16-31 follow one cycle later.

Cycle 6: write back (W)

Results land in each thread's own result register. Only now is warp A ready for an instruction that needs them.

The big picture: stages overlap

While A executes, B reads registers and C is being decoded. A new instruction starts every cycle, one warp after another.

Key idea: each instruction takes several cycles, but the SM starts a new one every cycle by overlapping warps

Lesson 3.4 notes

One instruction's timeline

In brief

One instruction is not finished in one cycle. It is picked, fetched and decoded, its registers are read, the lanes execute it, and the results are written back. Because each stage hands on to the next, the SM can start another warp's instruction every cycle.

Beyond the board

- **The stage count is illustrative.** Real SMs have more stages than five, and the details are not published. What matters is that the stages exist and overlap.
- **This is the short wait from Lesson 3.1.** A warp cannot issue an instruction that needs a result still in the pipeline. Other warps fill those cycles.
- **A load is different.** In the execute stage a load only computes its address. The data then has to come from a cache or from VRAM, and the warp waits for it.
- **Masked-off threads ride along.** Their lanes do nothing and their registers are left untouched, but the instruction takes just as long.

Check yourself

Suppose every instruction of a kernel needs the result of the one before it. Using the six-cycle timeline, about how many instructions finish in 60 cycles with 1 warp on the core? With 6 warps?

Answer on page 84.

Key terms

Pipeline stage: one step in processing an instruction.

Decode: working out what an instruction asks for.

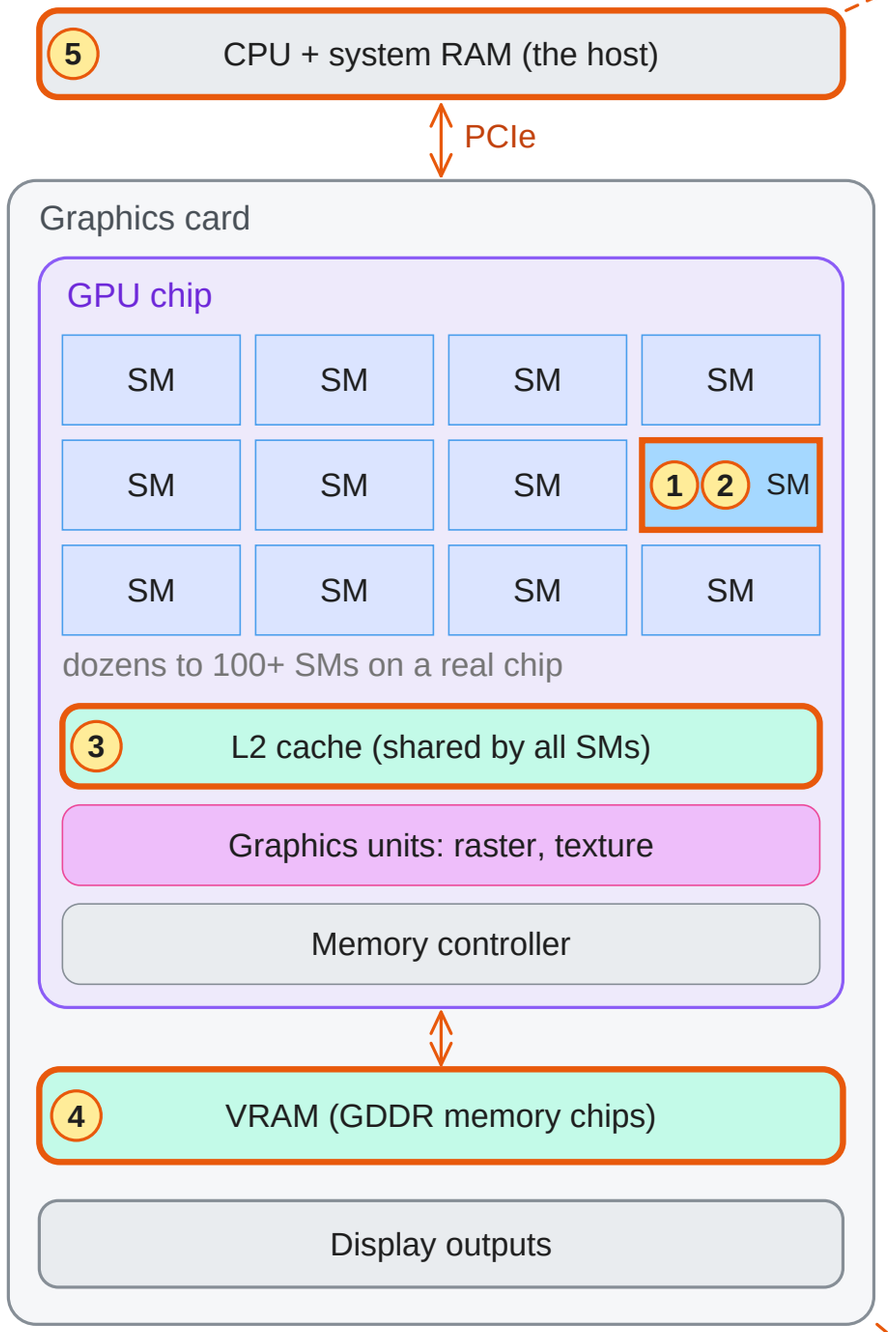
Write back: storing a result in its register.

Instruction cache: where the SM keeps the kernel's code.

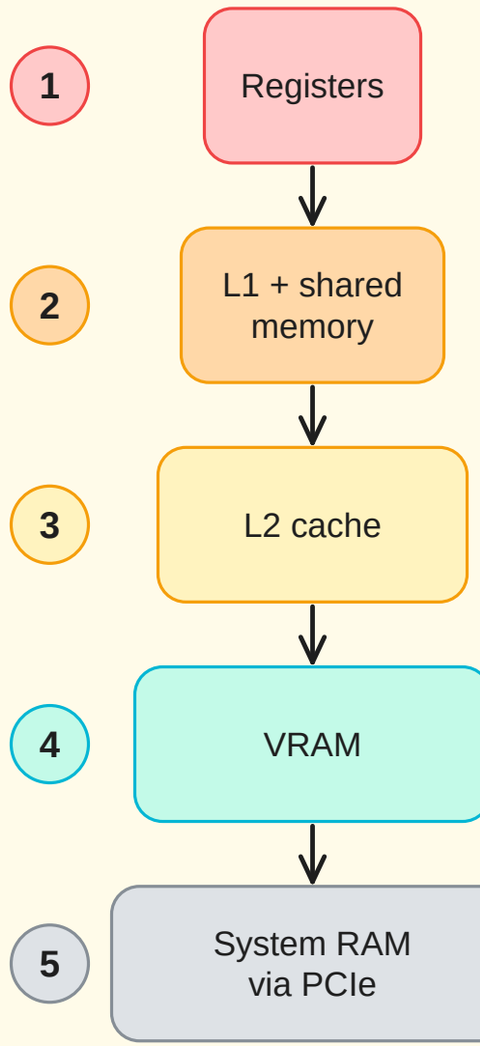
Lesson 4: The memory hierarchy

The whole GPU again, now highlighting everywhere data can live. Numbers on the map match the zoom.

The whole GPU (map)



Zoomed in: where data lives, fastest to slowest



Registers: private, instant

One set per thread, inside the SM. Read in about a cycle. Where all the math happens: data must reach a register to be used.

L1 cache + shared memory: per SM

Roughly 100-250 KB per SM, tens of cycles away. L1 fills itself. Shared memory is a scratchpad your code fills and reuses.

L2 cache: shared by the whole chip

A few MB to tens of MB for all SMs to share. Helps when data is reused, but it is tiny next to what the threads ask for.

VRAM: where your data actually lives

8-32 GB on the card (more in data centers). Hundreds of cycles away. Delivers hundreds of GB per second, and that rate is fixed.

System RAM: across the PCIe link

Tens of GB per second at best, far slower than VRAM. Copy data to the GPU once, then keep it there while the work runs.

Why bandwidth is the real bottleneck

A big GPU can run tens of trillions of operations per second, but VRAM hands over only a few hundred billion numbers per second. Simple code (like a + b) leaves the ALUs idle, waiting for data. Fast kernels do lots of math per number loaded and reuse data from shared memory.

Key idea: GPUs are usually limited by how fast data arrives, not by how fast they can compute

Lesson 4 notes

The memory hierarchy

In brief

Data lives at five distances from the arithmetic units, and each step outward is larger and slower: registers, L1 and shared memory inside the SM, the chip-wide L2 cache, VRAM on the card, and system RAM across the PCIe link. For most real programs the limit is how fast data arrives, not how fast the GPU computes.

Beyond the board

- **GPUs lean on bandwidth, not big caches.** A CPU gives each core megabytes of cache. A GPU gives each SM a small L1 and relies on a very wide path to VRAM, plus latency hiding, to cover the wait.
- **Access pattern matters as much as volume.** When the 32 threads of a warp read neighbouring addresses, the hardware fetches them as one wide block. When they read scattered addresses, it needs many separate fetches for the same amount of data. This is called coalescing (Lesson 7).
- **Shared memory is yours to manage.** L1 fills itself automatically. Shared memory is a scratchpad the program controls: load data from VRAM once, then let a group of threads reuse it many times.
- **The figures are orders of magnitude.** Exact cache sizes, latencies and bandwidth vary by chip and generation, but the steep drop in speed from one level to the next is consistent.

Check yourself

A kernel adds two huge arrays element by element, c[i] = a[i] + b[i]. Would a GPU with twice as many ALUs but the same VRAM run it noticeably faster? Why or why not?

Answer on page 84.

Key terms

VRAM: the GPU's own memory, on the card.

Bandwidth: how much data memory delivers per second.

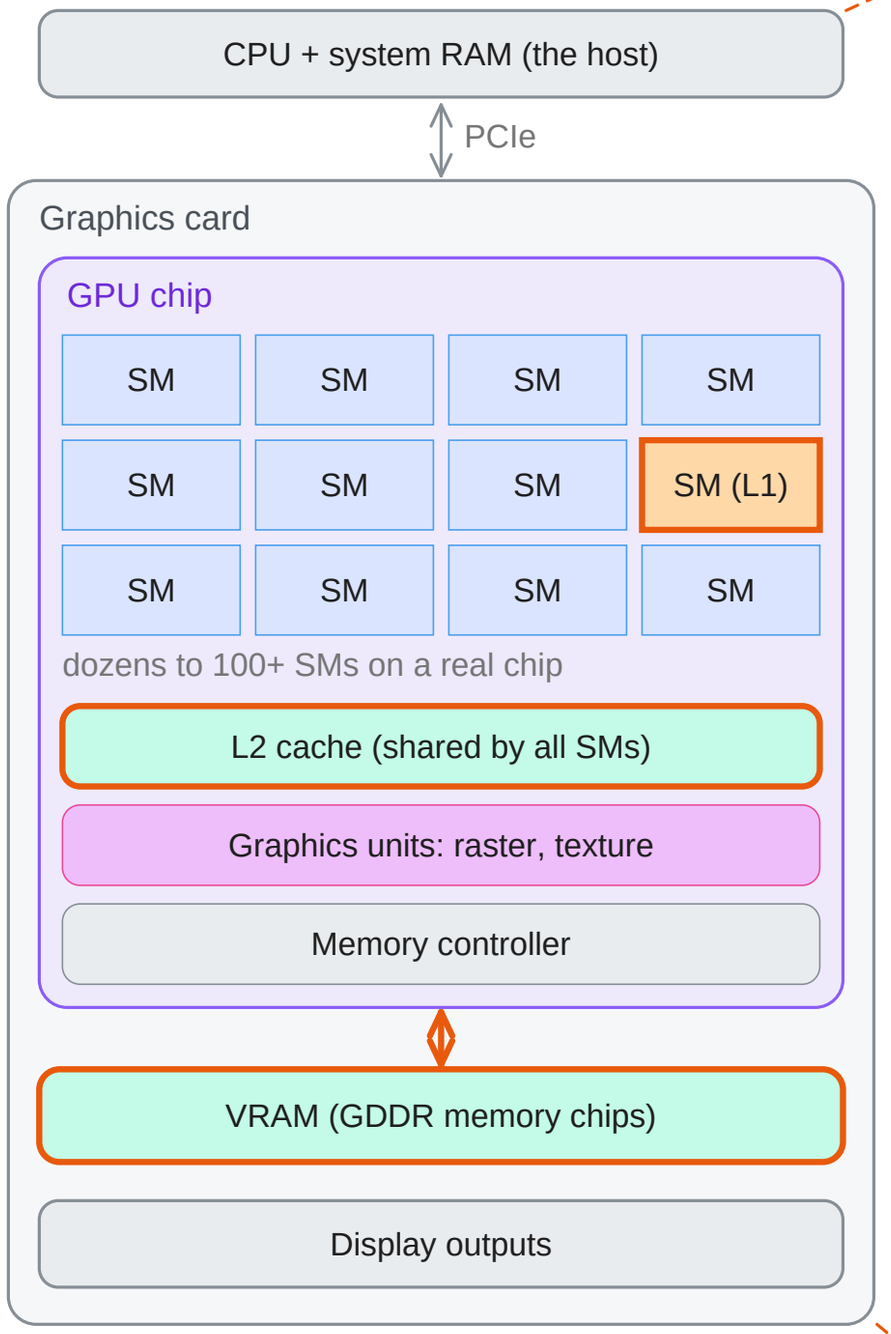
Shared memory: per-SM scratchpad the program controls.

PCIe: the link between the host and the card.

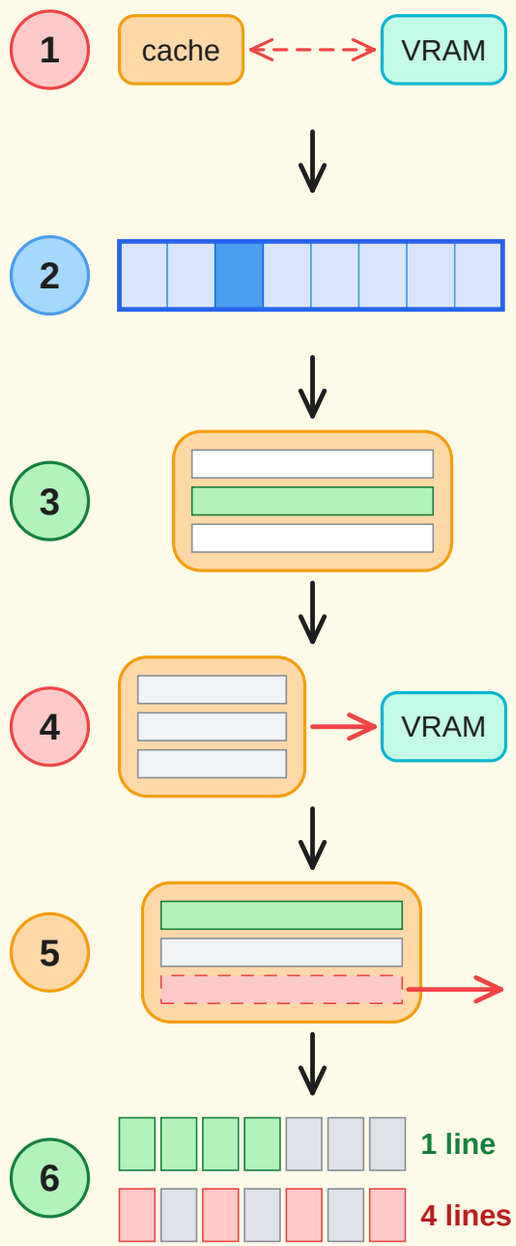
Lesson 4.1: How a cache works

Lines, hits and misses: the mechanism behind every level of the GPU's memory hierarchy.

The whole GPU (map)



Zoomed in: one cache, from request to answer



The problem: memory is far away

A trip to VRAM costs hundreds of cycles. A cache is a small, fast store close to the lanes that keeps copies of recent data.

Data moves in lines, not single values

Ask for one number and the cache brings in its whole line: about 128 bytes, which is 32 numbers of 4 bytes each.

Hit: the line is already here

The cache checks whether it holds the line for that address. If so, the value comes back in a handful of cycles.

Miss: go and fetch it

If not, the request goes to the next level: L2, then VRAM. The warp waits, and a copy of the line is kept for next time.

Full cache: something must leave

A cache holds only so many lines. To fit a new one it throws out an old one, usually the one unused for the longest.

Why layout matters so much

32 threads reading 32 neighbours touch one line. 32 threads reading scattered values touch 32 lines, 32 times the work.

Key idea: a cache keeps recent lines of data close, so nearby and recently used data is cheap and the rest is slow

Lesson 4.1 notes

How a cache works

In brief

A cache is a small, fast memory that keeps copies of recently used data near the lanes. Data moves in fixed-size lines. A request is a hit if its line is already there and a miss if it has to be fetched from further away. L1 and L2 both work this way.

Beyond the board

- **Two habits make caches work.** Programs tend to use data near what they just used, which is why fetching a whole line pays off. They also tend to reuse the same data soon, which is why keeping the line pays off.
- **How the lookup is fast.** Part of the address picks a small group of slots, and only those few are checked. The cache never searches everything it holds.
- **A GPU's L1 is shared by a crowd.** It holds on the order of a hundred kilobytes per SM, and thousands of threads compete for it. Lines are thrown out far sooner than on a CPU.
- **Line sizes differ.** 128 bytes is typical for NVIDIA GPUs. CPUs mostly use 64 bytes.

Check yourself

A thread reads an array of 4-byte numbers in order, starting from an empty cache with 128-byte lines. How many of the first 320 reads miss? What if it reads every 32nd element instead?

Answer on page 84.

Key terms

Cache line: the fixed-size block a cache moves and stores.

Hit: the line was already in the cache.

Miss: the line had to be fetched from further away.

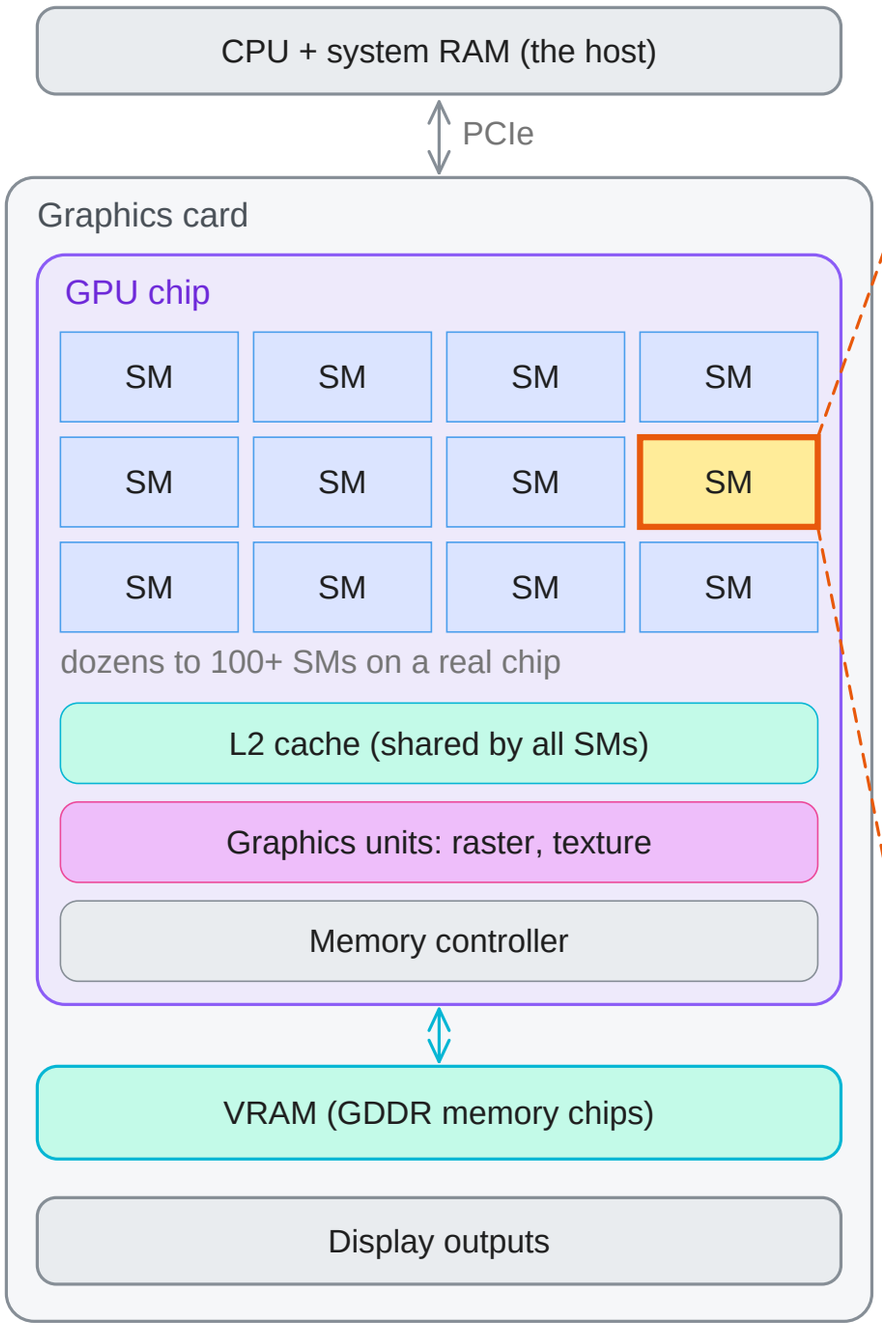
Eviction: throwing out a line to make room.

Locality: the tendency to use nearby or recent data.

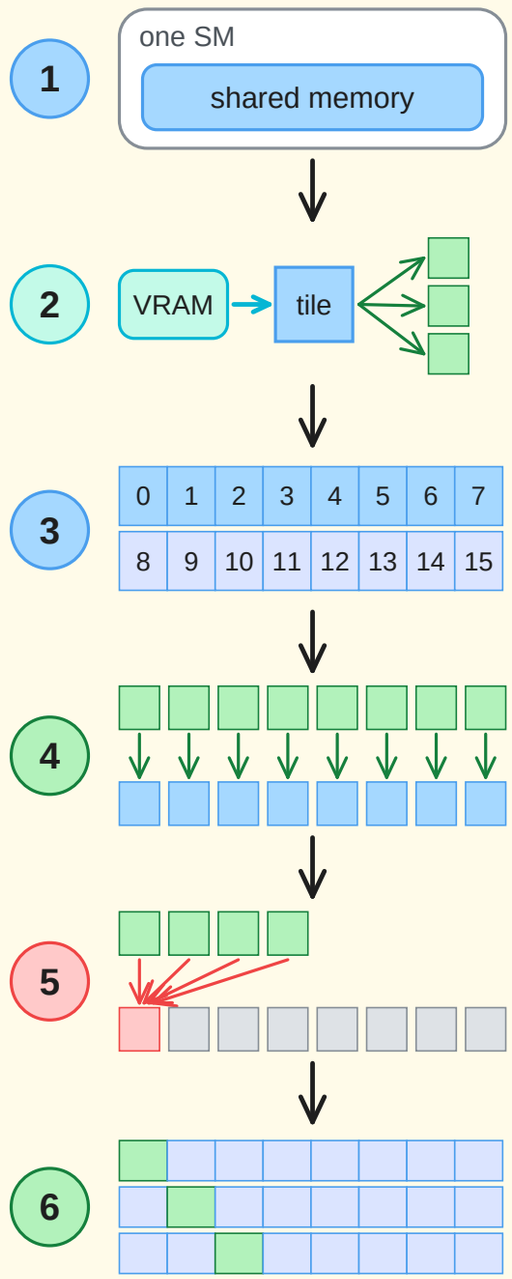
Lesson 4.2: Shared memory and bank conflicts

The scratchpad your code controls, how it is built, and the one access pattern that slows it down.

The whole GPU (map)



Zoomed in: shared memory inside one SM



A scratchpad inside the SM

Shared memory is fast storage in the SM, reserved per block and visible to all of that block's threads. Your code fills it.

Load once, reuse many times

Typical use: copy a tile of data from VRAM into shared memory once. Then all threads read it repeatedly at near-L1 speed.

It is built from 32 banks

Each 4-byte word lives in one bank, dealt out in turn: word 0 in bank 0, word 1 in bank 1, word 32 back in bank 0. (8 shown.)

No conflict: one cycle for the warp

A bank can serve one word per cycle. If each thread reads from a different bank, all 32 are served at the same time.

Bank conflict: threads queue up

If several threads want different words in the same bank, they are served one after another. 32 in one bank: 32 cycles.

The fix: shift the layout

Conflicts come from strides that are multiples of 32. Padding a 32-wide array to 33 columns spreads reads over banks.

Key idea: shared memory is fastest when a warp's threads land on different banks, and slow when they pile onto one

Lesson 4.2 notes

Shared memory and bank conflicts

In brief

Shared memory is a block of fast storage inside the SM that the program manages itself. It is built from 32 banks that can each serve one word per cycle. When a warp's 32 threads use 32 different banks, the whole warp is served at once. When they collide, they queue.

Beyond the board

- **Why 33 works.** In a 33-wide array, the element one row down is 33 words later, which lands in the next bank along. A warp reading down a column then hits 32 different banks.
- **Reading the same word is fine.** If all 32 threads read one identical word, it is fetched once and handed to all of them. Only different words in the same bank conflict.
- **The classic case.** Reading down the columns of a 32-wide tile, as a matrix transpose does, sends the whole warp to one bank.
- **It is not free space.** On several designs shared memory comes out of the same pool as L1, and the amount a block reserves is one of the three occupancy limits (Lesson 3.2).

Check yourself

In a warp, thread t reads word 2 x t of shared memory. How many threads land on each bank, and how many cycles does the read take? What about word 3 x t?

Answer on page 84.

Key terms

Shared memory: per-block scratchpad the program controls.

Bank: one of 32 slices that serves one word per cycle.

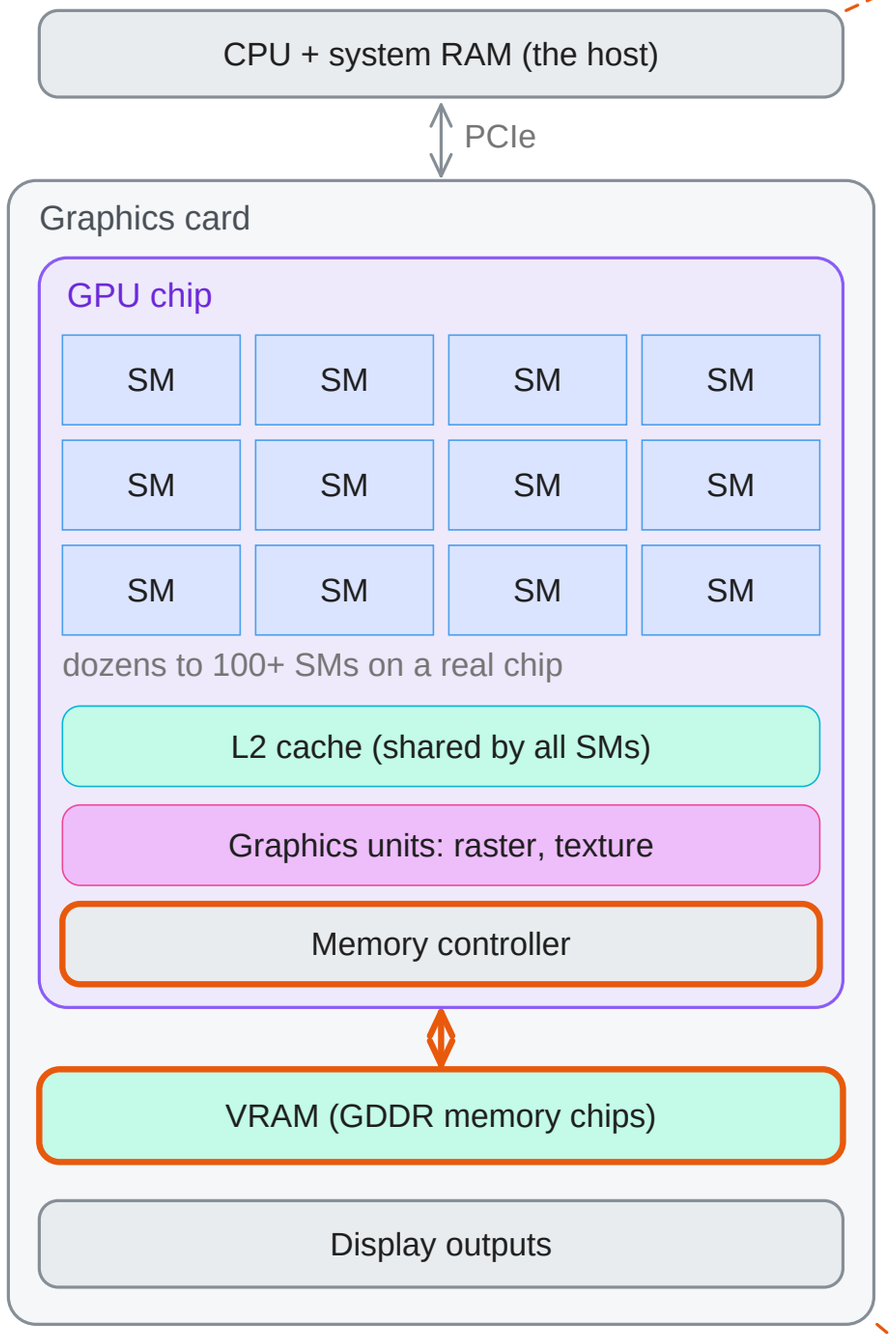
Bank conflict: threads needing different words in one bank.

Stride: the gap between the addresses threads use.

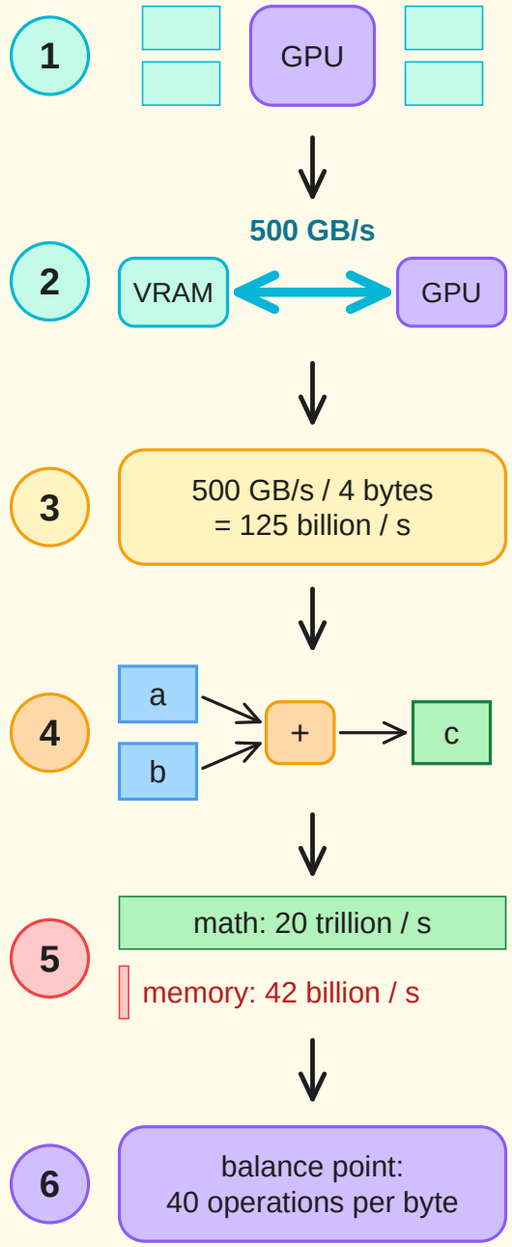
Lesson 4.3: VRAM and bandwidth math

Working out when a kernel is limited by how fast memory can feed it, not by how fast it can calculate.

The whole GPU (map)



Zoomed in: the pipe between VRAM and the chip



VRAM: memory chips beside the GPU

The GPU's main memory is a ring of separate chips on the card, holding from 8 to 24 GB or more. All data starts here.

Bandwidth: the width of the pipe

Bandwidth is how many bytes per second can cross between VRAM and the chip. A fast card manages roughly 500 GB/s.

Turn bytes into numbers

A 32-bit number is 4 bytes, so 500 GB/s carries 125 billion numbers per second. That is the most the chip can be fed.

Count what a kernel moves

c[i] = a[i] + b[i] reads two numbers and writes one: 12 bytes per element. So at most 500 / 12 = 42 billion elements/s.

Compare with what the lanes can do

The same chip can do 20 trillion additions per second. Memory lets it do 42 billion. The lanes sit idle over 99% of the time.

The balance point

20 trillion operations / 500 billion bytes = 40 operations per byte. Kernels doing less than that per byte are memory-bound.

Key idea: divide the chip's operations per second by its bytes per second to see how much math each byte must carry

Lesson 4.3 notes

VRAM and bandwidth math

In brief

Bandwidth is the number of bytes per second that can move between VRAM and the chip. Divide it by the bytes a kernel moves per element and you get the most elements it can process, whatever the lanes could do. The example chip does 20 trillion operations and moves 500 GB per second.

Beyond the board

- **Where the number comes from.** Bandwidth is wires times speed. A card with 384 data wires at 21 gigabits per second each moves 384 x 21 / 8, about 1,000 GB per second.
- **The range today.** Mid-range cards manage a few hundred GB per second, top consumer cards about 1,000, and data-center GPUs with stacked memory several thousand (Lesson 8.4).
- **Caches do not help data used once.** If each number is read a single time, every read is a miss and the full cost is paid.
- **The cure is doing more per trip.** Merge passes so data is loaded once and used for several steps, or keep a tile in shared memory and reuse it.

Check yourself

A GPU does 30 trillion operations per second and has 1,000 GB/s of bandwidth. What is its balance point? A kernel reads 4 bytes, does 200 operations and writes 4 bytes per element. Which limit binds, and how many elements per second can it process?

Answer on page 84.

Key terms

Bandwidth: bytes moved per second.

GB/s: billions of bytes per second.

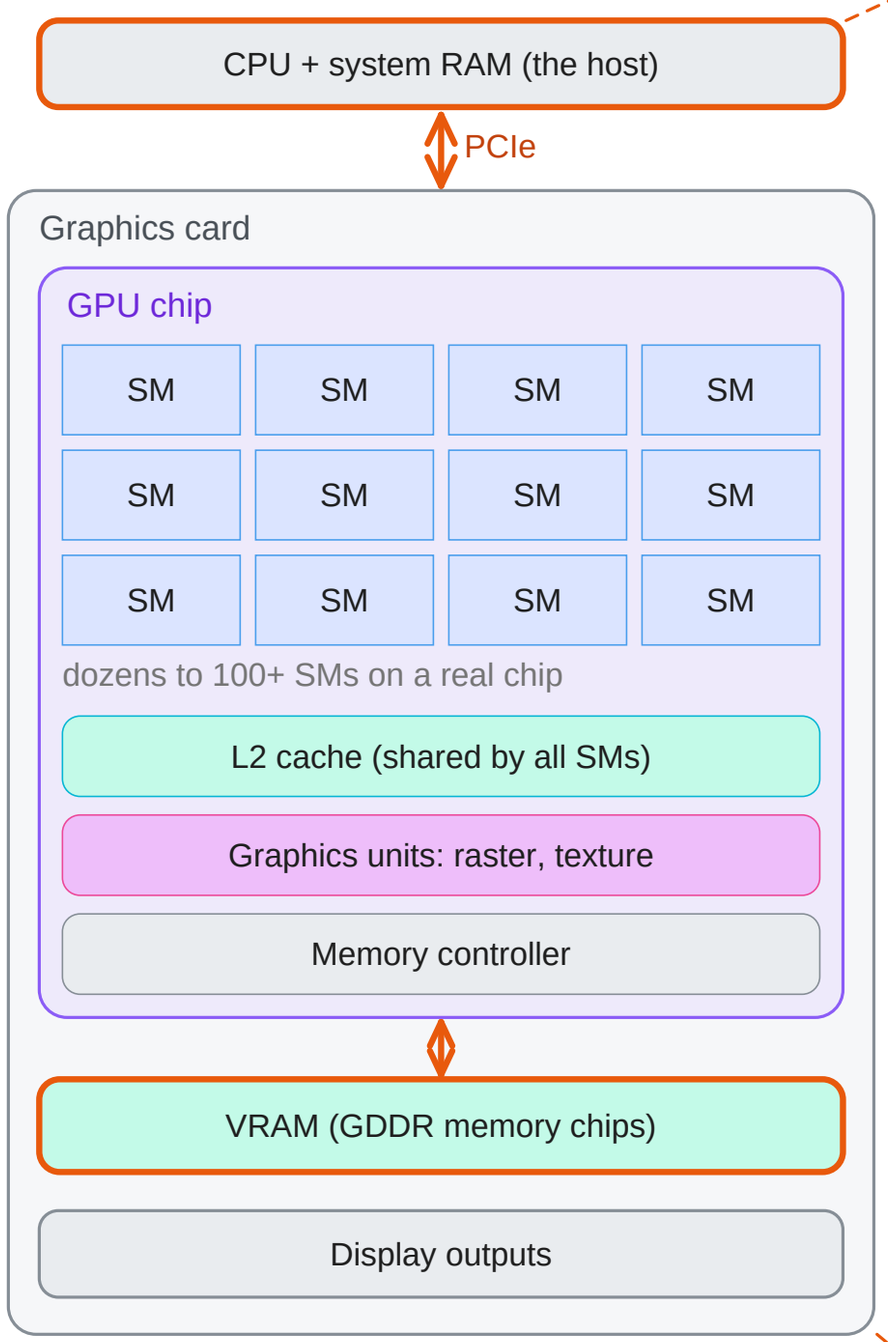
Memory-bound: limited by bandwidth, not by arithmetic.

Balance point: operations per byte at which the two limits meet.

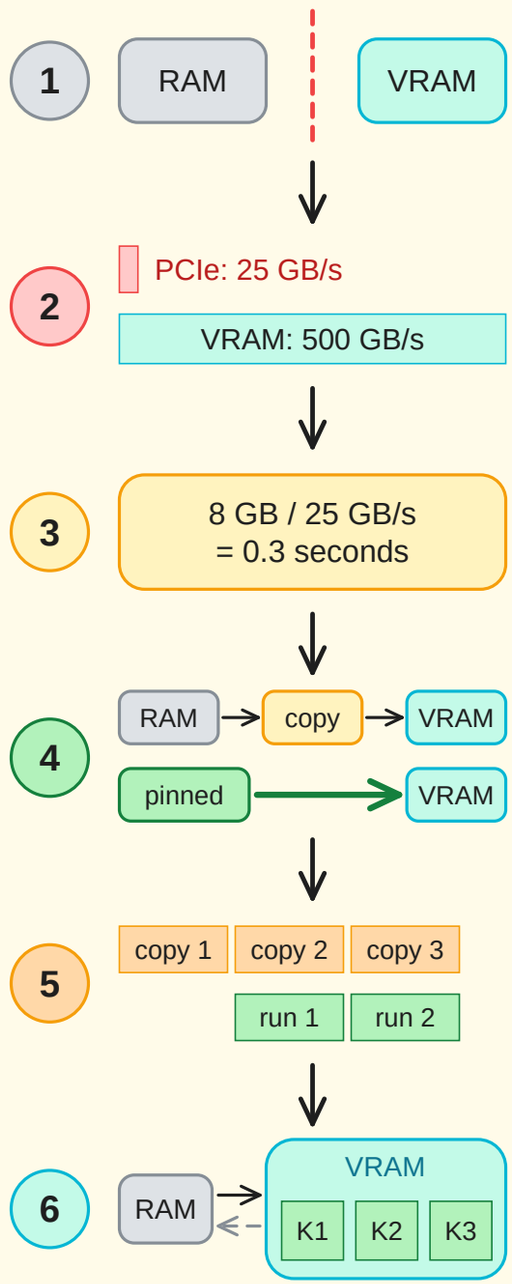
Lesson 4.4: Moving data between host and GPU

The trip across PCIe is the slowest path on the map. What it costs, and how programs keep it rare.

The whole GPU (map)



Zoomed in: the trip from RAM to VRAM



Two separate memories

Your data starts in system RAM, which the CPU owns. The GPU can only compute on data in VRAM, so it has to be copied over.

The link between them is narrow

A PCIe connection carries roughly 25 GB per second. The pipe from VRAM into the chip carries 500. That is 20 times less.

What a copy costs

Moving 8 GB takes about a third of a second. In that time the chip could have done 6 trillion operations.

Pinned memory: a direct route

Ordinary RAM must first be copied to a staging area the system promises not to move. Pinned memory skips that step.

Overlap copying with computing

Copies can run in the background. Send batch 2 while the GPU works on batch 1, and the copy time mostly disappears.

The rule: move once, keep it there

Copy inputs in once. Chain kernels so each reads the last one's output from VRAM. Copy back only the final answer.

Key idea: the host link is about 20 times narrower than VRAM's, so copy data over once and keep it on the GPU

Lesson 4.4 notes

Moving data between host and GPU

In brief

The CPU's memory and the GPU's memory are separate, joined by the PCIe link. That link carries tens of GB per second, far less than the path from VRAM into the chip. Fast programs copy data across once, keep it on the GPU, and overlap any copying with computing.

Beyond the board

- **The link's speed by generation.** A full-width PCIe connection carries about 16, 32 or 64 GB per second for generations 3, 4 and 5. Real copies achieve somewhat less.
- **Pinning has a price.** Pinned memory cannot be moved or swapped out by the operating system, so pinning a lot of it can slow down the rest of the machine.
- **Unified memory hides the copy.** Some systems let the CPU and GPU use the same pointers and move the data automatically when it is touched. It is convenient, but the copy still happens.
- **Integrated GPUs differ.** In phones and many laptops the GPU shares the system's memory. There is no copy to make, but there is also much less bandwidth.

Check yourself

Each training step copies a 2 GB batch at 20 GB/s and then computes for 0.4 seconds. What share of the step is copying? How long is a step if the copy overlaps the previous step's computing?

Answer on page 84.

Key terms

Host: the CPU and its system RAM.

PCIe: the link between the host and the card.

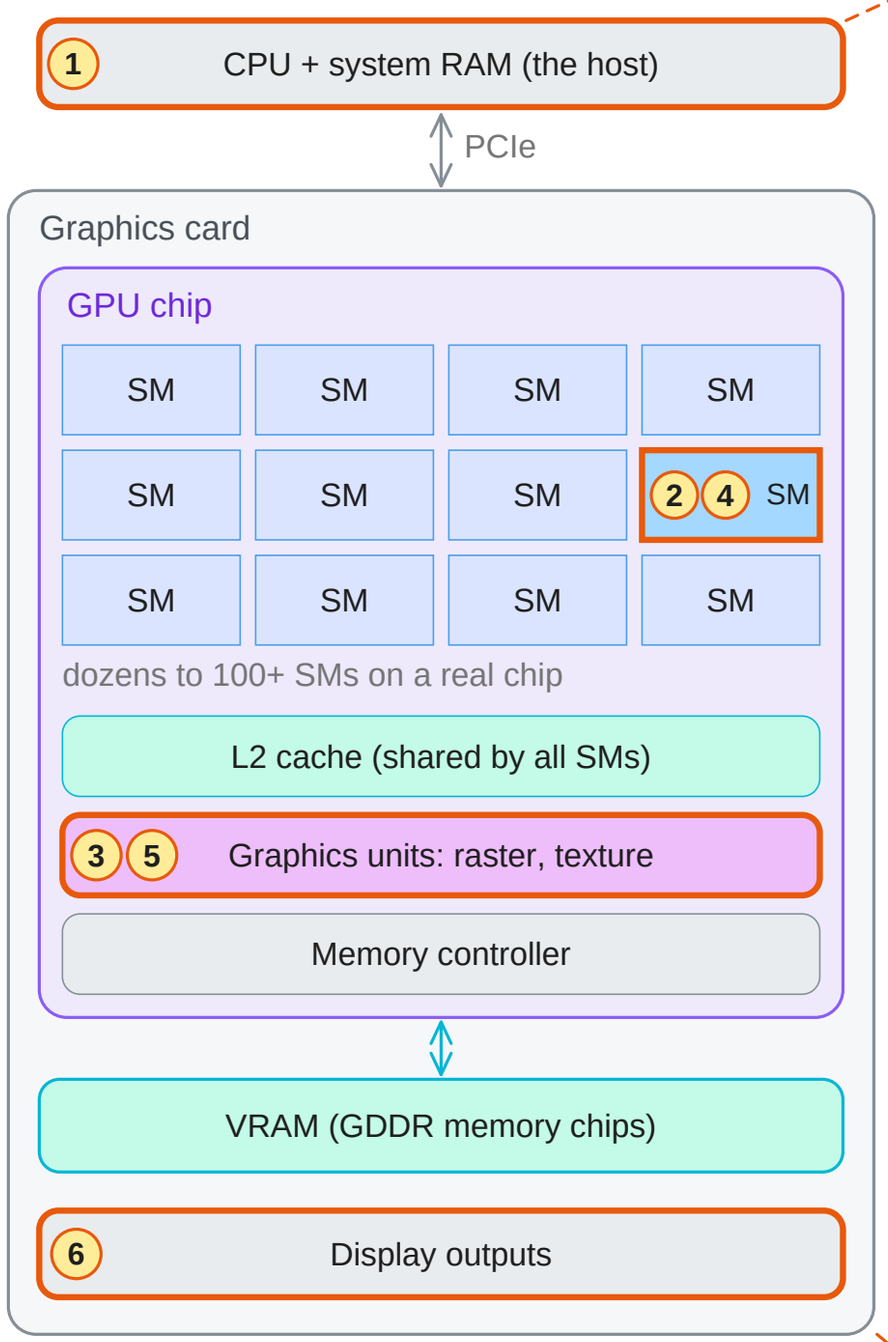
Pinned memory: RAM the system promises not to move.

Asynchronous copy: a copy that runs in the background.

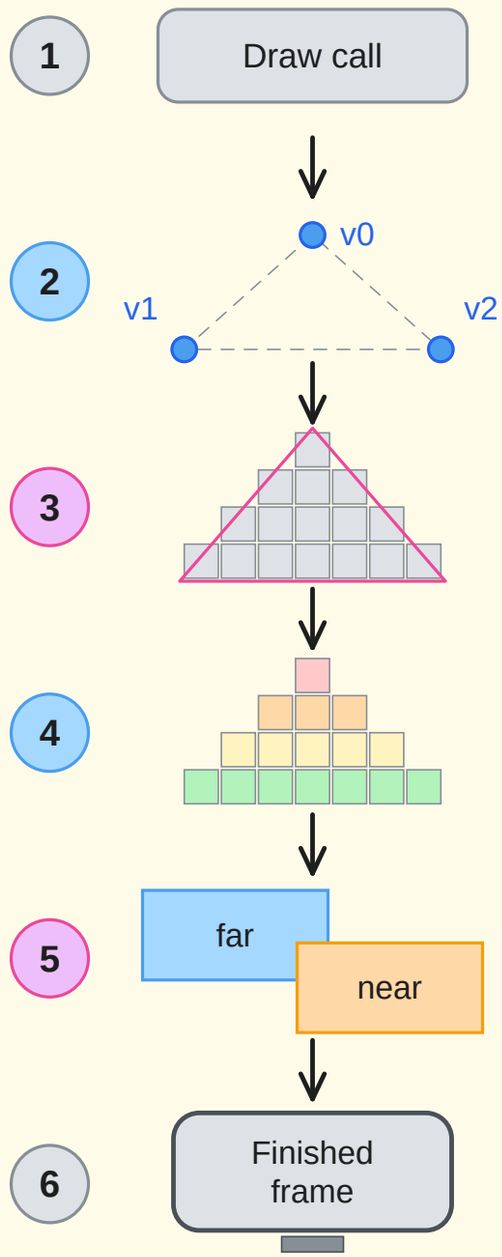
Lesson 5: The graphics pipeline

Follow one triangle to the screen. Blue steps run on SMs, pink steps on fixed graphics hardware.

The whole GPU (map)



Zoomed in: one triangle's trip through the pipeline



Draw call (CPU)

The game says: draw these triangles with these shaders. Vertex data and textures already sit in VRAM, uploaded earlier.

Vertex shader (runs on SMs)

A small program runs once per vertex and moves it from 3D space to a 2D screen position. One thread per vertex.

Rasterizer (fixed hardware)

Joins vertices into triangles, discards hidden or off-screen ones, then finds every pixel each one covers. Output: fragments.

Fragment shader (runs on SMs)

A program runs once per fragment to pick its color: lighting, plus texture lookups by texture units. Millions per frame.

Depth test + blend (fixed hardware)

Keeps the fragment nearest the camera, blends transparent ones, and writes the result to the framebuffer in VRAM.

Display

The display engine reads the finished framebuffer from VRAM and sends it to your monitor, 60+ times per second.

Key idea: programmable SMs do the per-vertex and per-pixel work, fixed hardware does the glue in between

Lesson 5 notes

The graphics pipeline

In brief

Drawing a frame is a pipeline. Programmable stages (the vertex and fragment shaders) run as ordinary threads on the SMs. Fixed-function stages (rasterizing, texture filtering, depth testing and blending) run on dedicated circuits in between.

Beyond the board

- **This workload is why GPUs look the way they do.** Every pixel can be shaded without knowing about its neighbours, so the fragment stage is exactly the "same operation on millions of independent items" from Lesson 1. A warp is simply 32 fragments running the same shader.
- **Why some steps are fixed hardware.** Rasterizing, texture filtering and depth testing are the same every time, so dedicated circuits do them faster and with less power than a program on the SMs could.
- **The order is simplified.** Real GPUs usually run the depth test before the fragment shader when they can, so hidden fragments are never shaded at all. Real pipelines also have optional stages, such as tessellation.
- **This is where general-purpose computing came from.** Once shaders became programmable, people realized the SMs could run any parallel math, not only lighting.

Check yourself

A 4K frame has about 8.3 million pixels. If each one needs a fragment-shader thread, roughly how many warps is that for a single frame?

Answer on page 84.

Key terms

Draw call: the CPU's request to draw a set of triangles.

Vertex shader: program run once per vertex.

Rasterizer: finds the pixels each triangle covers.

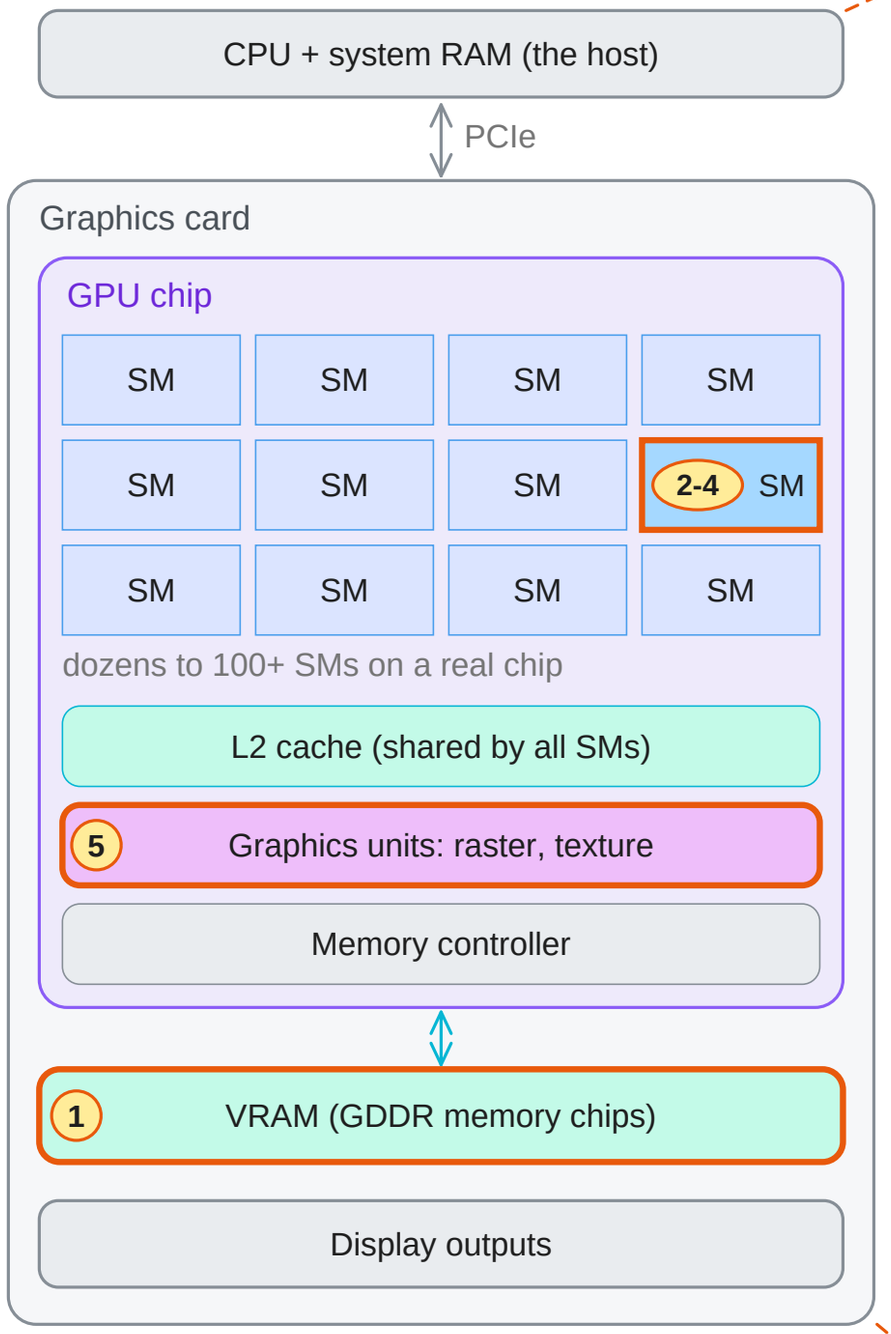
Fragment: a candidate pixel produced by the rasterizer.

Framebuffer: the image being built, held in VRAM.

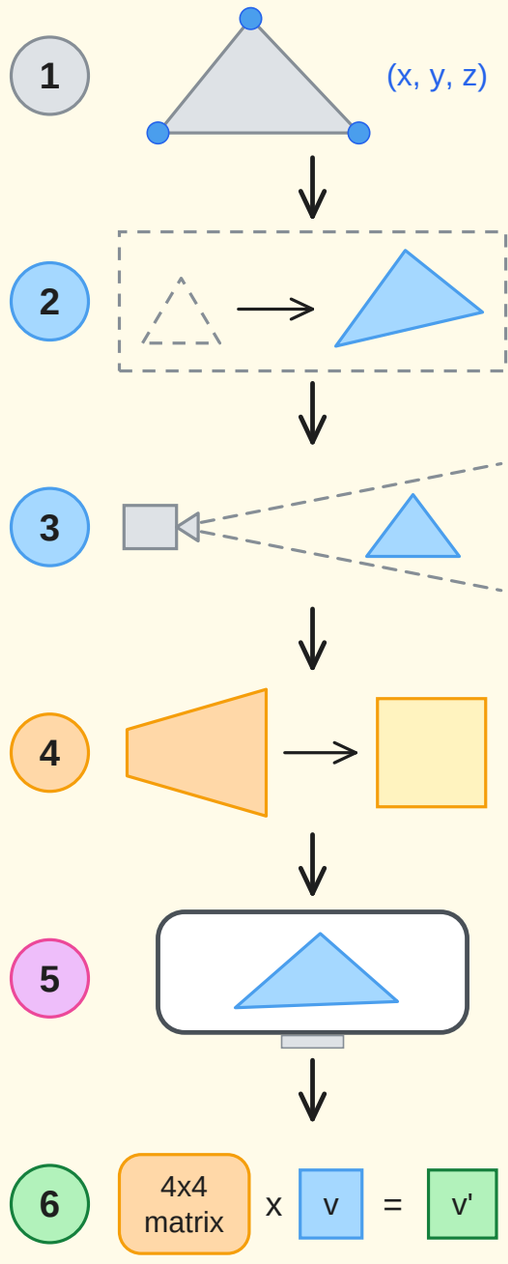
Lesson 5.1: Coordinate spaces and the vertex stage

How a 3D point in a model becomes a position on your screen, and what the vertex shader actually computes.

The whole GPU (map)



Zoomed in: one vertex's journey



A model is a list of vertices

Each vertex has a position (x, y, z) in the model's own space, plus extras like color and which way the surface faces.

Model to world: place the object

Moving, rotating and scaling put the model where it belongs in the scene. Every vertex now has a world position.

World to view: stand at the camera

Next, everything is re-expressed relative to the camera: straight ahead, left and right, up and down from where it stands.

Projection: far things shrink

The camera sees a pyramid-shaped region. Projection squashes it into a cube, which makes distant objects smaller.

To the screen: pixel coordinates

The cube is stretched to the window: x and y become pixel positions, and z is kept as depth for the depth test later.

On the hardware: one matrix multiply

Steps 2 to 4 combine into one 4x4 matrix. The vertex shader multiplies each vertex by it: one thread per vertex, 32 per warp.

Key idea: the vertex stage is one matrix multiply per vertex, repeated for millions of independent vertices

Lesson 5.1 notes

Coordinate spaces and the vertex stage

In brief

A vertex starts as a point in its model's own coordinates. It is placed in the world, re-expressed relative to the camera, and projected so that distant things shrink. Those three moves combine into a single 4x4 matrix, and the vertex shader applies it once per vertex.

Beyond the board

- **Why the matrix is 4x4 for 3D points.** Each position gets a fourth number, set to 1. That extra slot lets a move (not just a rotation or a scale) be written as a multiplication, and it lets projection be one too.
- **How far things shrink.** After the multiply, x, y and z are divided by the fourth number, which projection sets to the distance from the camera. Larger distance, smaller result.
- **Clipping.** Triangles that end up partly outside the cube are cut at its faces before they are rasterized. Triangles fully outside are dropped.
- **Real vertex shaders do more.** They also transform the surface direction for lighting, and may bend the model first to animate a character.

Check yourself

A scene has 3 million vertices, and a vertex shader costs about 32 operations per vertex. How many operations is that per frame, and per second at 60 frames per second? What share is that of a chip that can do 20 trillion per second?

Answer on page 84.

Key terms

Vertex: a corner point of a triangle, with its data.

Model, world, view space: coordinates relative to the object, the scene, the camera.

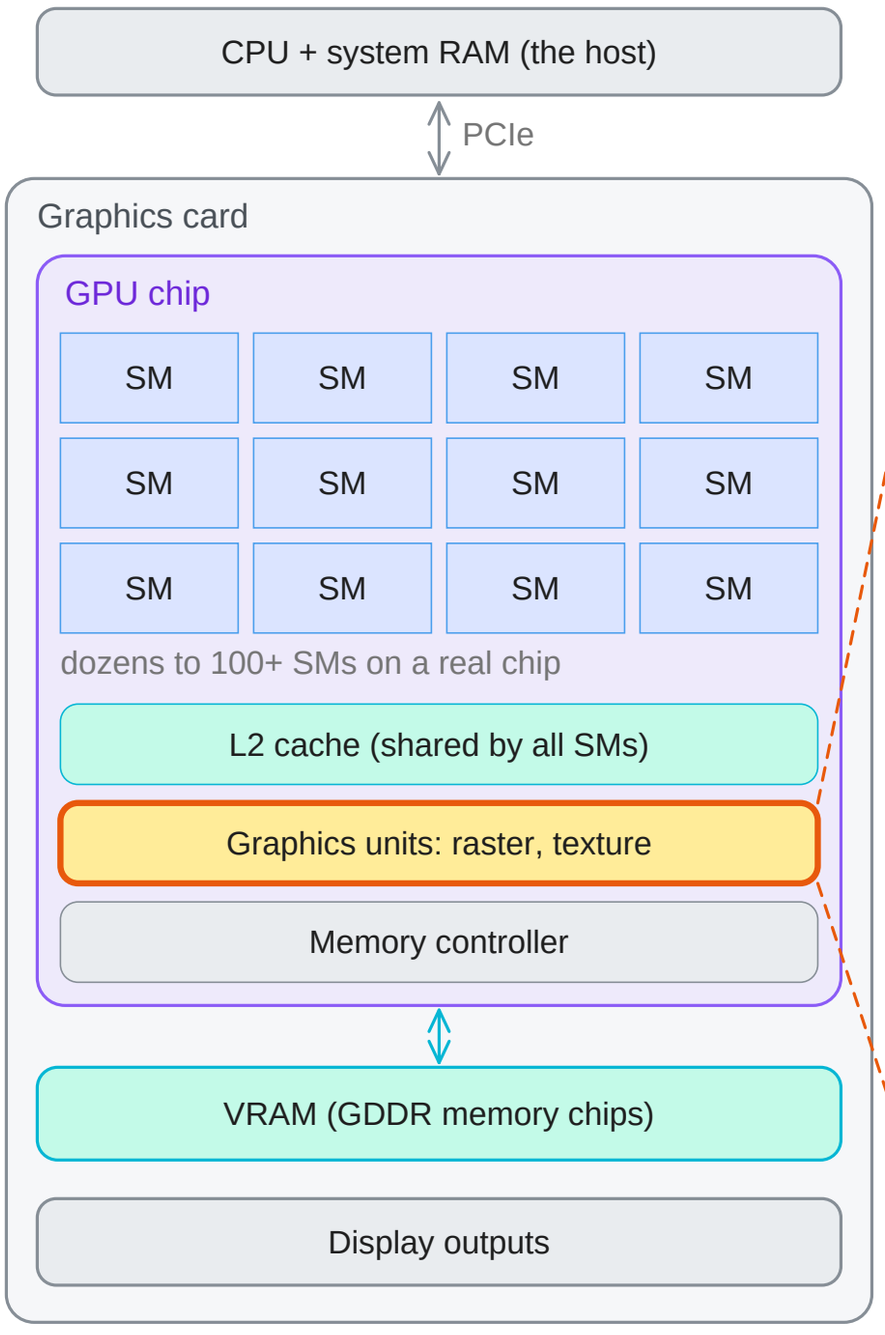
Projection: the step that makes distant things smaller.

Clipping: cutting away what falls outside the view.

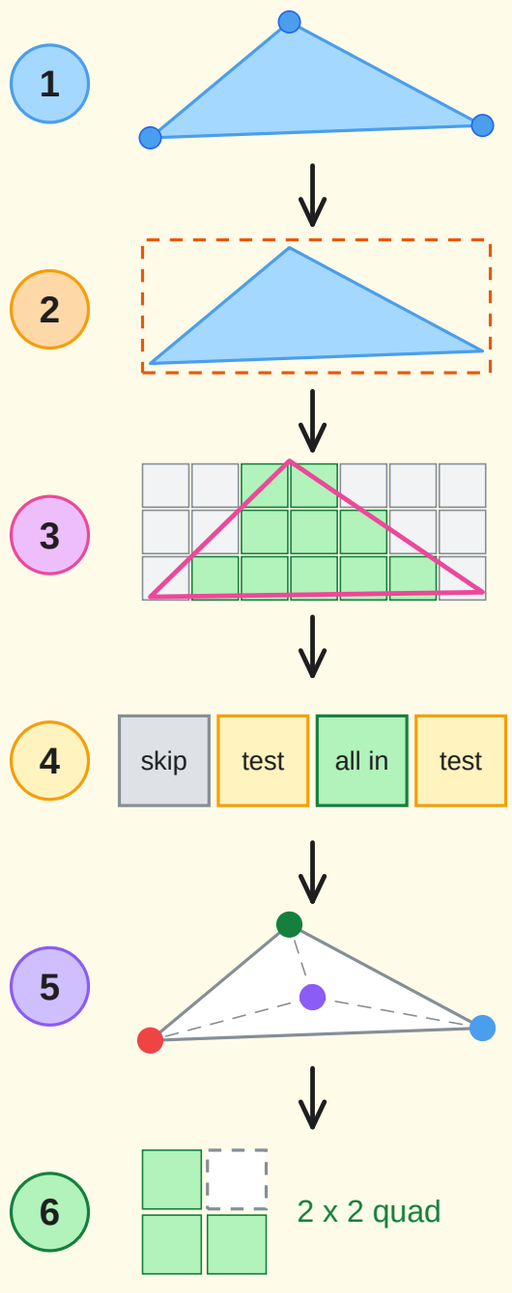
Lesson 5.2: Rasterization

How fixed hardware decides which pixels a triangle covers, and what it hands to the fragment shader.

The whole GPU (map)



Zoomed in: the rasterizer



Input: three screen positions

After the vertex stage, a triangle is three points in pixel coordinates, each with a depth and extras such as color.

Start with a bounding box

Only pixels inside the smallest rectangle around the triangle can be covered, so the rest of the screen is never looked at.

The edge test

For each pixel center, three small sums say which side of each edge it is on. On the inner side of all three: covered.

Coarse first, then fine

Whole tiles of pixels are tested first. Tiles fully outside are skipped, tiles fully inside are accepted in one go.

Interpolation: blend the corners

Each covered pixel gets three weights: how close it is to each corner. Depth, color and the rest are blended by them.

Output: fragments, in 2x2 quads

Fragments leave in blocks of four, even if some are outside the triangle. The texture stage needs the neighbours (5.3).

Key idea: rasterization is three cheap side-of-line tests per pixel, done in bulk by dedicated hardware

Lesson 5.2 notes

Rasterization

In brief

The rasterizer turns a triangle's three screen positions into the list of pixels it covers. It tests pixel centers against the triangle's three edges, working on whole tiles first. For each covered pixel it also works out how to blend the values stored at the three corners.

Beyond the board

- **The sums are almost free.** Moving one pixel to the right changes each edge sum by a fixed amount, so the hardware steps across with additions instead of recomputing.
- **The weights come from the same sums.** The three edge values, scaled, are the three blending weights (barycentric coordinates). They are corrected for perspective so textures do not warp.
- **Shared edges need a tie-break.** A pixel center exactly on the edge between two triangles must belong to one of them only. A fixed rule decides, so no pixel is drawn twice or skipped.
- **Tiny triangles are wasteful.** Setting up a triangle has a fixed cost, and quads are shaded whole. Scenes made of pixel-sized triangles pay both costs millions of times.

Check yourself

A triangle covers a single pixel. How many fragments are shaded, and how much of that is wasted? Compare 2 million such triangles with one triangle that covers 2 million pixels.

Answer on page 85.

Key terms

Rasterizer: finds the pixels each triangle covers.

Bounding box: the smallest rectangle around a shape.

Edge test: a sum that says which side of a line a point is on.

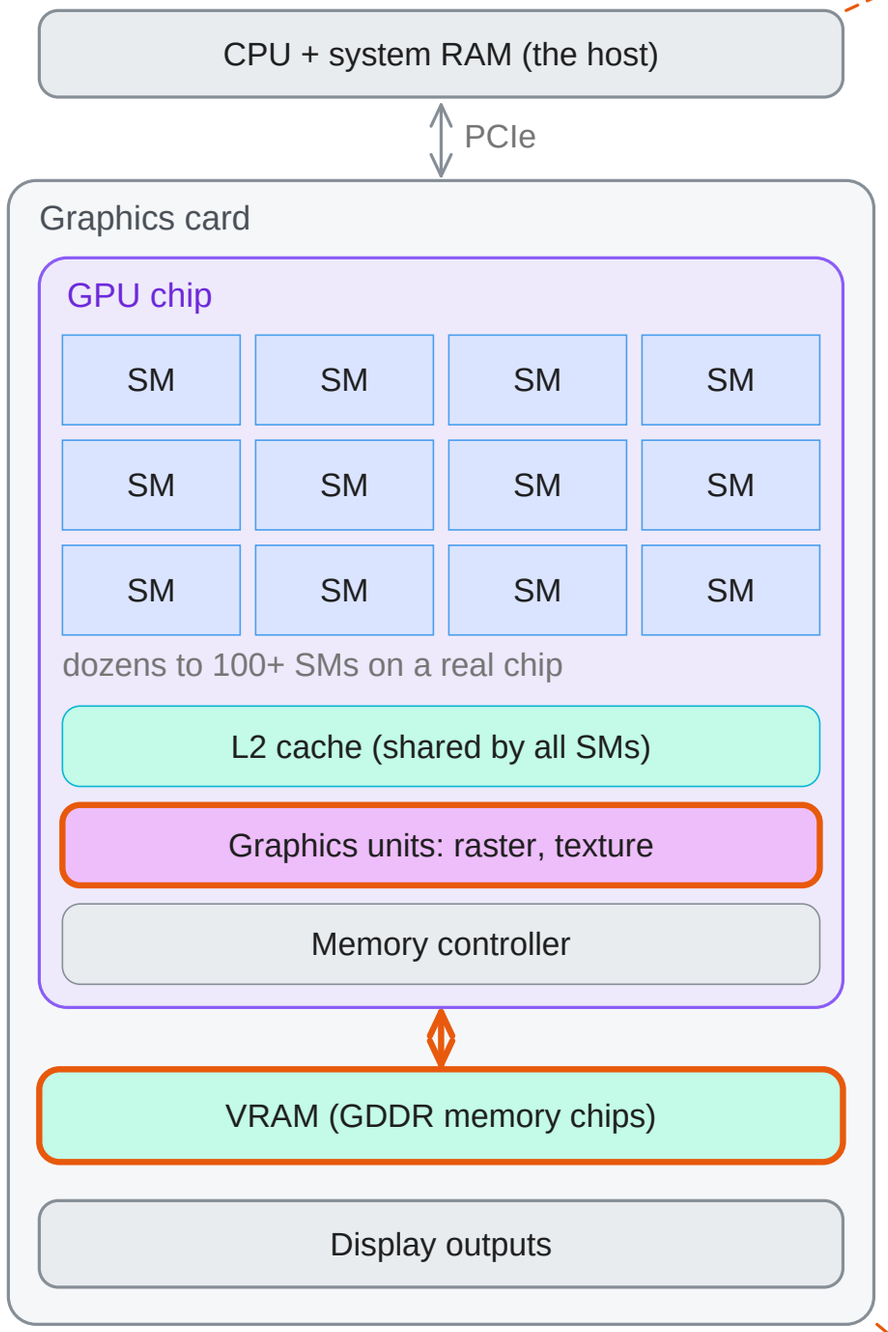
Fragment: a candidate pixel, with its blended values.

Quad: a 2 by 2 block of fragments.

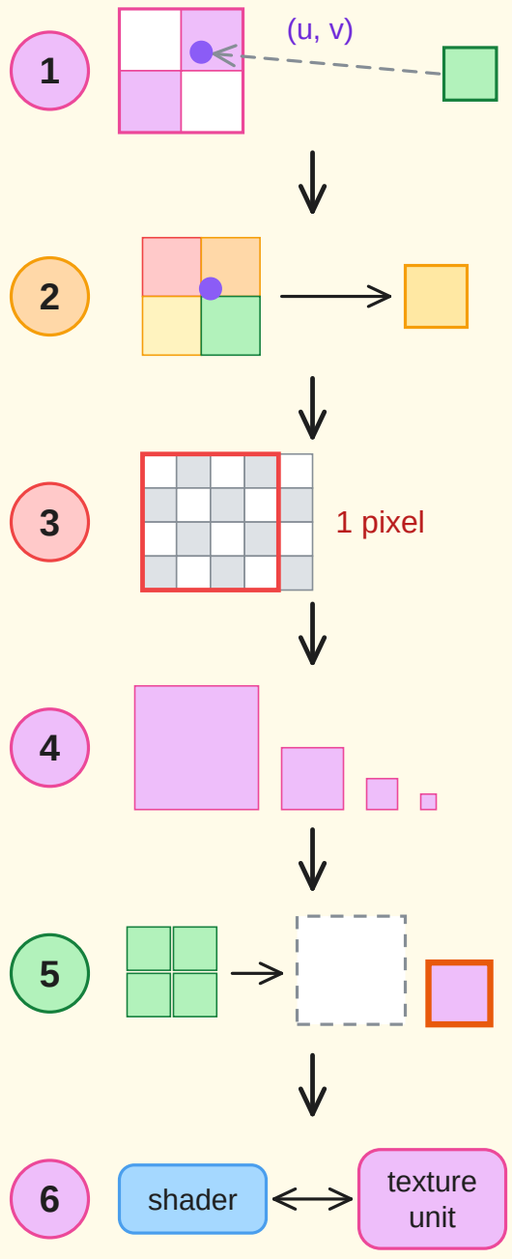
Lesson 5.3: Texturing and mipmaps

How a fragment looks up its color from an image, and why the hardware keeps several sizes of every texture.

The whole GPU (map)



Zoomed in: one texture lookup



A texture is an image to look up

Each fragment carries two numbers, u and v, from 0 to 1. They say where on the image this fragment takes its color.

Bilinear filtering: blend 4 texels

The spot rarely lands on one texture pixel (a texel). So the 4 nearest are read and blended by distance.

The far-away problem

On a distant surface one screen pixel covers many texels. Reading only 4 of them gives flicker and noise.

Mipmaps: pre-shrunk copies

The texture is stored at full size, half, a quarter, and so on down to 1 texel. All the copies add only a third more.

Picking the level: why quads exist

The hardware compares u and v between neighbours in the 2x2 quad. A big jump per pixel means: use a smaller copy.

Texture units do the work

The shader just asks for a sample. Fixed units compute addresses, fetch up to 8 texels, decompress and blend them.

Key idea: a texture sample is a filtered memory read, and mipmaps keep it both good-looking and cache-friendly

Lesson 5.3 notes

Texturing and mipmaps

In brief

A texture lookup, or sample, reads an image stored in VRAM at the spot a fragment asks for. The hardware blends the nearest texels so the result is smooth. For surfaces far away it reads from a smaller, pre-shrunk copy of the image called a mipmap.

Beyond the board

- **Why the copies cost only a third.** Each level has a quarter of the pixels of the one before: 1/4 + 1/16 + 1/64 and so on adds up to 1/3.
- **Better filters read more.** Trilinear filtering blends two neighbouring levels, 8 texels in all. Anisotropic filtering takes several such samples along a slanted surface.
- **Mipmaps help the caches too.** Neighbouring pixels read neighbouring texels of a small image, instead of jumping across a large one. That is the locality from Lesson 4.1.
- **Textures are stored compressed.** Block-compressed formats are 4 to 8 times smaller, and the texture unit decodes them as it reads.
- **To the scheduler it is a memory wait.** A warp that asks for a sample waits like any other, and another warp runs.

Check yourself

A texture is 4,096 by 4,096 texels at 4 bytes each. How big is it, and how big with its mipmaps? If every pixel of a 4K frame (8.3 million) takes 3 trilinear samples, how many texel reads is that?

Answer on page 85.

Key terms

Texel: one pixel of a texture.

Sample: a filtered read from a texture.

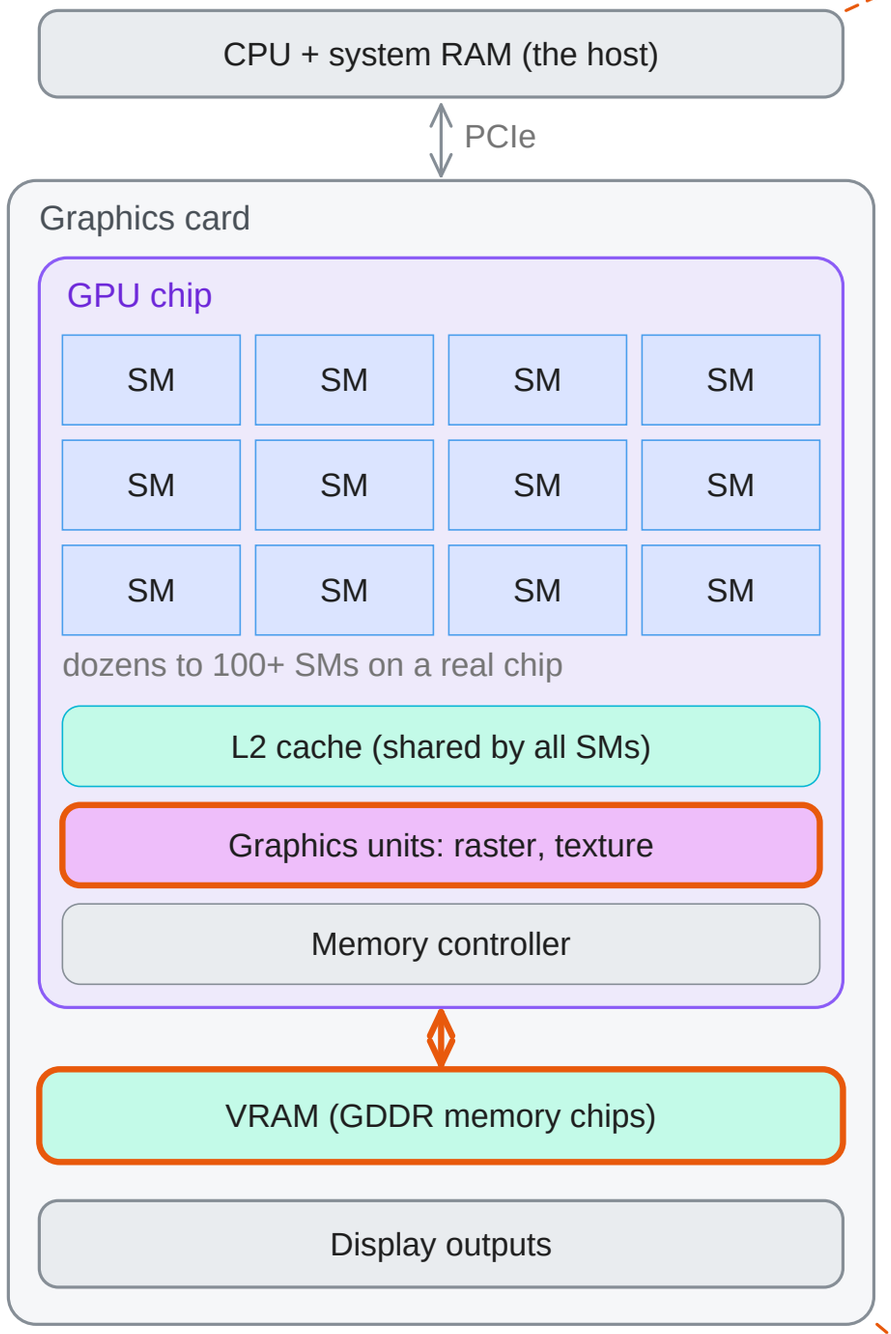
Bilinear filtering: blending the 4 nearest texels.

Mipmap: a chain of pre-shrunk copies of a texture.

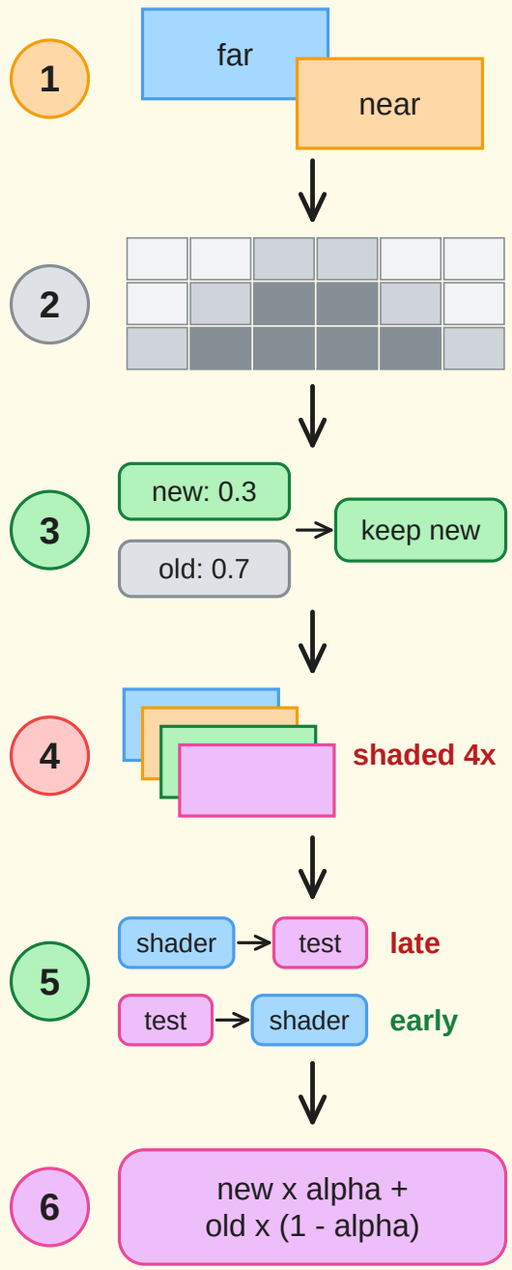
Lesson 5.4: Depth, early depth testing and blending

How the GPU decides which fragment wins each pixel, and how it avoids shading the ones that lose.

The whole GPU (map)



Zoomed in: the end of the pipeline



The problem: who is in front?

Many triangles land on the same pixel. The nearest one must win, whatever order they happen to be drawn in.

The depth buffer

Alongside the color image, VRAM holds one depth value per pixel: how far away the nearest thing drawn there so far is.

The depth test

Each new fragment compares its depth with the stored one. Nearer: it replaces color and depth. Farther: thrown away.

Overdraw: shading what nobody sees

If the test comes after the shader, every layer gets fully shaded. Four surfaces stacked on a pixel cost 4 times the work.

Early depth test: check first

Testing before the shader means hidden fragments are never shaded. It helps most when near objects are drawn first.

Blending: see-through surfaces

Glass or smoke cannot just replace the pixel. The new color is mixed with the one already there, so order matters.

Key idea: the depth buffer makes the nearest fragment win, and testing early saves the cost of shading hidden ones

Lesson 5.4 notes

Depth, early depth testing and blending

In brief

Several triangles usually cover the same pixel. A depth buffer records how far away the nearest one so far is, and each new fragment is tested against it. Doing that test before the fragment shader saves the cost of shading surfaces nobody will see.

Beyond the board

- **When early testing is not allowed.** If a shader can throw a fragment away or change its depth, the outcome is not known in advance, so the test has to wait until afterwards.
- **Whole tiles can be rejected.** The hardware keeps the farthest depth in each tile of the screen. A triangle that lies behind it is dropped there without any per-pixel test.
- **A depth pre-pass.** Some engines draw the scene twice: first depth only, which is cheap, then again with full shading. In the second pass only the visible fragments survive.
- **Transparent objects go last.** They are drawn after the solid ones, from far to near, and are tested against the depth buffer without writing to it.
- **Depth has limited precision.** Two surfaces almost the same distance away can flicker as they take turns winning.

Check yourself

A 4K frame has 8.3 million pixels, with an average of 3 surfaces on each and a 500-operation shader. How many operations with late testing? With early testing drawn nearest-first? With early testing drawn farthest-first?

Answer on page 85.

Key terms

Depth buffer: one distance value per pixel, kept in VRAM.

Overdraw: shading a pixel more than once.

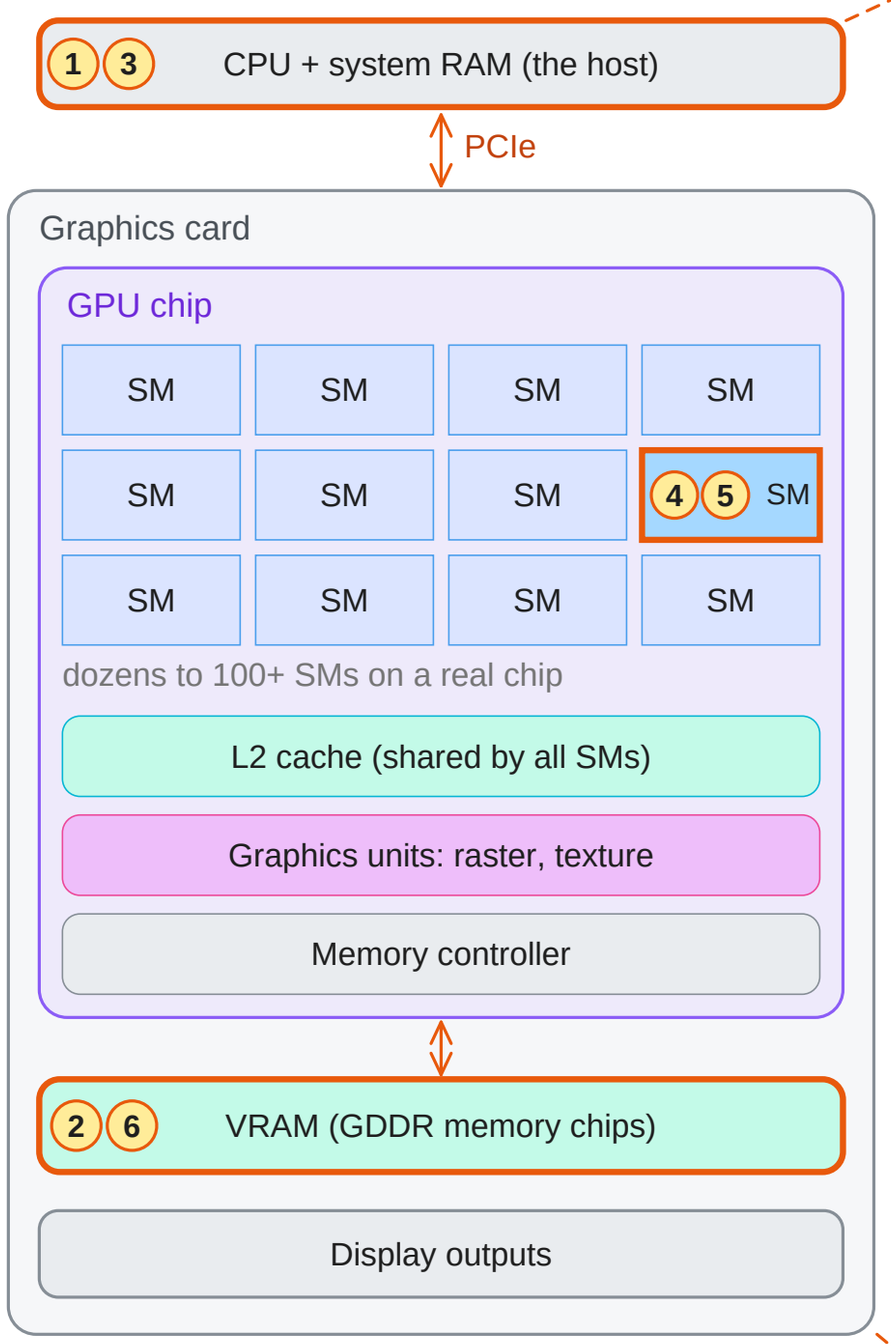
Early depth test: testing before the fragment shader runs.

Blending: mixing a new color with the one already there.

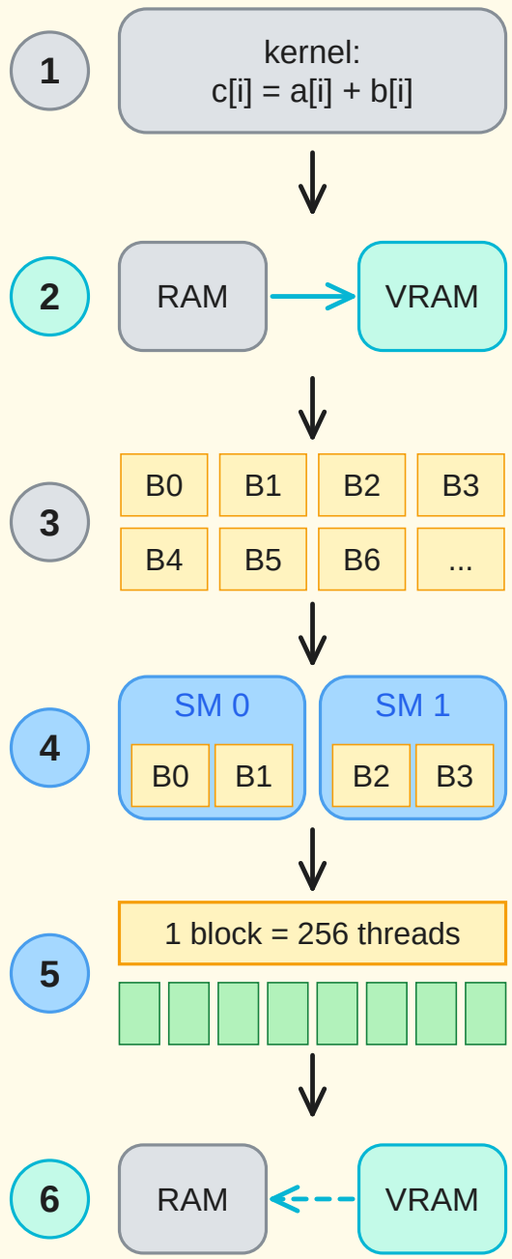
Lesson 6: Programming the GPU directly

From one function to a million threads. Gray steps run on the CPU, teal in memory, blue on the SMs.

The whole GPU (map)



Zoomed in: how your code becomes warps



Write a kernel (CPU side)

A kernel is a function that describes the work for ONE item. You write plain scalar code. The GPU runs it once per thread.

Copy the data into VRAM

Input arrays travel from system RAM over PCIe into VRAM. This is the slow link from Lesson 4, so do it once, not per step.

Launch a grid of blocks

The CPU asks for N threads, grouped into blocks: 1 million threads might be 3,907 blocks of 256. All blocks form the grid.

Blocks are handed to SMs

Each block runs on one SM from start to finish. An SM holds several at once. The rest queue until an SM has room.

Each block is cut into warps

256 threads = 8 warps of 32. Each thread gets an index and uses it to pick its own data: thread i handles element i.

Results land in VRAM

Outputs are written to VRAM. Copy them back to the CPU only when needed, or feed them straight into the next kernel.

Key idea: you write code for one element, and the GPU stamps it out as millions of threads in blocks and warps

Lesson 6 notes

Programming the GPU directly

In brief

In general-purpose GPU programming you write a kernel, a function describing the work for one element. The CPU launches it as a grid of blocks, each block runs on one SM, and the hardware cuts each block into warps of 32.

```
// Step 1: the kernel, written for ONE element
__global__ void add(float* a, float* b, float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;  // which element am I?
    if (i < n) c[i] = a[i] + b[i];
}

// Step 3: launch 3,907 blocks of 256 threads each
add<<<3907, 256>>>(a, b, c, 1000000);
```

Beyond the board

- **The first line of the kernel is the whole trick.** Each thread combines its block number and its position inside the block to work out which element is its own.
- **Why the if is there.** 3,907 blocks of 256 is slightly more than a million threads, so the last few have nothing to do.
- **Why blocks exist at all.** A block is the unit of cooperation. Its threads sit on the same SM, so they can share that SM's shared memory and wait for each other at sync points. Threads in different blocks cannot.
- **You never write warps.** Blocks and threads are what you choose. Warps are how the hardware runs them.
- **Other names for the same idea.** OpenCL calls a block a work-group and a thread a work-item, and graphics APIs offer the same model as compute shaders.

Check yourself

Suppose you choose a block size of 100 threads instead of 256. How many warps does each block need, and what gets wasted?

Answer on page 85.

Key terms

Kernel: a function run once per thread on the GPU.

Grid: all the blocks of one kernel launch.

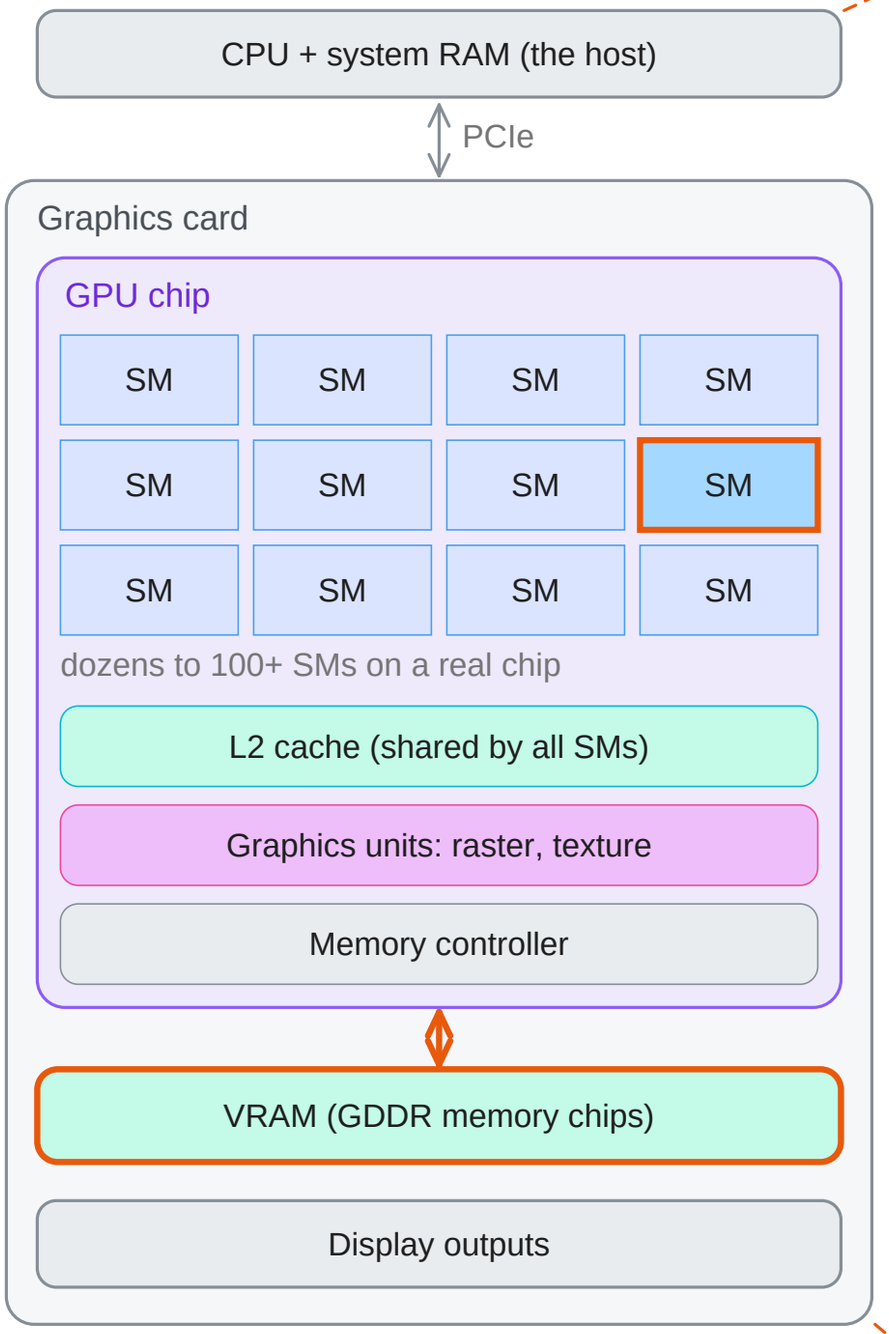
Block: threads that run together on one SM.

CUDA: NVIDIA's GPU programming language, an extension of C++.

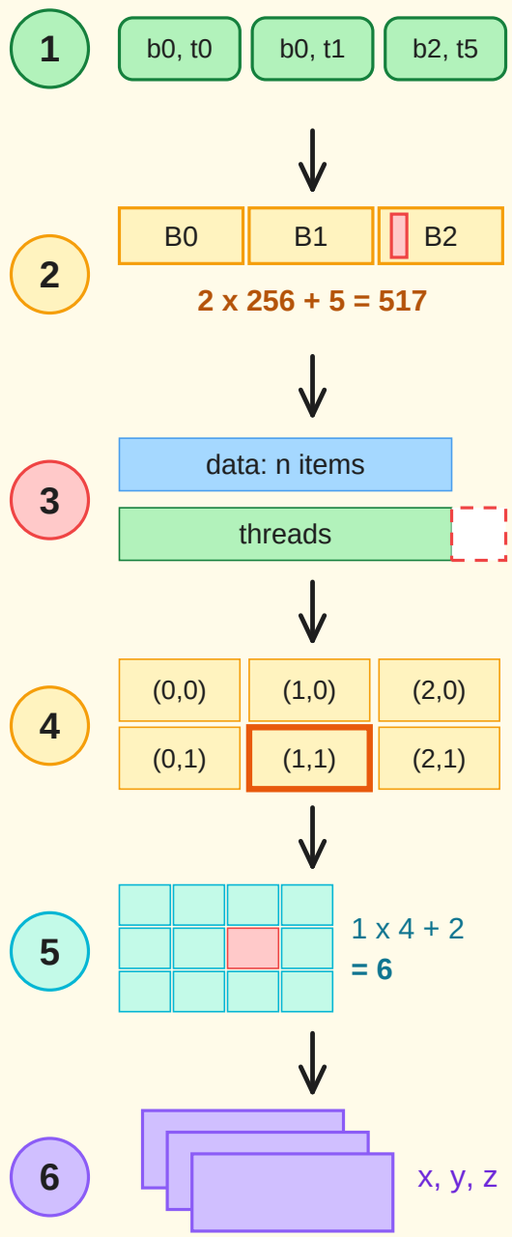
Lesson 6.1: Thread indexing in 1D, 2D and 3D

Every thread runs the same code. How each one works out which piece of the data is its own.

The whole GPU (map)



Zoomed in: from thread coordinates to data



Same code, different coordinates

Every thread runs the identical kernel. All that differs is two built-in numbers: which block it is in, and its spot there.

1D: one global index

index = block number x block size + spot. Thread 5 of block 2, with 256 per block, handles element 2 x 256 + 5 = 517.

Guard the end

Blocks come whole, so the grid is rounded up. The last block has threads past the data, and an if (i < n) switches them off.

2D: blocks and threads in a grid

For an image, blocks and threads each get an x and a y. The same formula runs once per axis to give pixel (x, y).

From (x, y) to a memory address

Memory is one long row, so pixel (x, y) lives at y * width + x. Neighbours in x are neighbours in memory.

3D, and which threads share a warp

Volumes add z the same way. Threads are numbered x first, so a warp's 32 threads run along x: neighbouring addresses.

Key idea: a thread combines its block number and its position in the block to find its own element

Lesson 6.1 notes

Thread indexing in 1D, 2D and 3D

In brief

All threads of a kernel run identical code. Each one gets two built-in coordinates, its block's number and its own position inside the block, and combines them into an index that says which element of the data belongs to it.

```
// 1D
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n) out[i] = f(in[i]);

// 2D image
int x = blockIdx.x * blockDim.x + threadIdx.x;
int y = blockIdx.y * blockDim.y + threadIdx.y;
if (x < width && y < height) {
    int i = y * width + x;     // where pixel (x, y) lives in memory
    out[i] = f(in[i]);
}
```

Beyond the board

- **Rounding the grid up.** The usual block count is (n + blockSize - 1) / blockSize in integer arithmetic, which is n / blockSize rounded up.
- **Warps in a 2D block.** With blocks of 16 by 16, one warp is two rows of 16 threads. Its addresses are two runs of 16 neighbours, which still coalesces well (Lesson 7.1).
- **The address formula matters for speed.** Indexing as y * width + x keeps a warp's reads next to each other. Swapping the roles of x and y would scatter them.
- **A CUDA block holds at most 1,024 threads.** Common choices are 128 or 256 in 1D and 16 by 16 in 2D.

Check yourself

An image is 1,920 by 1,080 and the blocks are 16 by 16. How many blocks are launched, and how many threads are switched off by the guard? Which pixel does thread (3, 7) of block (10, 2) handle, and where does it live in memory?

Answer on page 85.

Key terms

blockIdx: which block this thread is in.

threadIdx: this thread's position inside its block.

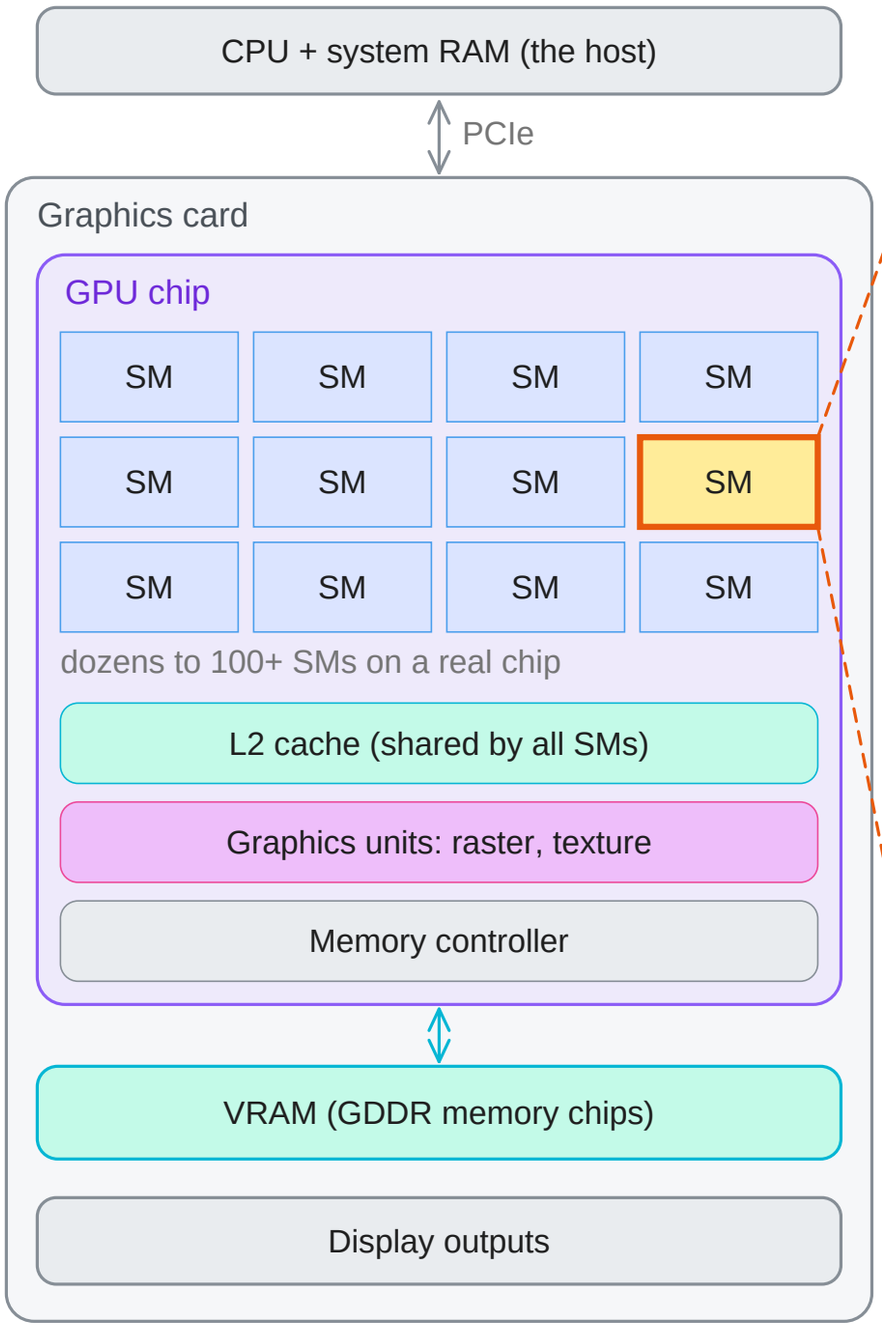
blockDim: how many threads a block has.

Row-major order: storing an image one row after another.

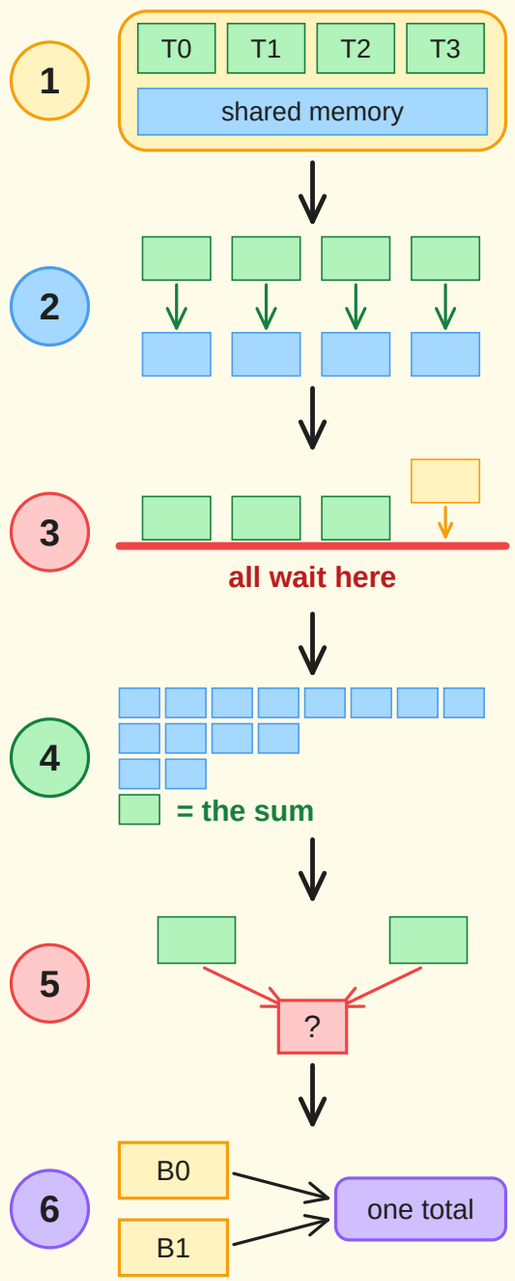
Lesson 6.2: Cooperation inside a block

How a block's threads share data safely, using one worked example: adding up the block's values.

The whole GPU (map)



Zoomed in: one block computing a sum



Threads in a block can work together

A block's threads sit on one SM and see the same shared memory. That lets them pass values to each other. Example: a sum.

Phase 1: write to your own slot

Each thread copies its input into its own position in shared memory. No two threads touch the same slot, so nothing clashes.

The barrier: wait for everyone

The block's warps run in no fixed order. A sync point makes every thread wait until all of them have reached it.

Phase 2: add in pairs, round by round

Half the threads each add a partner's value to their own. Barrier. Halve again. 256 values are summed in 8 rounds.

Without the barrier: a race

A thread may read a slot before its owner has written it. The answer is then wrong, but only sometimes, depending on timing.

Across blocks: no barrier exists

Blocks cannot wait for each other. Each writes its result to its own output slot, or uses an indivisible add on one total.

Key idea: threads in a block share memory safely by writing to their own slots and waiting at barriers between phases

Lesson 6.2 notes

Cooperation inside a block

In brief

Threads of one block can pass values to each other through shared memory. To do it safely they work in phases: every thread writes only its own slot, then all of them wait at a barrier before anyone reads. The example sums a block's 256 values in 8 rounds.

```
__global__ void blockSum(const float* in, float* out, int n) {
    __shared__ float s[256];
    int t = threadIdx.x;
    int i = blockIdx.x * blockDim.x + t;

    s[t] = (i < n) ? in[i] : 0.0f;     // phase 1: own slot
    __syncthreads();                   // everyone has written

    for (int half = blockDim.x / 2; half > 0; half /= 2) {
        if (t < half) s[t] += s[t + half];   // add a partner's value
        __syncthreads();                     // finish the round
    }
    if (t == 0) out[blockIdx.x] = s[0];      // one result per block
}
```

Beyond the board

- **Every thread must reach the same barrier.** A barrier inside an if that only some threads enter leaves the others waiting forever. Here the if guards the add, not the barrier.
- **Barriers cost time.** A warp that arrives early has to stop, which is the fourth kind of wait from Lesson 3.1. Use as few as the algorithm needs.
- **Races differ from run to run.** Without the barrier the result depends on which warp the scheduler happened to run first, so a bug may show up only now and then.
- **Across blocks.** An atomic add lets many threads update one total as an indivisible step.

Check yourself

In the code, how many barriers does each thread pass with 256 threads per block? When half is 64, how many threads do an add, and in how many of the block's 8 warps? What goes wrong if the first barrier is removed?

Answer on page 85.

Key terms

Barrier: a point where threads wait for each other.

__syncthreads(): CUDA's barrier for the threads of one block.

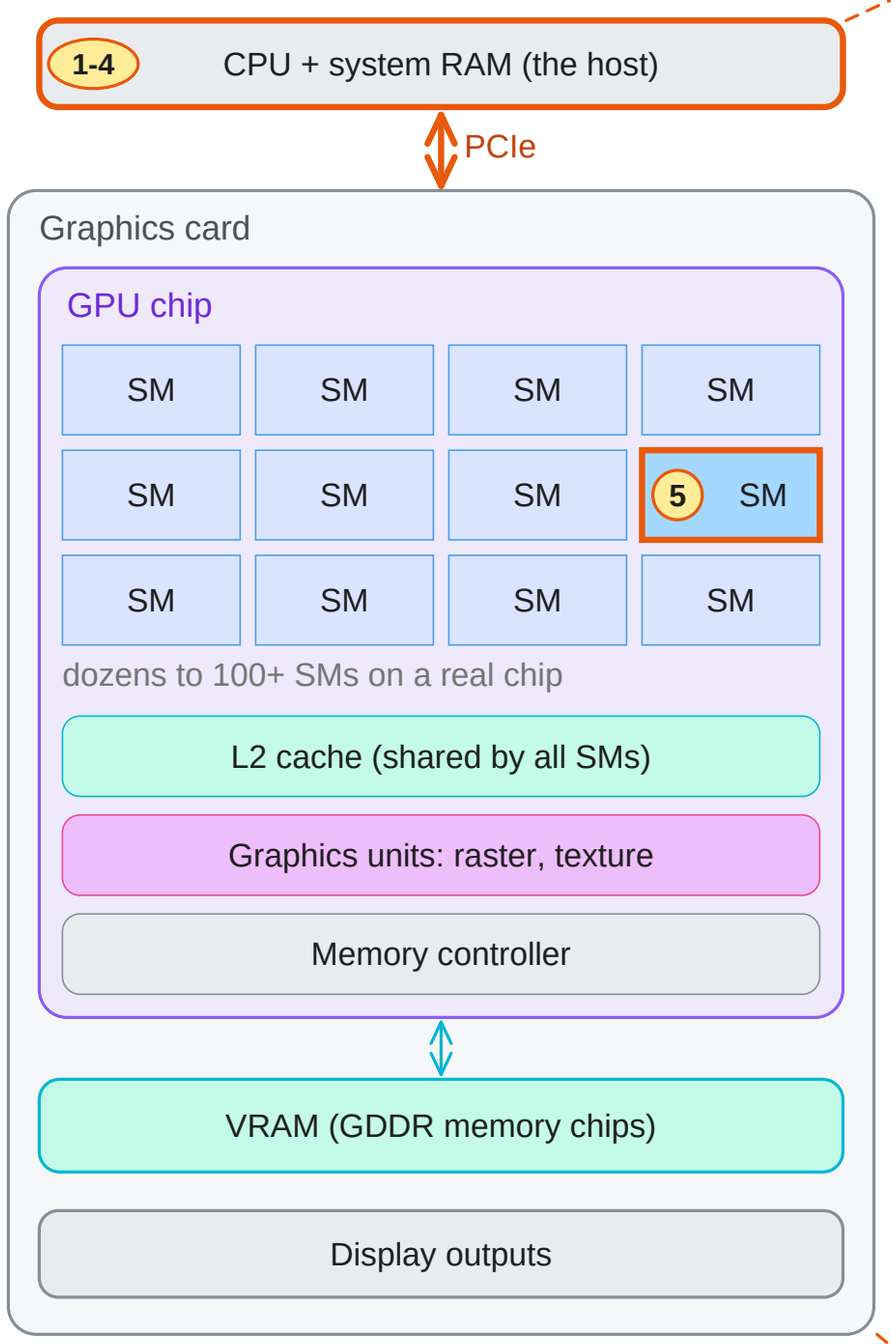
Race: a result that depends on timing.

Atomic operation: an update performed as one indivisible step.

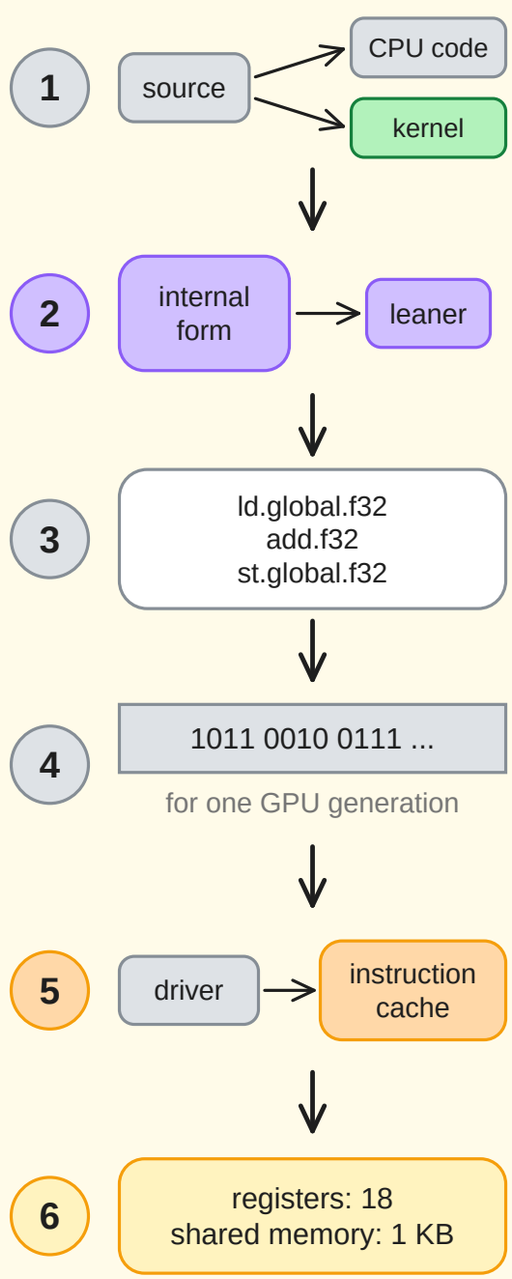
Lesson 6.3: From source code to machine instructions

What happens to a kernel between the text you write and the instructions an SM fetches.

The whole GPU (map)



Zoomed in: the path of a kernel



One file, two destinations

The compiler splits your source. Ordinary code becomes CPU instructions. Functions marked as kernels take the GPU route.

An intermediate form, then optimizing

The kernel is turned into a simplified internal form and improved there: dead code goes, short ifs become selects.

Portable assembly (PTX)

Next comes readable assembly for an idealized NVIDIA GPU: load, add, store. It is not tied to any one chip.

Real machine code

The last step targets one GPU generation and assigns the registers. This sets the registers per thread from Lesson 3.2.

Loading it onto the GPU

At launch the driver uploads the code. SMs fetch it through the instruction cache, as in Lesson 3.4's fetch stage.

What this means for you

You can ask the compiler what it chose: registers used, shared memory needed. Those numbers explain your occupancy.

Key idea: a kernel is compiled in stages, and the last stage fixes the registers per thread that occupancy depends on

Lesson 6.3 notes

From source code to machine instructions

In brief

A kernel is compiled in stages. It is split from the CPU code, optimized in an internal form, written out as portable assembly, and finally turned into machine code for one GPU generation. The driver then loads it onto the GPU. Below: c[i] = a[i] + b[i] in simplified PTX.

```
ld.global.f32   %f1, [%rd_a];     // load a[i]
ld.global.f32   %f2, [%rd_b];     // load b[i]
add.f32         %f3, %f1, %f2;    // add them
st.global.f32   [%rd_c], %f3;     // store into c[i]
```

Beyond the board

- **Why two assembly stages.** The portable form lets one compiled program run on GPUs that did not exist when it was built. The driver finishes the job for the chip it finds.
- **The last step can happen on your machine.** That is the pause the first time a program runs, and the "compiling shaders" screen in games. The result is usually cached.
- **Other ecosystems have the same idea.** Vulkan and OpenCL use an intermediate form called SPIR-V, and Direct3D uses DXIL.
- **With a framework you skip all of this.** PyTorch and similar libraries call kernels that were compiled in advance.

Check yourself

The compiler reports 48 registers per thread. On the SM from Lesson 3.2 (65,536 registers, a cap of 64 warps), how many warps fit and what is the occupancy? What if you got it down to 32?

Answer on page 85.

Key terms

Compiler: the program that turns source code into instructions.

Intermediate form: a simplified version used for optimizing.

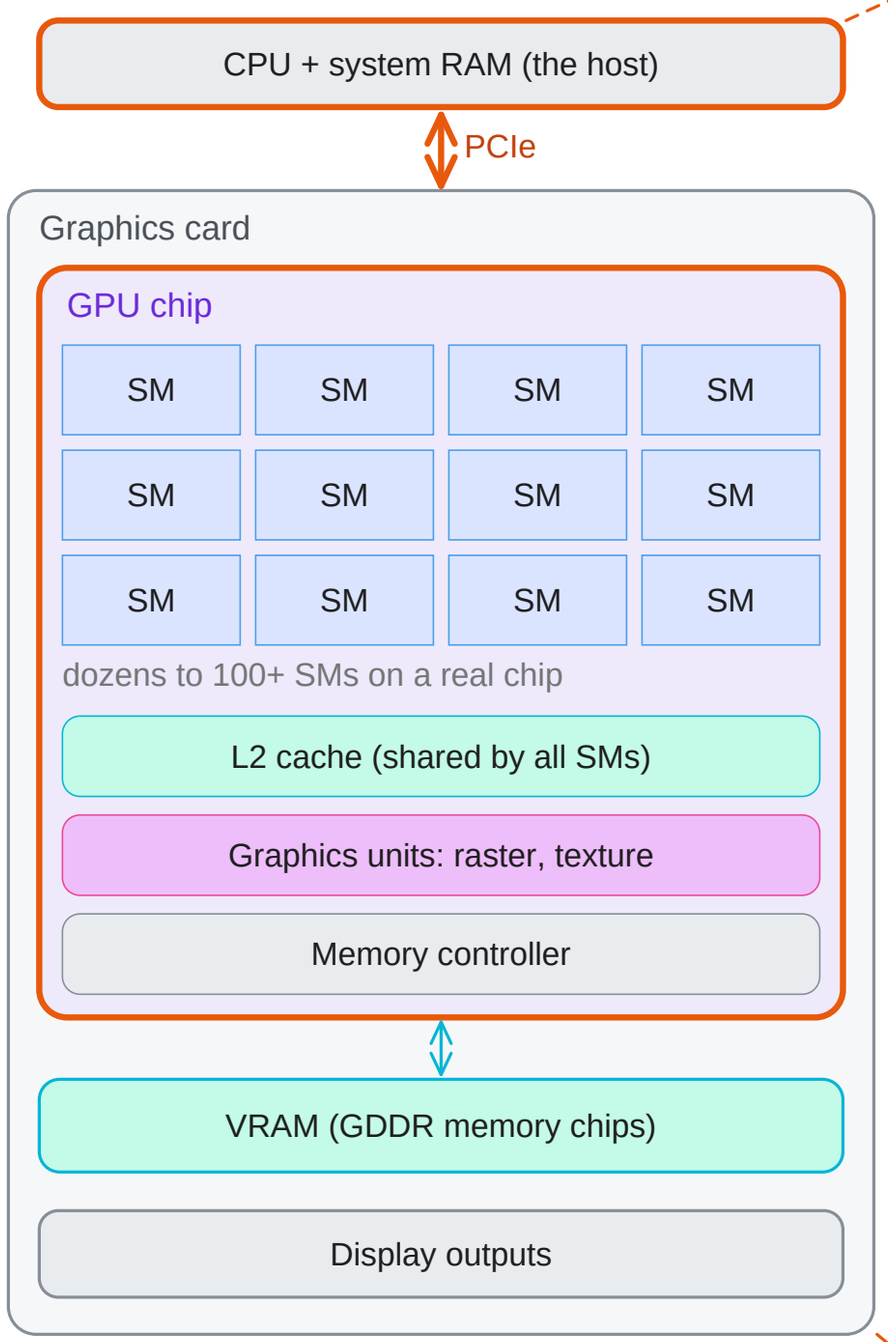
PTX: NVIDIA's portable assembly language.

Driver: the software on the host that controls the GPU.

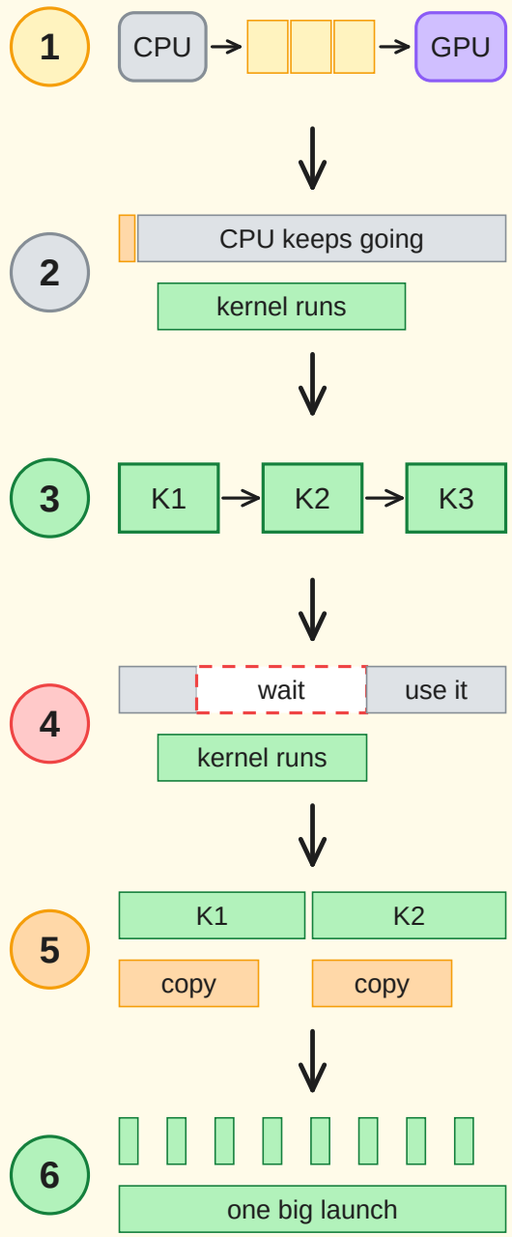
Lesson 6.4: Launches and asynchronous execution

What happens when the CPU starts a kernel, and why it does not wait for the answer.

The whole GPU (map)



Zoomed in: the CPU and GPU as two timelines



A launch is a message, not a call

The CPU does not run the kernel. It puts a command in a queue: which kernel, how many blocks, which arguments.

The CPU does not wait

The launch returns in microseconds. The CPU carries on with its own work while the GPU runs the kernel in the background.

One queue keeps its order

Commands in the same queue run one after another. Kernel 2 can safely read what kernel 1 wrote, with no extra code.

Waiting on purpose

To use a result on the CPU, you must wait for the GPU: a sync call, or a copy back that blocks until the queue is empty.

Several queues can overlap

Independent work goes in separate queues, called streams. A copy in one can run while a kernel in another computes.

Tiny launches waste the GPU

Every launch has a fixed cost. Thousands of tiny kernels leave the GPU idle between them. Fewer, bigger launches are faster.

Key idea: the CPU queues work for the GPU and moves on, so the two run side by side until you ask for a result

Lesson 6.4 notes

Launches and asynchronous execution

In brief

Launching a kernel does not run it. The CPU places a command in a queue and carries on, while the GPU works through the queue in order. The two only meet when the CPU asks for a result. Separate queues, called streams, let independent work overlap.

Beyond the board

- **The classic timing mistake.** If you start a clock, launch a kernel and stop the clock, you have measured the launch, not the kernel. Wait for the GPU first, then stop the clock.
- **Errors arrive late.** A fault inside a kernel is often reported by a later call, because that is the first time the CPU hears back from the GPU.
- **Frameworks hide the wait.** In PyTorch, printing a value or converting it to an ordinary number forces a wait. Doing that inside a loop quietly makes the whole loop synchronous.
- **More streams are not more GPU.** Streams only overlap work that uses different resources, such as a copy and a kernel. Two heavy kernels still share the same SMs.

Check yourself

A loop makes 10,000 launches. Each launch costs the CPU 10 microseconds, and each kernel runs for 5 microseconds. How long does the loop take, and how busy is the GPU? What if the work were one kernel?

Answer on page 85.

Key terms

Launch: the CPU's request to run a kernel.

Asynchronous: started now, finished later, without waiting.

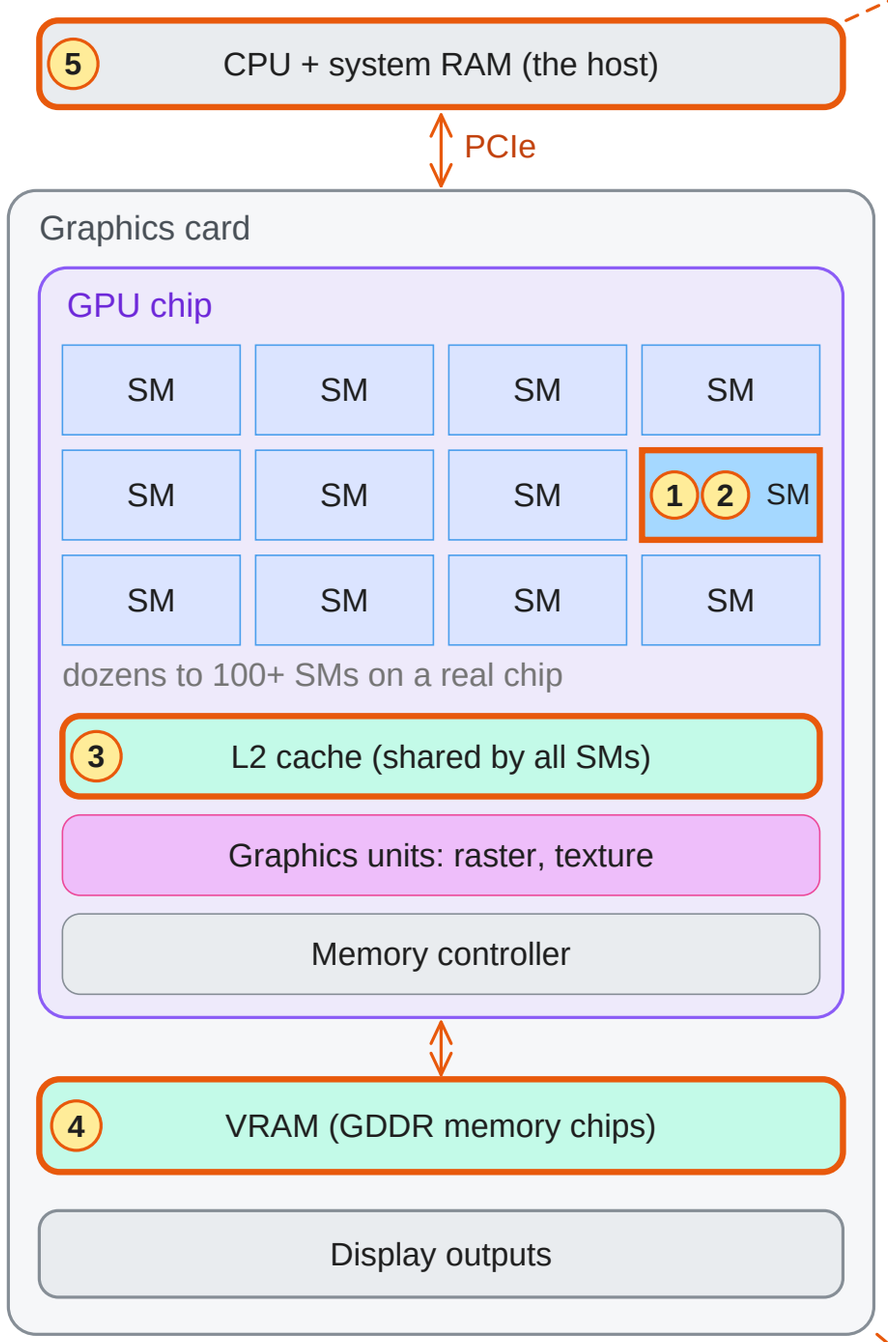
Synchronize: wait until the GPU has finished its queue.

Stream: one ordered queue of GPU work.

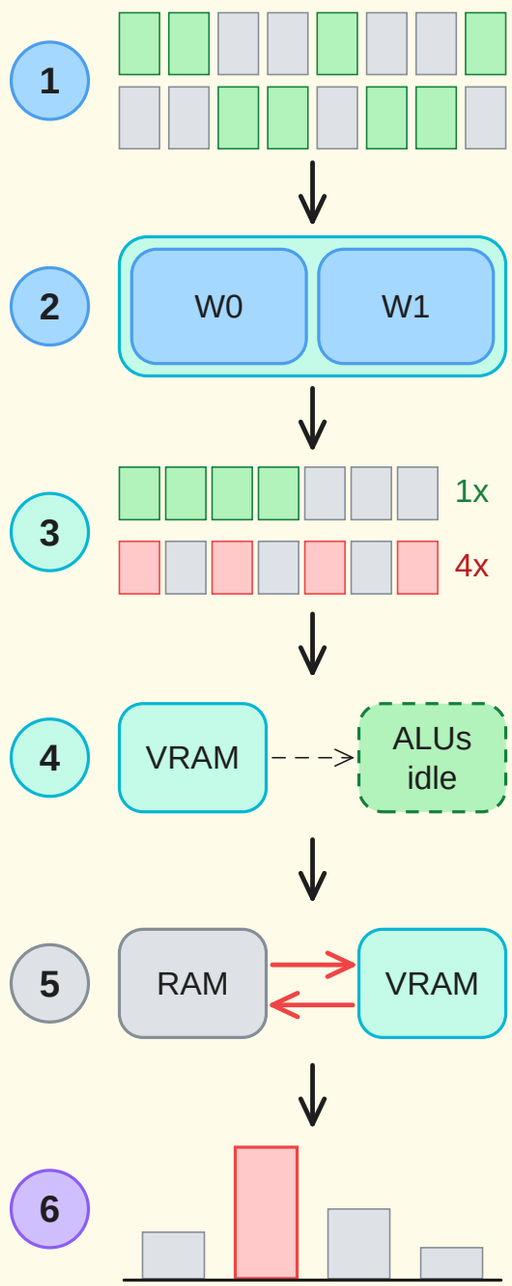
Lesson 7: Why GPU code runs fast or slow

Five ways GPU code loses speed, from inside the SM outward, and how to tell which one you have.

The whole GPU (map)



Zoomed in: where the time goes



Divergence: warp threads disagree

At an if/else, a warp runs each path in turn with the other lanes idle. Fix: keep similar items together, use math not ifs.

Low occupancy: too few warps fit

Heavy register or shared-memory use per thread leaves room for few warps, so the scheduler runs dry. Fix: slim each thread.

Scattered reads: no coalescing

32 threads reading neighbours = one wide fetch. Reading all over = up to 32 fetches. Fix: store data so thread i reads item i.

Memory-bound: ALUs wait on VRAM

Too little math per number loaded, so bandwidth sets the speed. Fix: reuse data in shared memory, merge passes together.

Chatty transfers: PCIe in the loop

Copying to and from the CPU every step swamps everything else. Fix: keep data on the GPU and copy in large batches.

Measure before you fix

A profiler shows which limit you are hitting. Fixing the wrong one changes nothing: only the tightest limit counts.

Key idea: a GPU is only as fast as its tightest limit, and that is usually memory or idle lanes, not arithmetic

Lesson 7 notes

Why GPU code runs fast or slow

In brief

GPU code loses speed in five typical ways, at increasing distances from the arithmetic units: divergence and low occupancy inside the SM, scattered reads and bandwidth limits in memory, and needless transfers across PCIe. Only the tightest limit matters, so measure before fixing.

Beyond the board

- **Check from the outside in.** In practice the costliest mistakes are usually the outer ones: needless transfers first, then memory access patterns, and only then divergence and occupancy.
- **Maximum occupancy is not always best.** More resident warps helps hide memory waits, but past a certain point they compete for the same cache and bandwidth. The goal is enough warps to keep the scheduler busy.
- **Every one of these traces back to an earlier lesson.** Divergence comes from the shared program counter (Lesson 2), occupancy from the fixed register file (Lesson 3), coalescing and bandwidth from the memory hierarchy (Lesson 4), and transfers from the PCIe link (Lessons 4 and 6).
- **Reading the small pictures.** Step 1 shows two passes over one warp with idle lanes in gray. Step 3 shows four neighbouring cells needing one fetch and four scattered cells needing four.

Check yourself

A kernel computes out[i] = sqrt(data[index[i]]), where index holds random positions. Which of the five limits would you suspect first, and why?

Answer on page 85.

Key terms

Divergence: idle lanes when a warp's threads branch apart.

Occupancy: how full an SM is with resident warps.

Coalescing: merging a warp's neighbouring reads into one fetch.

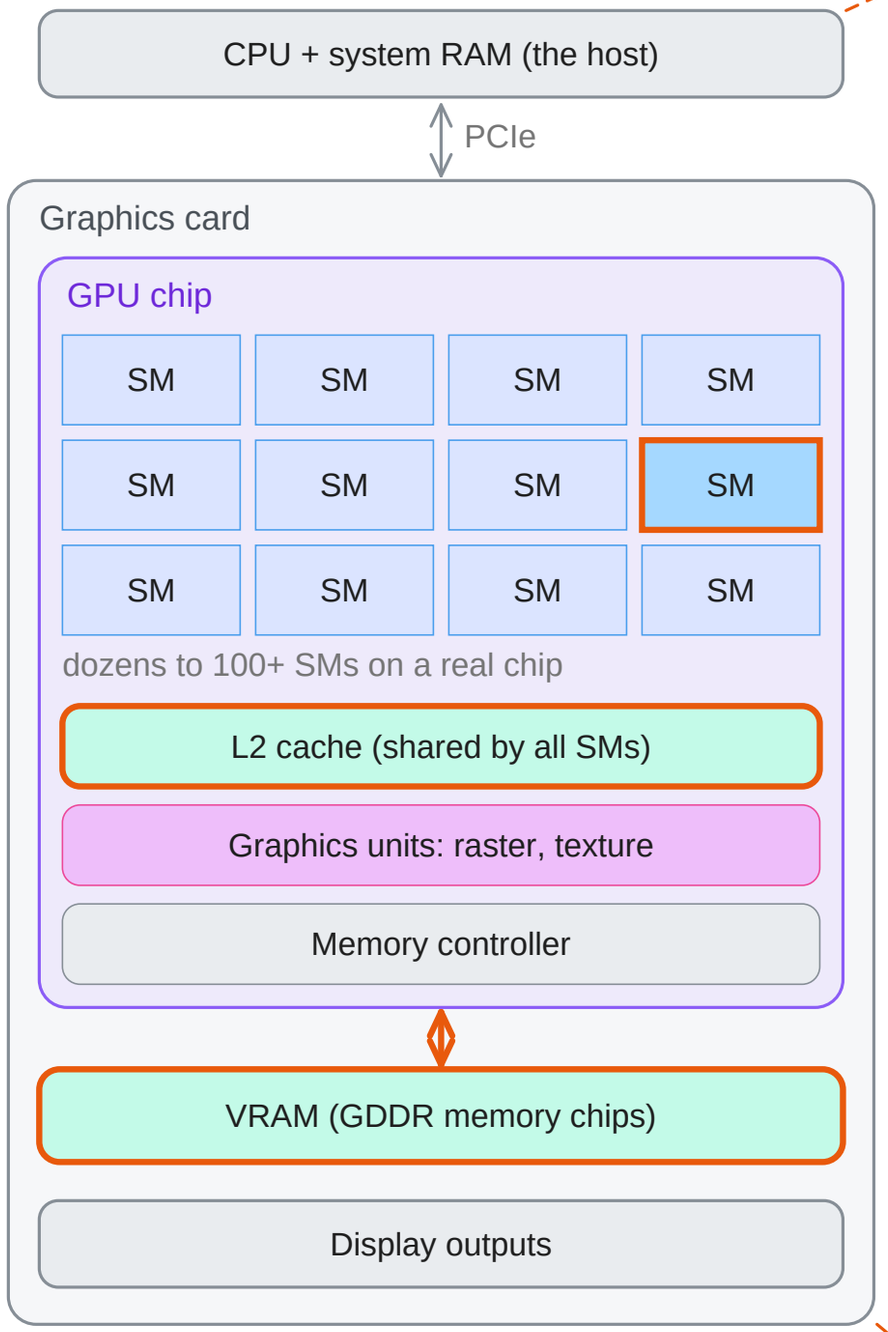
Memory-bound: limited by bandwidth, not arithmetic.

Profiler: tool that measures where the time goes.

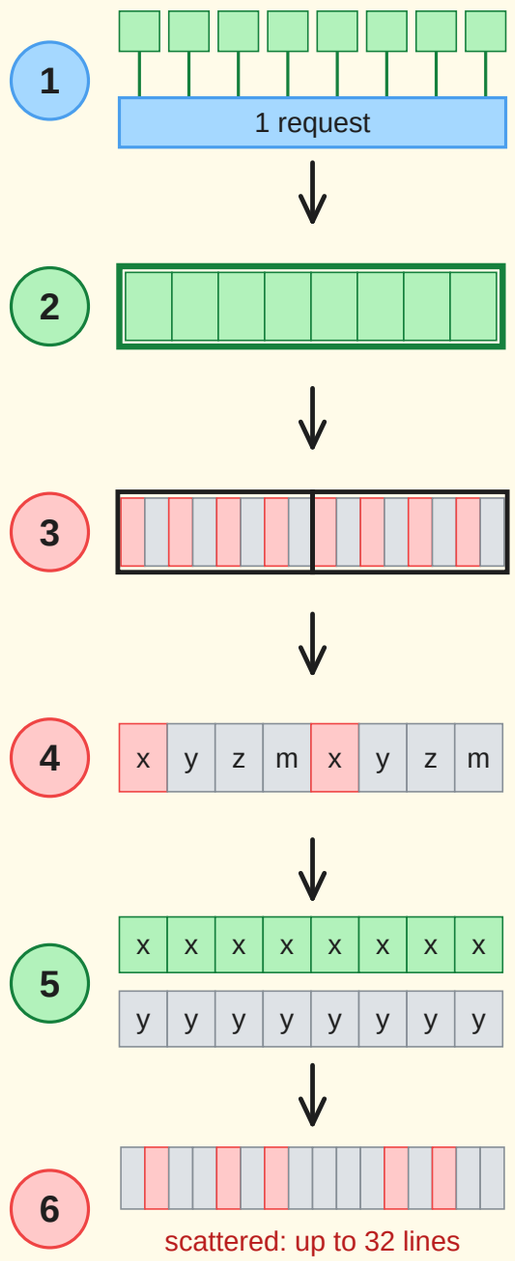
Lesson 7.1: Coalescing and data layout

How the way you arrange data in memory decides how many fetches each warp needs.

The whole GPU (map)



Zoomed in: one warp's memory request



Coalescing: merging a warp's reads

When a warp loads from memory, the hardware looks at all 32 addresses and fetches the fewest lines that cover them.

Best case: 32 neighbours, 1 line

Thread t reads element t. 32 numbers of 4 bytes are 128 bytes, exactly one line. One fetch serves the whole warp.

Strides spread the warp out

If thread t reads element 2 x t, the warp spans 2 lines and half of what arrives is never used. Stride 32: 32 lines.

Array of structures: a hidden stride

Storing each particle as x, y, z, mass together means a kernel that needs only x reads every 4th number: 4 lines.

Structure of arrays: the fix

Store all the x values together, all the y values together, and so on. Reading x is back to neighbours: 1 line per warp.

Indirect reads: the hard case

With data[index[i]], addresses can land anywhere: up to 32 lines per warp. Sorting or regrouping the data is the usual cure.

Key idea: arrange data so the 32 threads of a warp read neighbouring addresses, and one fetch serves them all

Lesson 7.1 notes

Coalescing and data layout

In brief

When a warp reads memory, the hardware merges its 32 addresses into as few line fetches as possible. Neighbouring addresses need one fetch; scattered ones need up to 32. How the program lays out its data decides which of those happens.

```
// Array of structures: x y z m  x y z m  x y z m ...
struct Particle { float x, y, z, mass; };
Particle p[N];

// Structure of arrays: x x x x ...  y y y y ...  z z z z ...
struct Particles { float x[N], y[N], z[N], mass[N]; };
```

Beyond the board

- **Array of structures is fine when you use everything.** If a kernel reads x, y, z and mass together, the first layout wastes nothing. The cost appears when it needs one field.
- **Wasted bytes cost bandwidth.** Lines fetched but not used still crossed the memory bus, so a memory-bound kernel slows down in proportion (Lesson 4.3).
- **Writes coalesce in the same way.** A warp storing to neighbouring addresses needs one write.
- **For indirect reads, sort.** Arrange the items so that things used together are stored together. Sorting particles by position is a common example.

Check yourself

Each particle is a 32-byte structure, and a kernel reads one 4-byte number from each. How many 128-byte lines does a warp fetch, and what share of the fetched bytes is used? What about the structure-of-arrays layout?

Answer on page 85.

Key terms

Coalescing: merging a warp's reads into few fetches.

Stride: the gap between addresses that threads use.

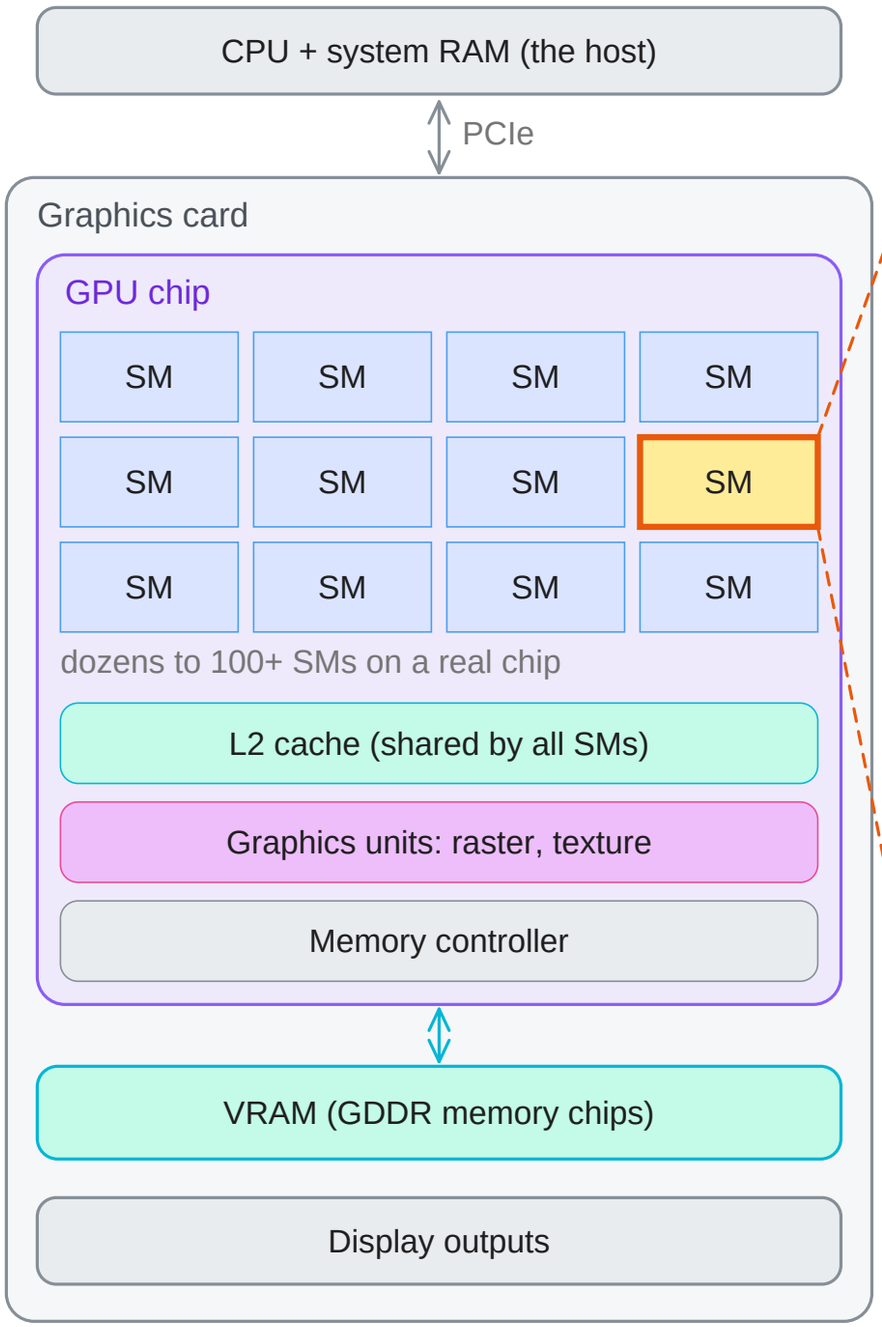
Array of structures: each item's fields stored together.

Structure of arrays: each field stored as its own array.

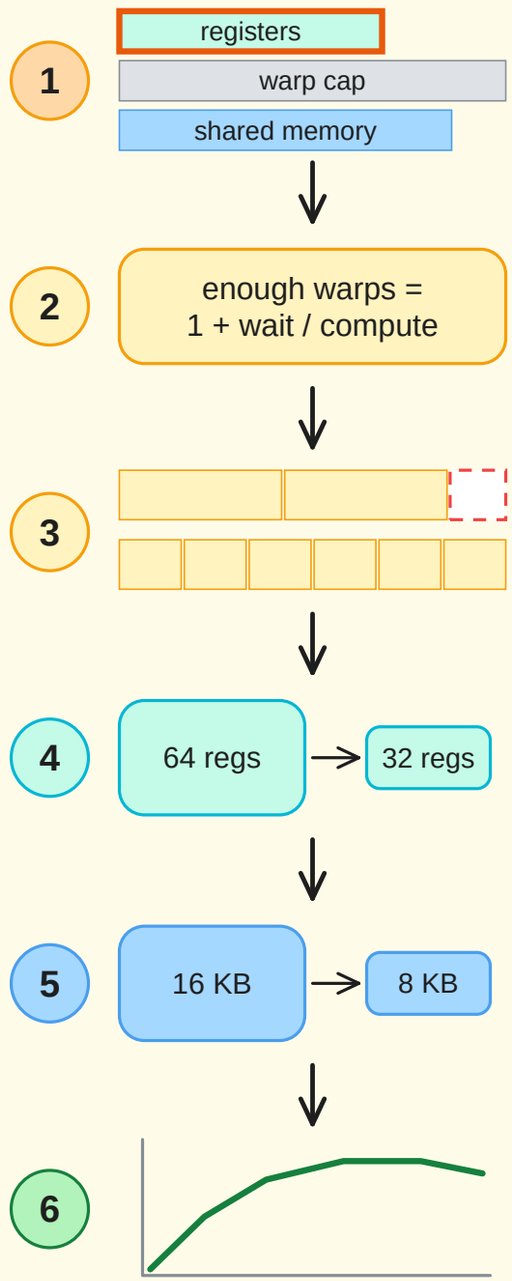
Lesson 7.2: Tuning occupancy

How block size, registers and shared memory trade against each other, and when more warps actually help.

The whole GPU (map)



Zoomed in: three levers on one SM



Recap: the tightest limit wins

Registers, the warp cap and shared memory each allow some number of warps. The smallest of the three is your occupancy.

How much is enough?

A core needs about 1 + wait / compute warps. Memory-heavy kernels want many. Math-heavy kernels are fine with few.

Lever 1: block size

SMs take whole blocks. Big blocks leave unused room. Sizes like 128 or 256, always a multiple of 32, usually pack well.

Lever 2: registers per thread

Simpler code with fewer values alive at once needs fewer registers. A compiler setting can also cap them, at a price.

Lever 3: shared memory per block

Reserve only what the block uses. Halving a tile from 16 KB to 8 KB can double the number of blocks that fit.

Measure, do not assume

Try a few block sizes and time each one. Speed usually climbs with occupancy and then flattens, and sometimes dips.

Key idea: aim for enough warps to keep the schedulers busy, then stop, and let measurement decide

Lesson 7.2 notes

Tuning occupancy

In brief

Occupancy is set by the tightest of three limits, and each one has a lever: block size, registers per thread, and shared memory per block. The aim is not the highest number but enough warps to cover the kernel's waits, about 1 + wait / compute.

Beyond the board

- **Capping registers has a price.** Values that no longer fit in registers are kept in memory and reloaded when needed. Past a point, that costs more than the extra warps gain.
- **Why speed can dip.** More warps also share the same L1 cache and the same bandwidth. For a memory-bound kernel, extra warps just queue.
- **Gains often flatten early.** Many kernels run about as fast at half occupancy as at full.
- **A fourth limit.** An SM also holds only so many blocks at once, typically 16 or 32, which matters when blocks are very small.
- **Profilers report it.** Look for achieved occupancy, which is what the SMs actually held.

Check yourself

An SM has 65,536 registers, a cap of 64 warps and room for at most 32 blocks. A kernel uses 40 registers per thread and no shared memory. What is the occupancy with blocks of 32 threads? Of 256? Of 1,024?

Answer on page 86.

Key terms

Occupancy: resident warps divided by the warp cap.

Block size: how many threads a block has.

Register spill: a value kept in memory for lack of registers.

Achieved occupancy: what a profiler measured, not the theory.

Lesson 7.3: Compute-bound or memory-bound?

One chart, called a roofline, shows which limit a kernel is under and how much faster it could possibly get.

The whole GPU (map)

CPU + system RAM (the host)

PCIe

Graphics card

GPU chip

SM	SM	SM	SM
SM	SM	SM	SM
SM	SM	SM	SM

green: sets the peak math rate

L2 cache (shared by all SMs)

Graphics units: raster, texture

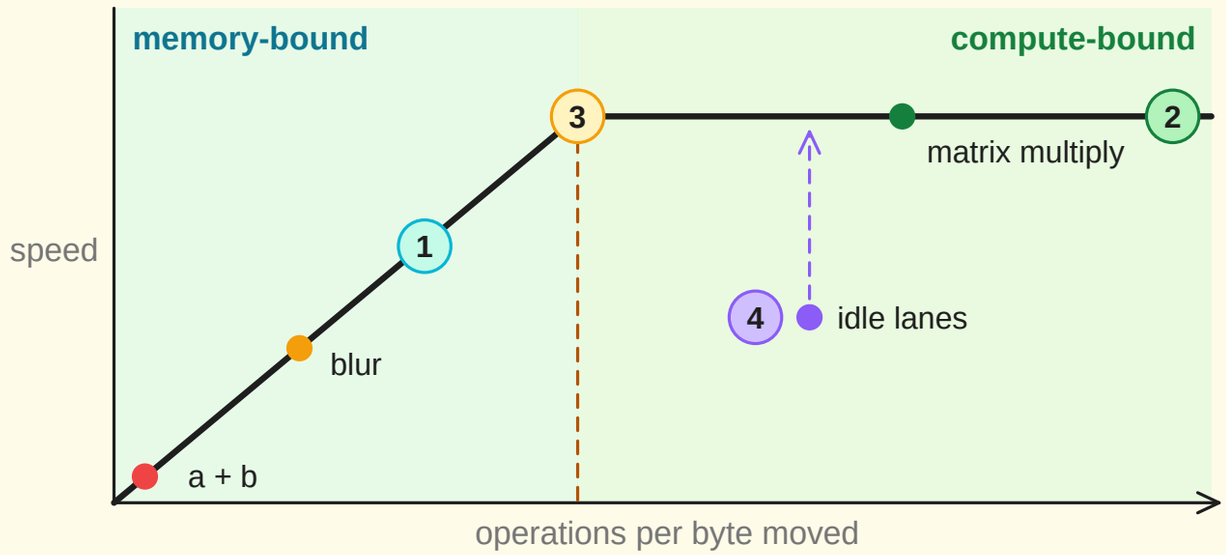
Memory controller

VRAM (GDDR memory chips)

teal: sets the bandwidth

Display outputs

The roofline: two limits on one chart



1 The sloped roof: memory sets the limit

Speed cannot exceed bandwidth x operations per byte. A kernel doing little math per byte sits under this slope, whatever lanes the chip has.

2 The flat roof: arithmetic sets the limit

Speed cannot exceed the chip's peak math rate either. Kernels doing a lot of math per byte hit this ceiling, and extra bandwidth would not help.

3 The balance point: where the two meet

Peak math rate / bandwidth. Example chip: 20 trillion / 500 billion bytes = 40 operations per byte. Left of it: memory-bound. Right: compute-bound.

4 Below the roof: something else is wrong

A kernel far under both roofs is losing speed to idle lanes: divergence, low occupancy, scattered reads or transfers. Fix those first (7.1, 7.2).

Key idea: find a kernel's operations per byte, see which roof is above it, and you know what kind of fix can help

Lesson 7.3 notes

Compute-bound or memory-bound?

In brief

A roofline chart puts a kernel's two limits on one picture. Along the bottom is how much arithmetic the kernel does per byte it moves. The sloped line is the limit set by bandwidth and the flat line is the limit set by the lanes. A kernel cannot be above either.

Beyond the board

- **The chart is a sketch.** Real rooflines are drawn on logarithmic axes, because kernels range from about 0.1 to thousands of operations per byte. An array add sits near 0.08.
- **Moving right.** Merging passes, reusing data in shared memory and using smaller number formats all raise the operations per byte, so the roof above the kernel is higher.
- **Raising the roof.** Tensor cores lift the flat part and stacked memory steepens the slope (Lessons 8.1 and 8.4). Either helps only the kernels underneath that part.
- **Why matrix multiply sits on the right.** Done in tiles that stay in cache, each number loaded is used in many multiplications.

Check yourself

On the example chip (20 trillion operations per second, 500 GB/s), a kernel moves 16 bytes and does 80 operations per element. What is its operations per byte, which roof is it under, and how many elements per second can it reach? What if it moved 8 bytes instead?

Answer on page 86.

Key terms

Roofline: a chart of a kernel's two speed limits.

Operations per byte: arithmetic done per byte moved through memory.

Compute-bound: limited by the lanes.

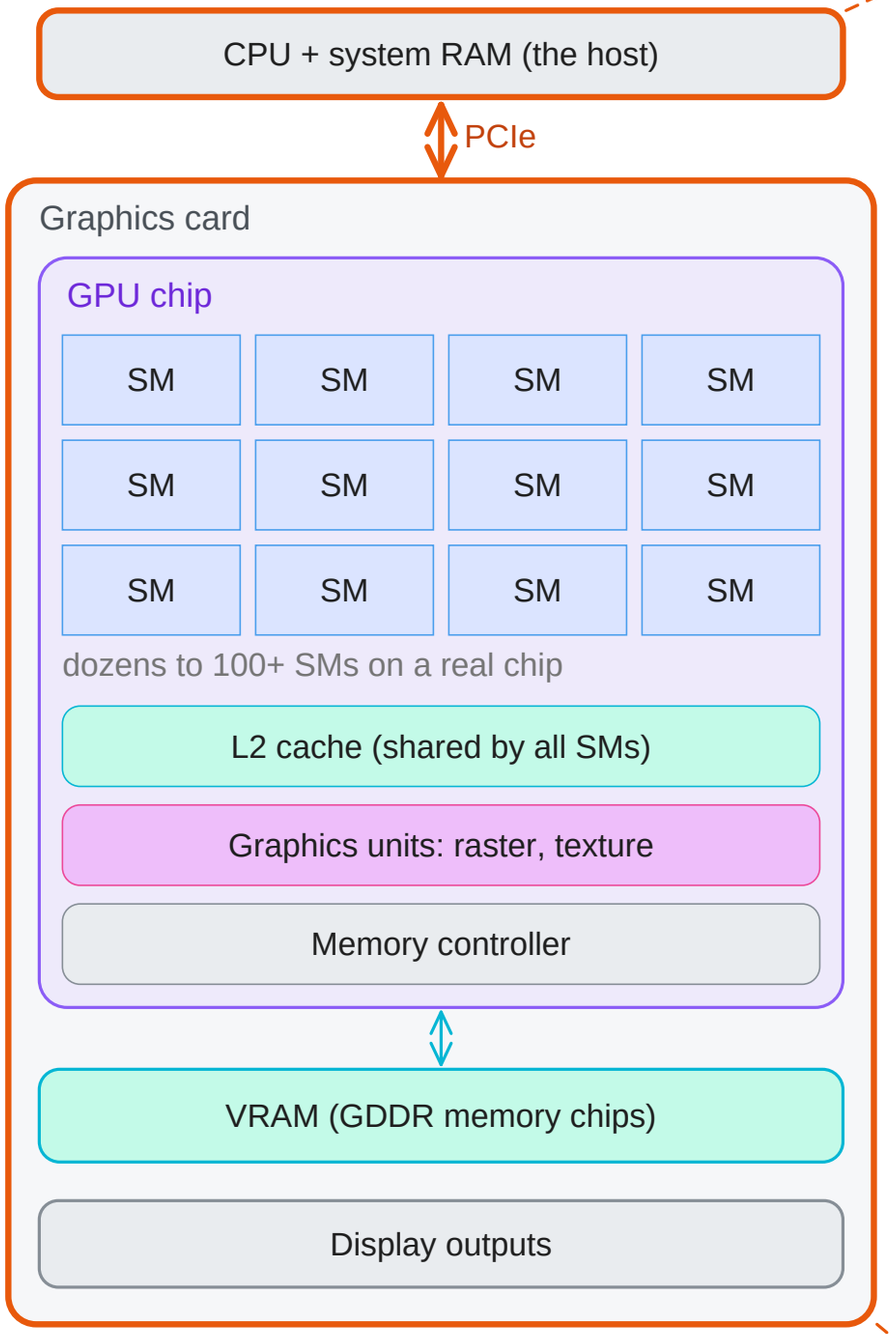
Memory-bound: limited by bandwidth.

Balance point: where the two limits meet.

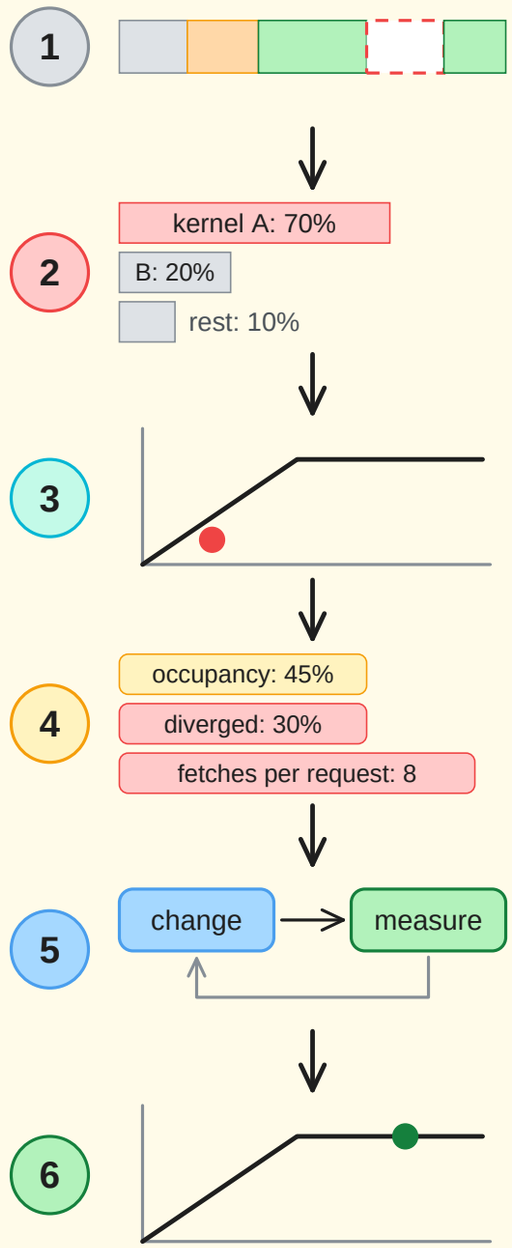
Lesson 7.4: Reading a profile

What the tools measure, and how to go from a table of numbers to the one change worth making.

The whole GPU (map)



Zoomed in: a profiling session, start to finish



First: is the GPU even busy?

A timeline shows CPU work, copies, kernels and gaps. If the GPU sits idle much of the time, no kernel tuning will fix that.

Rank the kernels by total time

One or two kernels usually take most of the time. Only they are worth work: a 2x gain on a 5% kernel saves 2.5%.

For the top kernel: which roof?

Compare the bytes it moves per second with the chip's bandwidth. Close to it: memory-bound. Far below: look at the lanes.

Check the idle-lane counters

Profilers report achieved occupancy, how often warps diverge, and how many fetches each memory request needed.

One change, then measure again

Change one thing and rerun the profile. Keep it only if the numbers improve. The bottleneck often moves somewhere new.

Know when to stop

When the top kernel sits near its roof, only a different algorithm or different hardware will make it faster.

Key idea: measure first, fix the largest cost, measure again, and stop when the kernel reaches its roof

Lesson 7.4 notes

Reading a profile

In brief

A profile shows where the time actually goes. Read it from the outside in: first whether the GPU is busy at all, then which kernels take the most time, then which limit the top kernel is under. Change one thing, measure again, and stop when the kernel is near its roof.

Beyond the board

- **"Utilization" says less than it seems.** The percentage most monitoring tools show only means some kernel was running. A GPU can read 100% while most of its lanes are idle.
- **Profile the real workload.** Use realistic data sizes, and ignore the first run, which includes compiling and loading.
- **The order is outside-in.** Transfers and gaps first, then memory behaviour, then divergence and occupancy. It is the same order as Lesson 7's five limits, reversed.
- **Tools.** NVIDIA's Nsight Systems shows the timeline and Nsight Compute shows the counters for one kernel. PyTorch and other frameworks have their own profilers built in.

Check yourself

A profile shows kernel A at 70% of the run time, kernel B at 20% and everything else at 10%. What is the overall speedup from making B twice as fast? From making A twice as fast? What are the shares after the second change?

Answer on page 86.

Key terms

Profiler: a tool that measures where the time goes.

Timeline: what the CPU and GPU were doing, moment by moment.

Counter: a hardware tally, such as fetches or diverged branches.

Bottleneck: the one limit currently setting the speed.

Lesson 8: What modern GPUs add

Where today's GPUs go beyond the basic design. Each addition targets one workload or one bottleneck.

The whole GPU (map)

CPU + system RAM (the host)

PCIe

Graphics card

GPU chip

SM SM SM SM

SM SM SM 1 2 SM

SM SM SM SM

dozens to 100+ SMs on a real chip

L2 cache (shared by all SMs)

3 Graphics units: raster, texture

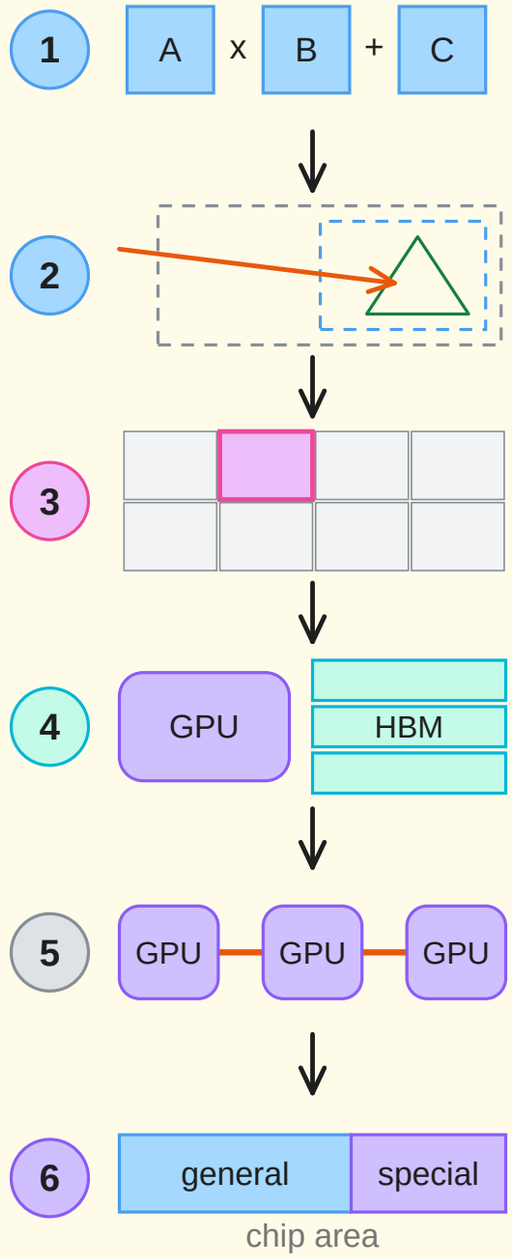
Memory controller

4 VRAM (GDDR memory chips)

Display outputs

5 Other GPUs (fast direct links)

Zoomed in: what modern GPUs add



Tensor cores: matrix math in one step

A unit that multiplies and adds small matrices at reduced precision. AI is mostly matrix math, so it gets many times faster.

Ray-tracing units: find what a ray hits

Fixed hardware walks a tree of bounding boxes and tests ray-triangle hits. The SMs then shade whatever each ray found.

Tile-based rendering: the mobile way

Phone GPUs split the screen into small tiles and finish each one in on-chip memory. Less memory traffic, less power.

Stacked memory: more bandwidth

Data-center GPUs place stacked memory (HBM) right beside the chip for several times the bandwidth of ordinary VRAM.

GPU-to-GPU links: many act as one

Large jobs span many GPUs. Dedicated links between them are far faster than PCIe, so data moves without going through the CPU.

The trade-off behind all of these

Each special unit is chip area that idles when its workload is not running. Designers bet on which workloads matter most.

Key idea: modern GPUs add specialized hardware for the workloads that matter most, at the cost of area and generality

Lesson 8 notes

What modern GPUs add

In brief

Modern GPUs add specialized hardware on top of the basic design. Each addition targets one workload or one bottleneck, and each costs chip area that sits idle when that workload is not running.

Beyond the board

- **Tensor cores** speed up the arithmetic itself, but only for matrix-shaped work. They make the memory bottleneck from Lesson 4 even more visible, because the math now finishes faster than data can arrive.
- **Ray-tracing units** exist because following a ray through a scene is branchy and scattered, which is exactly the kind of work SMs handle badly (Lesson 7). Fixed hardware takes over that part.
- **Tile-based rendering** is the bandwidth lesson applied to phones: keep the work in small, fast on-chip memory and touch slow memory as rarely as possible.
- **Stacked memory and GPU-to-GPU links** both widen the slowest paths on the map, which are VRAM and the route off the card.
- **Ideas, not products.** Model names, speeds and capacities change with every hardware generation; the trade-off between general and specialized area does not.

Course recap

You started with why a GPU trades single-thread speed for parallel volume. You can now trace a program from a kernel on the CPU, through blocks and warps on an SM, out to memory and back, and explain what limits it along the way.

Key terms

Tensor core: unit for small matrix multiply-accumulate.

Bounding-box tree: the structure a ray is tested against.

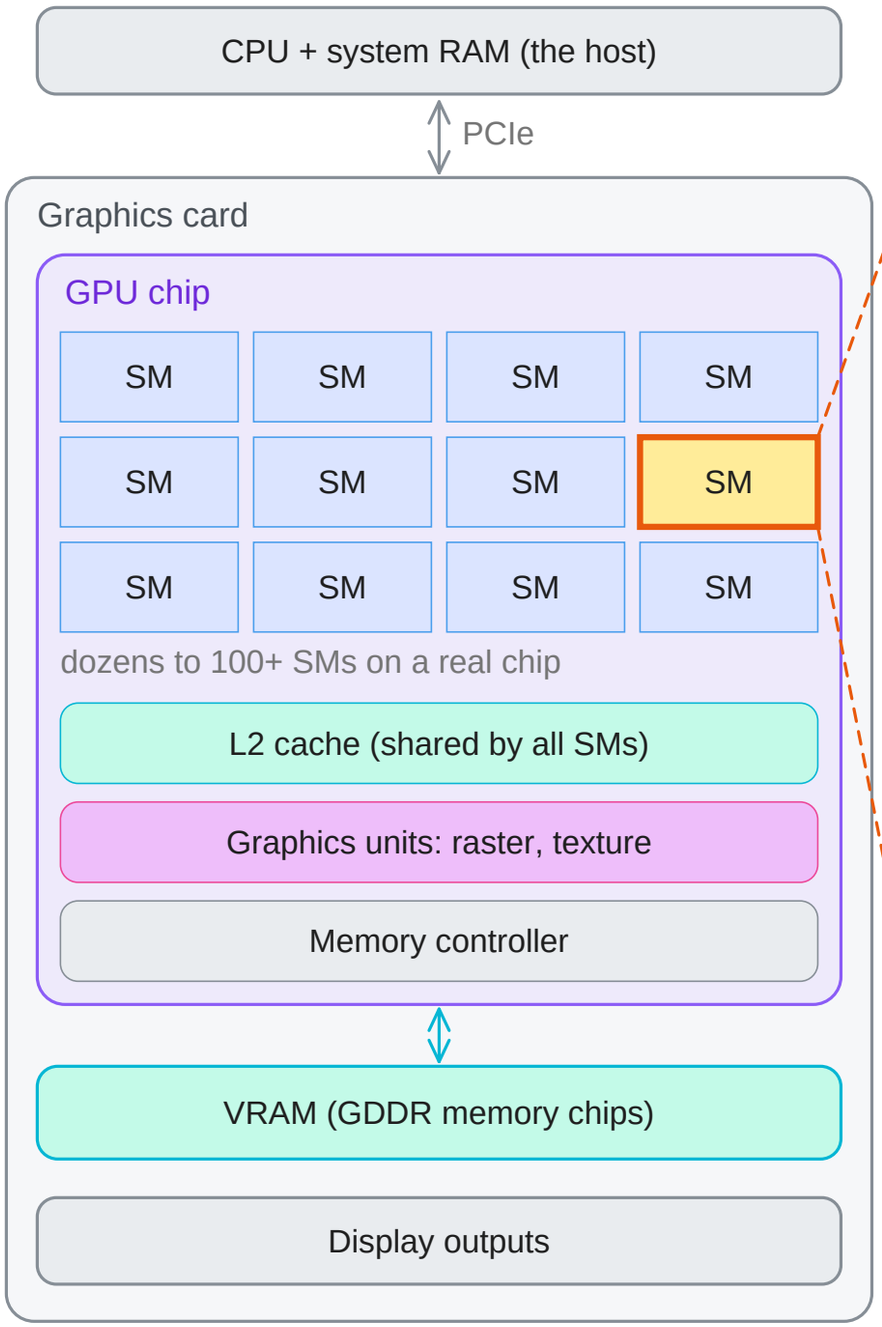
Tile-based rendering: finishing the screen one small tile at a time.

HBM: high-bandwidth memory, stacked beside the chip.

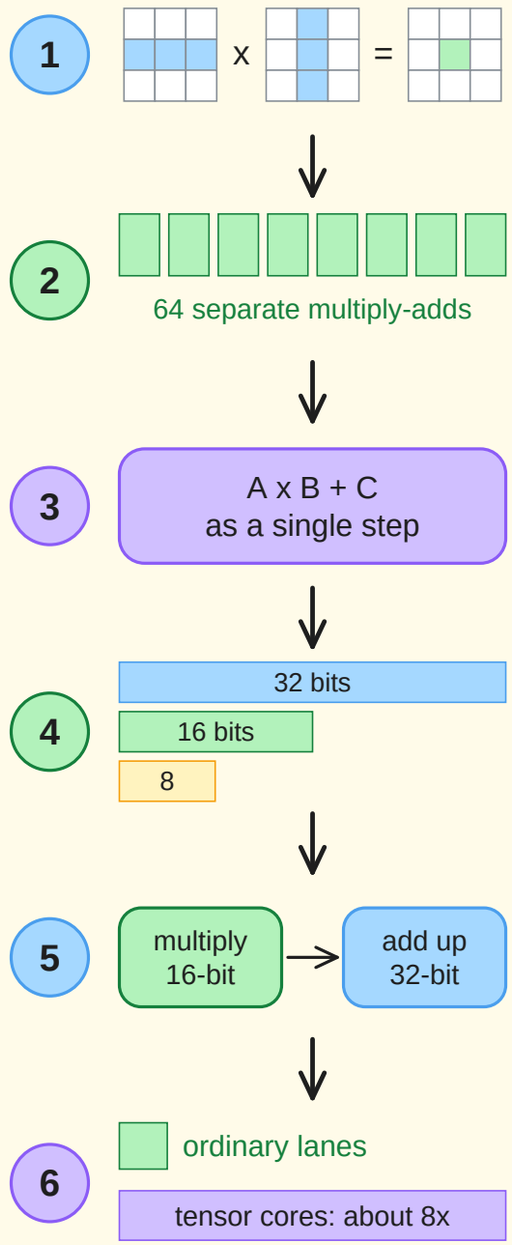
Lesson 8.1: Tensor cores and mixed precision

Why a specialized matrix unit beats the ordinary lanes, and what you give up to use it.

The whole GPU (map)



Zoomed in: one matrix multiply, two ways



The job: matrix multiplication

Neural networks are mostly matrix math. Each output number is a row times a column: many multiplies, added up.

On ordinary lanes: one step at a time

A 4x4 by 4x4 product is 64 multiply-adds. Each one is fetched, read from registers, run in a lane and written back.

A tensor core does it in one go

Multipliers and adders are wired together into one unit for a small matrix. No per-step fetching or register traffic.

Smaller numbers: 16 or 8 bits

A 16-bit multiplier is far smaller than a 32-bit one, so many more fit. The data is half the bytes too, easing memory.

Mixed precision keeps it accurate

Multiply in 16 bits but add the results up in 32. Training also keeps a 32-bit master copy of what it is learning.

The payoff, and the catch

Matrix work runs roughly 8 times faster than on the lanes. Nothing else speeds up, and memory becomes the limit sooner.

Key idea: a tensor core trades generality and precision for many more multiply-adds per unit of chip area

Lesson 8.1 notes

Tensor cores and mixed precision

In brief

A tensor core is a unit built for one job: multiplying small matrices and adding the result. It gets its speed from wiring the multipliers and adders together and from using smaller 16-bit or 8-bit numbers. Mixed precision keeps the answers accurate enough.

Beyond the board

- **The size of the gain.** On NVIDIA's V100, the ordinary lanes peak at about 15 trillion operations per second and the tensor cores at about 125 trillion.
- **What 16 bits can hold.** About 3 to 4 significant digits, against 7 for 32 bits. Training rescales values so the small ones are not lost. Running a trained model often gets by on 8.
- **What it does to the roofline.** The flat roof rises about 8 times, so the balance point moves 8 times to the right. Kernels that were compute-bound become memory-bound (Lesson 7.3).
- **You rarely call them yourself.** Libraries for matrix math and neural networks use tensor cores automatically when the number format allows it.
- **Other chips have the same idea.** It appears under names such as matrix units and neural engines.

Check yourself

A chip does 20 trillion operations per second on its lanes and 160 trillion on its tensor cores, with 500 GB/s of bandwidth. What are the two balance points? A matrix kernel does 60 operations per byte. Which roof is it under each way, and how much faster is it really?

Answer on page 86.

Key terms

Tensor core: a unit for small matrix multiply-and-add.

Half precision: 16-bit floating-point numbers.

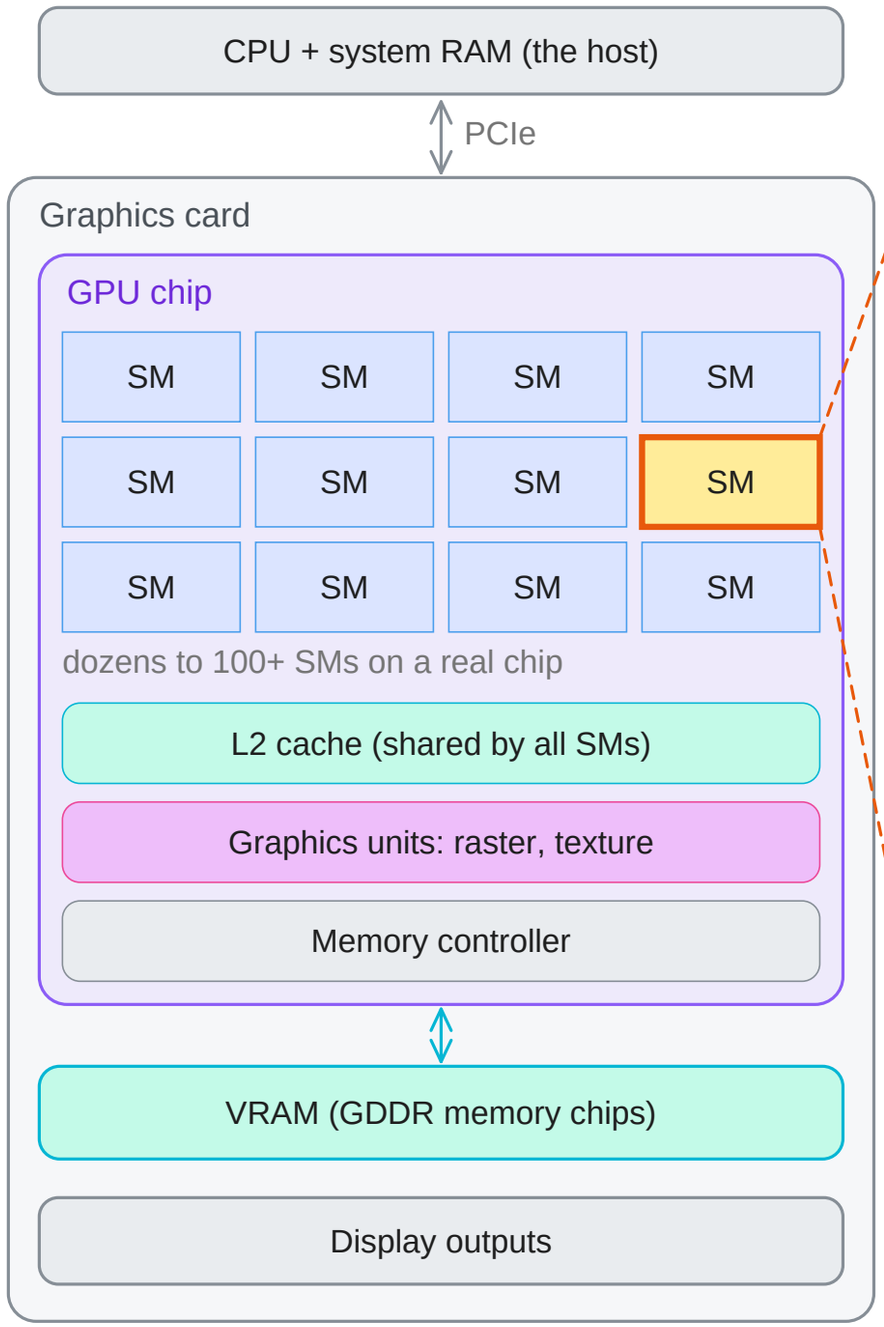
Mixed precision: multiplying in 16 bits, accumulating in 32.

Inference: running a trained model, as opposed to training it.

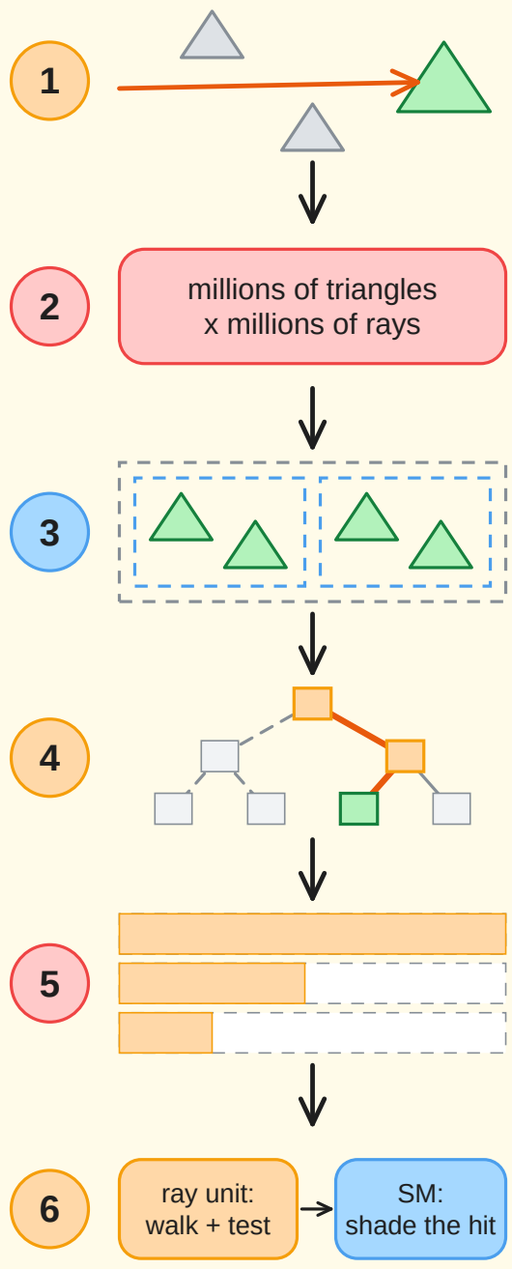
Lesson 8.2: How a ray finds its triangle

The tree of boxes behind ray tracing, and why walking it got its own hardware.

The whole GPU (map)



Zoomed in: one ray through the scene



The question: what does this ray hit?

Reflections, shadows and bounced light all need it: follow a ray from a point and find the first triangle in its way.

Testing everything is hopeless

A scene has millions of triangles, and a frame needs millions of rays. Every ray against every triangle is far too slow.

The fix: boxes inside boxes

Nearby triangles are wrapped in a box. Boxes are wrapped in bigger boxes, up to one box around the whole scene.

Walking the tree

If a ray misses a box, everything inside is skipped. A million triangles take about 20 levels, so a few dozen tests.

Why SMs are bad at this

Each ray takes its own path: different branches, scattered memory reads, and a different length. Warps fall apart.

Dedicated ray-tracing units

Fixed hardware walks the tree and does the box and triangle tests. The SMs only shade the point each ray ended up at.

Key idea: a tree of boxes turns millions of tests into dozens, and fixed hardware walks it so the SMs do not have to

Lesson 8.2 notes

How a ray finds its triangle

In brief

Ray tracing asks one question millions of times: what is the first triangle this ray hits? A tree of nested boxes answers it in a few dozen tests instead of millions. Walking that tree is branchy and scattered, so modern GPUs do it in dedicated units.

Beyond the board

- **The tree has a name.** It is a bounding volume hierarchy, or BVH. Building a good one takes time, and it has to be updated when things move.
- **Two levels for moving scenes.** Each object has its own tree, and a top-level tree holds the objects. Moving an object changes only the top level.
- **Games use very few rays.** Often about one ray per pixel for each effect. The result is noisy, and a neural network cleans it up, which is a job for tensor cores.
- **It is a hybrid.** Most games still rasterize what the camera sees directly and use rays for reflections, shadows and bounced light.
- **Where the units sit.** On NVIDIA GPUs each SM has a ray-tracing unit, and the tree itself lives in VRAM.

Check yourself

A scene has 16 million triangles in a tree where every box holds two smaller ones. About how deep is the tree? If a ray tests 3 boxes per level and then 4 triangles, how many tests is that, and how does it compare with testing everything?

Answer on page 86.

Key terms

Ray: a line followed from a point in one direction.

Bounding box: a box drawn around a group of triangles.

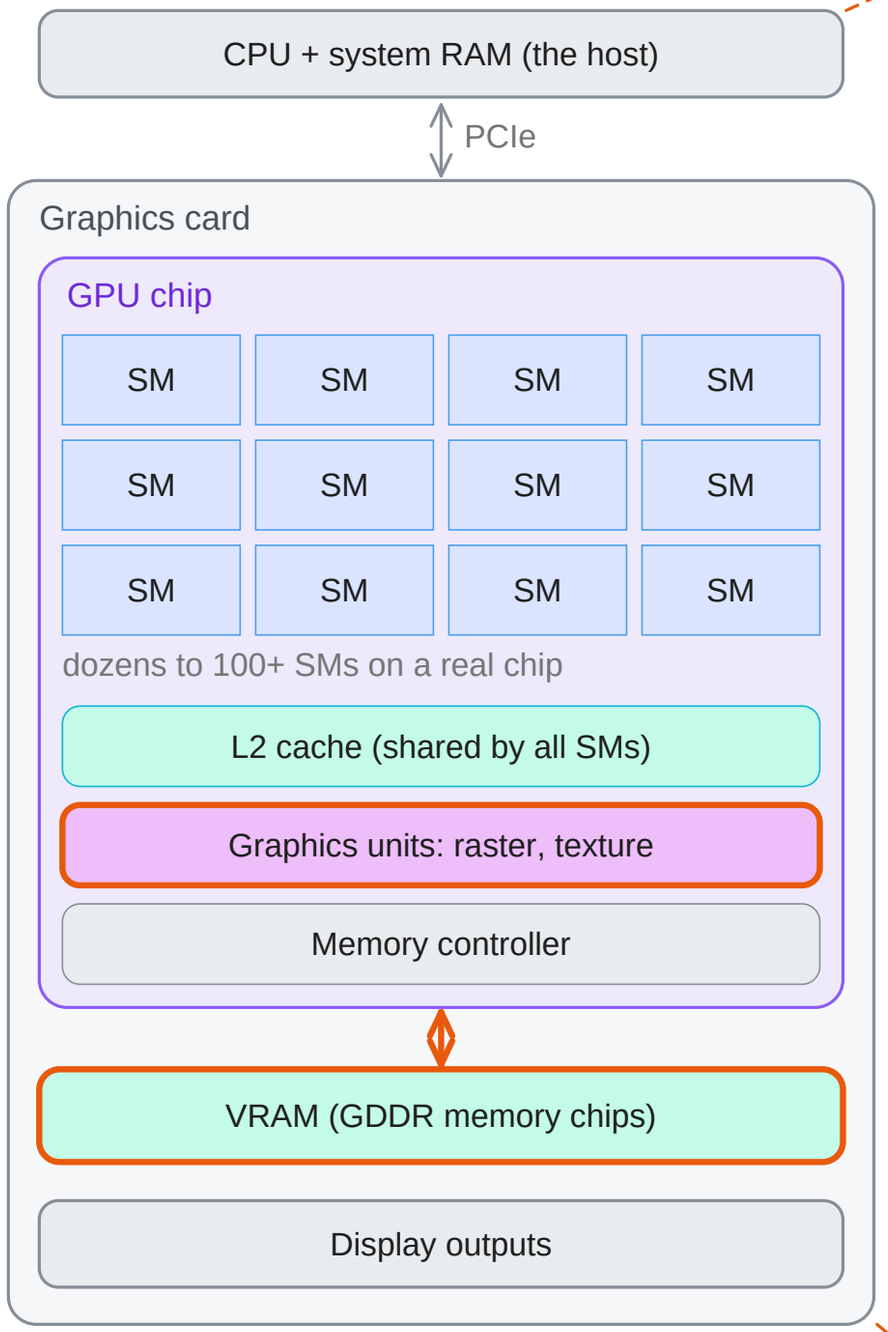
BVH: bounding volume hierarchy, the tree of boxes.

Traversal: walking the tree to find what a ray hits.

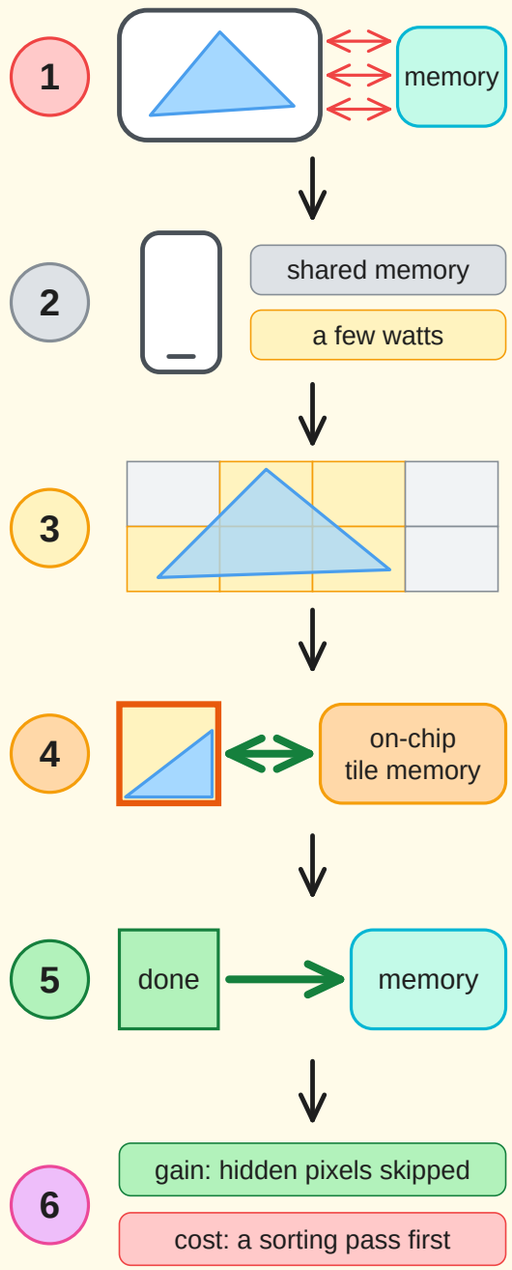
Lesson 8.3: Tile-based rendering

How phone GPUs draw a frame in small pieces to save memory traffic and power.

The whole GPU (map)



Zoomed in: one frame, tile by tile



The desktop way: draw as you go

Each triangle is rasterized as it arrives. Its fragments read and write depth and color in memory, all over the screen.

Why that hurts on a phone

A phone GPU shares the system's memory, with a fraction of a desktop's bandwidth, and must run on a few watts.

Step 1: sort triangles into tiles

The screen is cut into small tiles, such as 32 x 32 pixels. First, every triangle is listed under each tile it touches.

Step 2: finish one tile at a time

All of a tile's triangles are drawn into a small memory on the chip itself. Depth and color never leave it while it works.

Step 3: write each pixel out once

Only the finished colors go to main memory, in one burst per tile. The depth values are usually just thrown away.

What it gains and what it costs

Seeing a tile's triangles together lets the chip skip hidden pixels entirely. The price is the sorting pass up front.

Key idea: finish each small tile in fast on-chip memory and touch main memory once per pixel

Lesson 8.3 notes

Tile-based rendering

In brief

A desktop GPU draws each triangle as it arrives, reading and writing depth and color in memory all over the screen. A phone GPU sorts the triangles into small screen tiles first, then finishes one tile at a time in memory on the chip and writes each pixel out once.

Beyond the board

- **A map for a different machine.** A phone has no separate VRAM. Its GPU shares the system's memory, so the highlighted path here is the route to that shared memory.
- **Who does this.** The GPUs in nearly all phones and tablets, from Arm, Qualcomm, Imagination and Apple, including Apple's laptop chips.
- **It is Lesson 4 applied to graphics.** The tile memory plays the role of shared memory: load once, work locally, write out once.
- **Hidden surfaces can be skipped entirely.** Because all of a tile's triangles are known before shading starts, some designs shade only what ends up visible.
- **What breaks it.** Switching what is being drawn to in the middle of a frame forces tiles to be written out and read back.

Check yourself

A phone screen is 2,400 by 1,080 pixels and tiles are 32 by 32. How many tiles is that? How much on-chip memory does one tile need at 8 bytes per pixel (color and depth)? With 3 surfaces per pixel, how many bytes are written to main memory per frame, drawing as you go and with tiles?

Answer on page 86.

Key terms

Tile: a small square of the screen, such as 32 by 32 pixels.

Binning: listing each triangle under the tiles it touches.

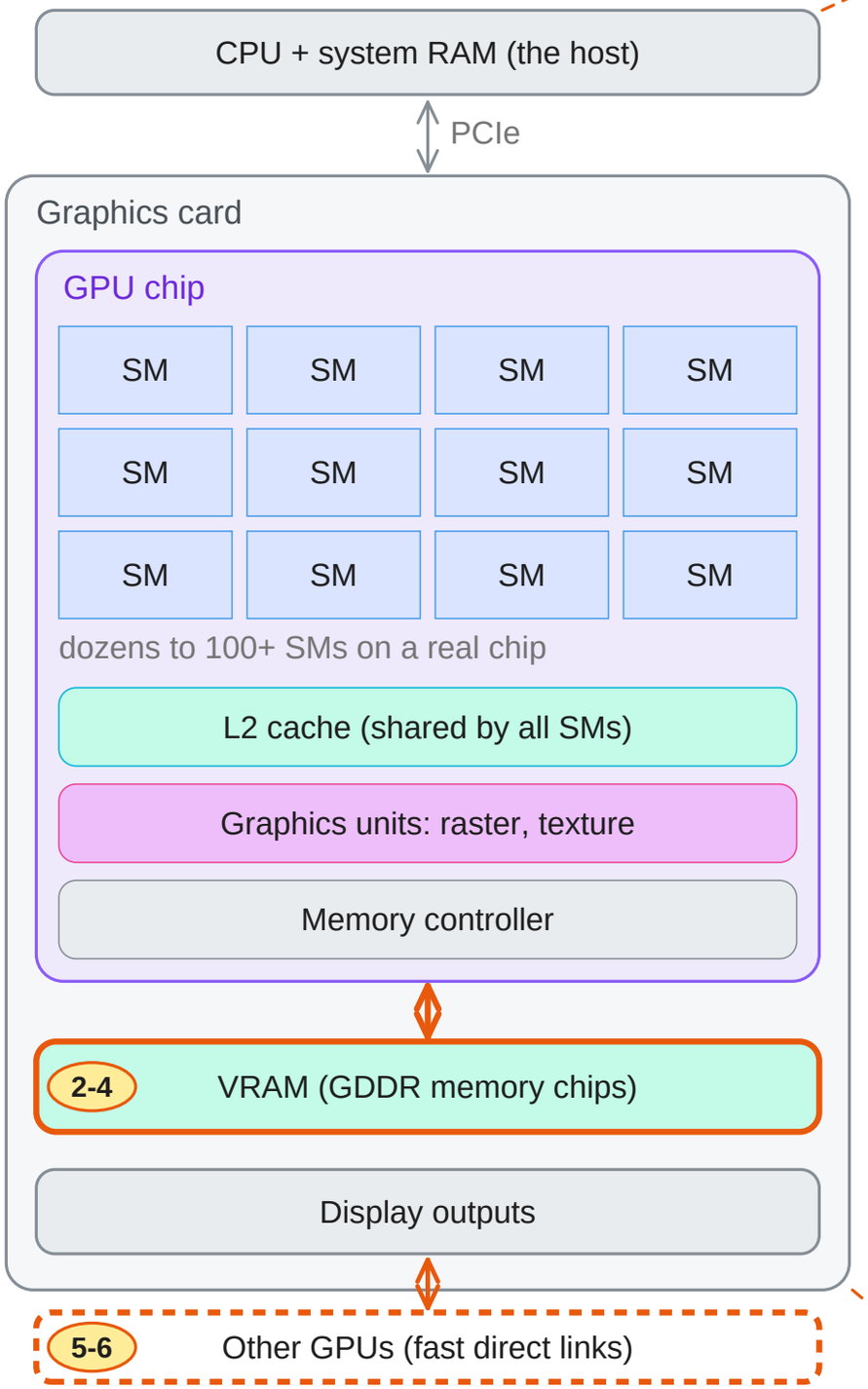
Tile memory: on-chip storage for the tile being drawn.

Immediate mode: the desktop way: draw each triangle as it arrives.

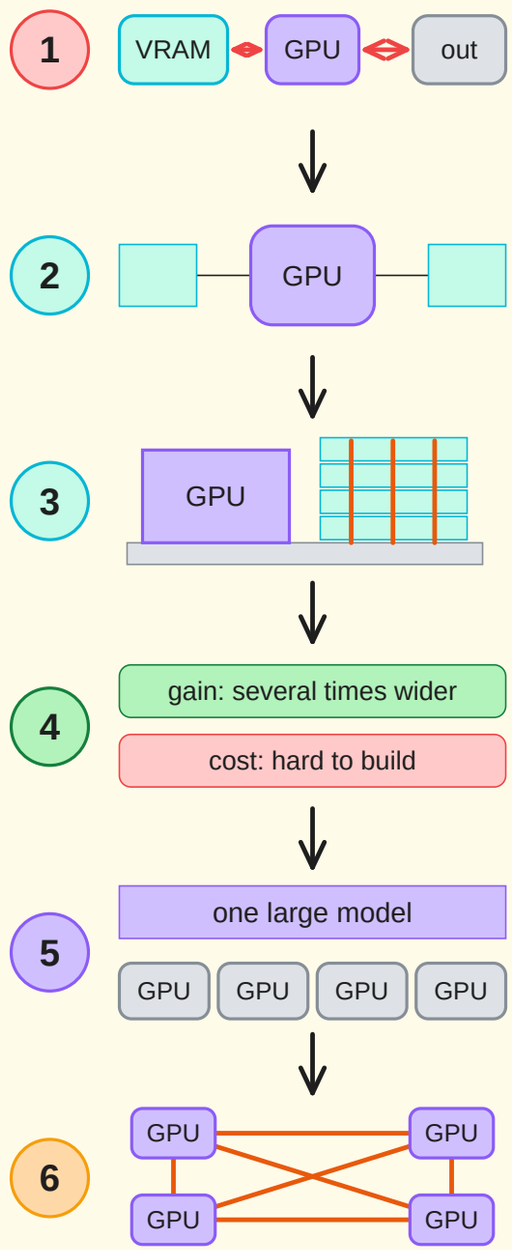
Lesson 8.4: Stacked memory and multi-GPU links

The hardware that widens the two slowest paths on the map: into the chip, and off the card.

The whole GPU (map)



Zoomed in: widening the pipes



The two slowest paths on the map

Big jobs hit two limits: how fast VRAM feeds the chip, and how fast data leaves the card. Both have hardware fixes.

Why ordinary VRAM tops out

Its chips sit around the GPU on the board, centimetres away. That allows only a few hundred wires, each already at its limit.

Stacked memory (HBM)

Memory layers are stacked and wired straight down through the silicon, then placed millimetres from the GPU.

What stacking buys, and costs

Thousands of short wires give several times the bandwidth. But it is costly to build, so only data-center GPUs get it.

One GPU is no longer enough

The largest models do not fit in one GPU's memory. The work is split across many, which must swap data constantly.

Direct GPU-to-GPU links

Dedicated links, many times faster than PCIe, connect GPUs to each other so data skips the CPU. Eight can act as one.

Key idea: when arithmetic is plentiful, progress comes from moving data faster, into each GPU and between them

Lesson 8.4 notes

Stacked memory and multi-GPU links

In brief

With tensor cores, arithmetic is rarely the limit any more. The limits are the path from memory into the chip and the path between GPUs. Stacked memory widens the first by putting thousands of wires millimetres from the chip. Direct GPU-to-GPU links widen the second.

Beyond the board

- **The numbers.** A top consumer card moves about 1,000 GB per second with a few hundred data wires. Stacked memory has over 1,000 data wires per stack, and a GPU with several stacks moves several thousand GB per second.
- **On the roofline.** Stacked memory steepens the sloped roof, and tensor cores raise the flat one. Data-center GPUs get both.
- **Link speeds.** NVIDIA's NVLink carries hundreds of GB per second between GPUs, against tens for PCIe.
- **More GPUs never scale perfectly.** The time spent exchanging data does not shrink as GPUs are added. That is Amdahl's law from Lesson 1.3.
- **The pattern repeats.** Inside an SM, inside a chip, inside a server: keep the lanes fed, and keep the data close to where it is used.

Check yourself

A model needs 300 GB of memory and each GPU has 80 GB. How many GPUs are needed at least? If 10% of the run time is data exchange that does not speed up with more GPUs, what is the most that 8 GPUs can gain over one?

Answer on page 86.

Key terms

HBM: high-bandwidth memory, stacked beside the chip.

Through-silicon via: a wire running straight down through a chip.

NVLink: NVIDIA's direct GPU-to-GPU link.

Scaling: how much faster a job gets as hardware is added.

Check yourself

Answers to the check questions

Lesson 1. What happens when a thread has to wait a long time for memory?

The core does not wait with it. The SM switches to another warp that is ready and comes back later. This is latency hiding (Lesson 2), and it is why slow memory access costs a GPU far less than it would cost a CPU.

Lesson 1.1. Cycles lost to wrong guesses per 100 instructions?

100 instructions contain 20 branches. 5% of those is 1 wrong guess, which costs 20 cycles. That is a lot next to the few dozen cycles a wide core would otherwise need for 100 instructions, and it is why predictors get so much chip area.

Lesson 1.2. Power cost of going from 4 GHz to 5 GHz?

25% more power, because power rises in step with frequency. To cancel that out, voltage squared would have to fall by a factor of 1.25, which means voltage itself falling by about 11%. That is the drop each new generation used to deliver and no longer can.

Lesson 1.3. 95% parallel, and that part made 100 times faster?

1 / (0.05 + 0.95 / 100) = about 16.8 times faster, not 100. The serial 5% now takes about 84% of the run time, so the next thing to work on is the serial part, not the GPU code.

Lesson 1.4. Classify grayscale, largest value, Fibonacci and histogram.

(a) Independent items: each pixel needs only itself. (b) Combining as a tree: compare in pairs. (c) A chain: each number needs the two before it. (d) Shared writes: many pixels add to the same bin.

Lesson 2. 31 threads take the if, 1 takes the else. How much is wasted?

The warp still spends a full pass on the else path with 31 of its 32 lanes masked off. About 97% of the warp's capacity is idle for that stretch.

Lesson 2.1. 1,000 elements with 512-bit registers: lanes, steps, leftovers?

512 / 32 = 16 lanes. 62 full steps cover 992 elements. The last 8 are handled by a short ordinary loop, or by one final vector step with a mask that switches off the unused lanes.

Lesson 2.2. 64 registers per thread: slots per warp, warps per SM?

64 x 32 = 2,048 slots per warp. 65,536 / 2,048 = 32 warps. A kernel that used half as many registers would fit twice as many.

Lesson 2.3. 31 threads need 10 iterations, one needs 1,000?

The warp runs 1,000 iterations. Useful work is 31 x 10 + 1,000 = 1,310 lane-iterations out of 32 x 1,000 = 32,000, so about 96% of the capacity is wasted.

Lesson 2.4. 20-cycle bursts and 300-cycle waits: warps needed?

1 + 300 / 20 = 16 warps. With 8 warps the core does 8 x 20 = 160 cycles of work in every 320, so it is busy 50% of the time.

Lesson 3. Twice the registers per thread, same arithmetic. Why slower?

Doubling the registers per thread halves the number of warps that fit in the SM's register file. With fewer warps resident, the scheduler more often finds nothing ready while the others wait on memory, so the execution units sit idle.

Check yourself

Answers to the check questions (continued)

Lesson 3.1. 4 schedulers, 3 ready warps: instructions issued?

At most 3, when the ready warps belong to three different schedulers. If all 3 belong to one scheduler, only 1 is issued, because each scheduler issues from one warp per cycle.

Lesson 3.2. 40 registers, 256-thread blocks, 12 KB each: occupancy?

A block needs 40 x 256 = 10,240 registers, so the registers allow 6 blocks. The warp cap allows 64 / 8 = 8 blocks and shared memory allows 96 / 12 = 8. Registers decide: 6 blocks is 48 warps, an occupancy of 75%.

Lesson 3.3. 100 SMs of 128 lanes at 2 GHz: peak rate?

100 x 128 x 2 billion x 2 = about 51 trillion operations per second. Real programs fall short because memory cannot feed the lanes that fast, because lanes sit idle (divergence, partly filled warps, no ready warp), and because not every instruction is a multiply-add.

Lesson 3.4. Instructions finished in 60 cycles: 1 warp or 6 warps?

With 1 warp, each instruction must wait for the previous one to write back, so about 10 finish. With 6 warps, a different warp starts every cycle and close to 60 finish. No single warp ran faster; the core simply stopped idling.

Lesson 4. Would twice the ALUs speed up c[i] = a[i] + b[i]?

No, barely at all. The kernel does one addition for every three numbers moved through memory, so the ALUs are already waiting on VRAM. It is limited by bandwidth, and only faster memory would speed it up.

Lesson 4.1. In-order reads with 128-byte lines: how many misses?

320 numbers of 4 bytes fill 10 lines, so 10 reads miss and 310 hit. Reading every 32nd element puts each read on a new line, so all 320 miss.

Lesson 4.2. Thread t reads word 2t, then 3t: how many cycles?

With 2 x t, threads t and t + 16 share a bank: 2 threads on each of 16 banks, so 2 cycles. With 3 x t, all 32 threads land on different banks, because 3 and 32 share no factor, so 1 cycle.

Lesson 4.3. 30 trillion ops/s and 1,000 GB/s: which limit binds?

The balance point is 30 operations per byte. The kernel does 200 / 8 = 25 per byte, which is below it, so memory binds: 1,000 GB/s / 8 bytes = 125 billion elements per second. The lanes alone could have managed 150 billion.

Lesson 4.4. 2 GB copy plus 0.4 s of computing per step?

The copy takes 0.1 s of a 0.5 s step, so 20%. Overlapped, a step takes 0.4 s, which is 25% more batches per second from the same hardware.

Lesson 5. How many warps for one 4K frame?

8.3 million fragments divided by 32 is about 260,000 warps for a single frame, and a game does that 60 or more times per second.

Lesson 5.1. 3 million vertices at 32 operations each, 60 times a second?

96 million operations per frame, or about 5.8 billion per second. That is roughly 0.03% of the chip. The vertex stage is rarely the bottleneck; the frame's time goes to fragments.

Check yourself

Answers to the check questions (continued)

Lesson 5.2. Shading a triangle that covers a single pixel?

A whole quad is shaded: 4 fragments for 1 visible pixel, so 75% is wasted. Two million tiny triangles cost about 8 million shader runs, against about 2 million for the single large triangle that fills the same pixels.

Lesson 5.3. Size of a 4,096 x 4,096 texture, and reads per 4K frame?

64 MB, or about 85 MB with mipmaps. One frame makes 8.3 million x 3 x 8, about 200 million texel reads, which is why texture filtering has its own hardware.

Lesson 5.4. 3 surfaces per pixel: late, early nearest-first, early farthest-first?

Late: 8.3 million x 3 x 500, about 12.5 billion. Early and nearest-first: only the front surface is shaded, about 4.2 billion. Early but farthest-first: every fragment passes the test when it arrives, so about 12.5 billion again.

Lesson 6. Block size 100: how many warps, and what is wasted?

Each block needs 4 warps, because 3 warps only cover 96 threads. The fourth warp carries just 4 threads, so 28 of its 32 lanes sit idle for the whole run. This is why block sizes are chosen as multiples of 32.

Lesson 6.1. 1,920 x 1,080 with 16 x 16 blocks: blocks and idle threads?

120 x 68 = 8,160 blocks, because 1,080 / 16 rounds up to 68. That is 2,088,960 threads for 2,073,600 pixels, so 15,360 are switched off. The thread handles pixel (163, 39), which lives at 39 x 1,920 + 163 = 75,043.

Lesson 6.2. Barriers per thread, and what the first barrier protects?

9 barriers: one after phase 1 and one in each of the 8 rounds. When half is 64, threads 0 to 63 add, which is 2 of the 8 warps. Without the first barrier a thread may read its partner's slot before the partner has written it, so the sum is sometimes wrong.

Lesson 6.3. 48 registers per thread: warps and occupancy?

A warp needs 48 x 32 = 1,536 registers, so 42 warps fit: about 66% occupancy. At 32 registers per thread a warp needs 1,024, and all 64 fit: 100%.

Lesson 6.4. 10,000 tiny launches or one big one?

The loop takes about 10,000 x 10 microseconds = 100 ms, and the GPU is busy for half of that. As one kernel the same work takes about 50 ms.

Lesson 7. out[i] = sqrt(data[index[i]]) with random index. Which limit first?

Scattered reads. Because index holds random positions, the 32 threads of a warp each read data from a different place, so one warp can trigger up to 32 separate fetches. The sqrt itself is cheap by comparison.

Lesson 7.1. 32-byte structures, one number read from each?

A warp's 32 particles span 32 x 32 = 1,024 bytes, which is 8 lines, and it uses 4 bytes of every 32: 12.5%. With a structure of arrays the same numbers fill 1 line and 100% is used.

Check yourself

Answers to the check questions (continued)

Lesson 7.2. 40 registers per thread: block size 32, 256 or 1,024?

Registers allow 65,536 / (40 x 32) = 51 warps. Blocks of 32: the block limit gives 32 warps, 50%. Blocks of 256: 6 whole blocks fit, 48 warps, 75%. Blocks of 1,024: only 1 block fits, 32 warps, 50%. The middle size wins.

Lesson 7.3. 16 bytes and 80 operations per element: which roof?

80 / 16 = 5 operations per byte, far left of the balance point at 40, so it is memory-bound: 500 GB/s / 16 bytes = about 31 billion elements per second. At 8 bytes it does 10 per byte, is still memory-bound, and reaches about 62 billion. Halving the bytes doubled the speed.

Lesson 7.4. A is 70% and B is 20%: which to make twice as fast?

Doubling B gives 1 / (0.7 + 0.1 + 0.1) = 1.11 times faster. Doubling A gives 1 / (0.35 + 0.2 + 0.1) = 1.54 times. Afterwards A is 54%, B is 31% and the rest is 15%, so B has become worth a look.

Lesson 8.1. Tensor cores 8 times faster: real speedup at 60 ops per byte?

40 and 320 operations per byte. On the lanes the kernel is compute-bound at 20 trillion per second. On tensor cores it is memory-bound at 500 billion x 60 = 30 trillion. The real speedup is 1.5 times, not 8.

Lesson 8.2. 16 million triangles: tree depth and tests per ray?

16 million is 2 to the power 24, so about 24 levels. 24 x 3 + 4 = 76 tests, which is roughly 200,000 times fewer than 16 million.

Lesson 8.3. 2,400 x 1,080 in 32 x 32 tiles: memory traffic per frame?

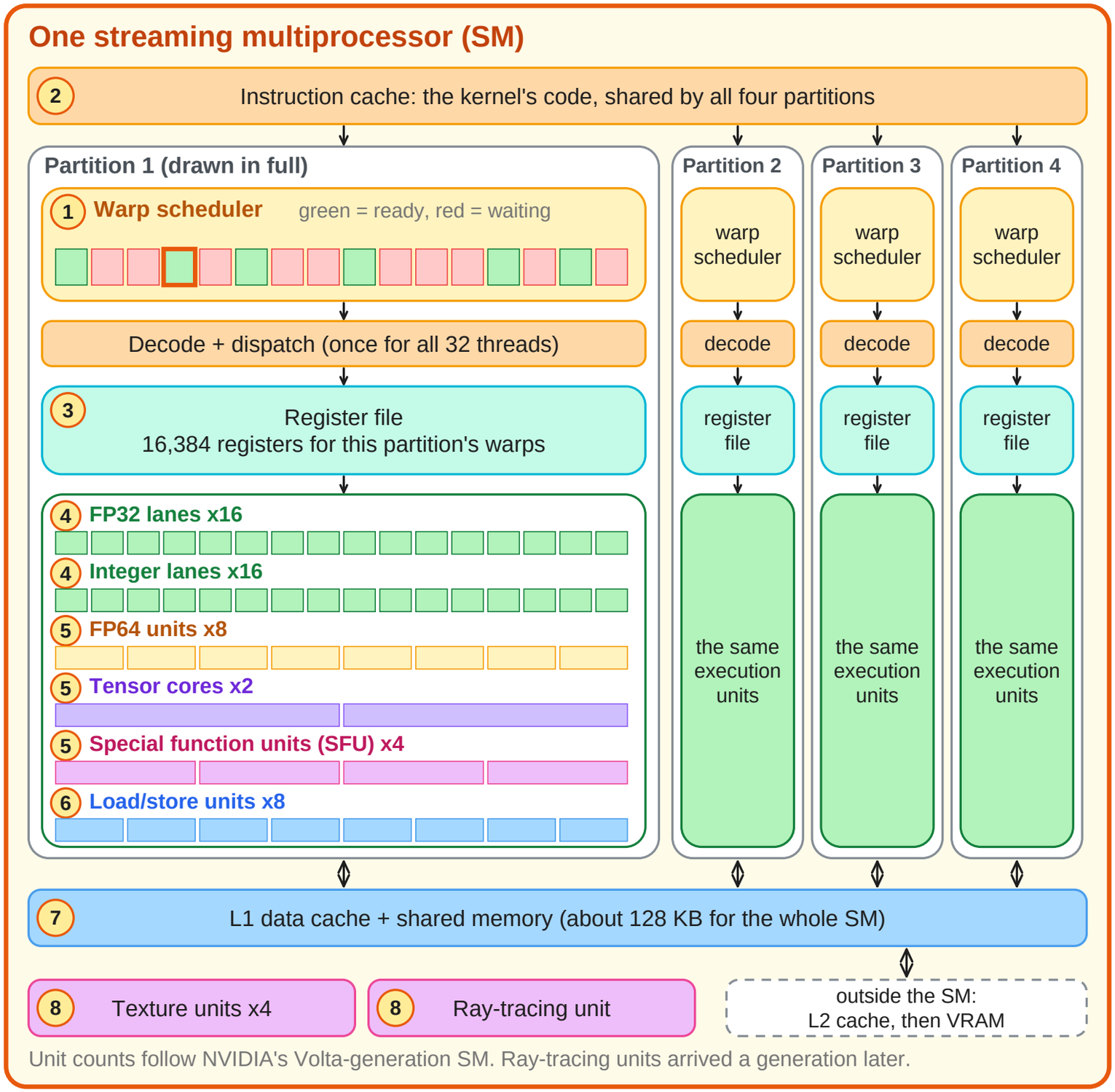
75 x 34 = 2,550 tiles, each needing 1,024 x 8 bytes, about 8 KB. Drawing as you go writes 2.6 million x 3 x 8 bytes, about 62 MB. With tiles only the final color is written: 2.6 million x 4 bytes, about 10 MB.

Lesson 8.4. 300 GB model on 80 GB GPUs, with 10% that does not scale?

300 / 80 = 3.75, so at least 4 GPUs. With 8 GPUs, Amdahl's law gives 1 / (0.10 + 0.90 / 8) = about 4.7 times, which is the same sum as Lesson 1.3. The course ends where it began: the part that cannot be spread out sets the limit.

Appendix: Inside one SM, in detail

Everything from Lessons 2 to 4 in one picture: the four partitions of an SM and the hardware they share.





This is one SM from the grid on every map. Numbers in brackets are the lessons to reread.

1 Warp scheduler

Tracks its warps as ready or waiting and issues one ready warp per cycle. (3.1)

2 Instruction cache and decode

The next instruction is fetched and decoded once for all 32 threads. (2.2, 3.4)

3 Register file

Private registers for every resident thread. Its size caps how many warps fit. (3.2)

4 FP32 and integer lanes

Everyday math. A warp of 32 goes through the 16 lanes as two halves. (3.3)

5 FP64 units, SFUs, tensor cores

Fewer, bigger units: double precision, sin and sqrt, small matrix products. (3.3, 8.1)

6 Load/store units

Compute memory addresses and pass each request on to the caches. (3.4, 7.1)

7 L1 cache and shared memory

Fast storage for the whole SM. Misses go on to L2 and then VRAM. (4.1, 4.2)

8 Texture and ray-tracing units

Fixed hardware the SM calls on for texture samples and ray hits. (5.3, 8.2)

Key idea: an SM is four small cores that share one instruction cache, one L1 and shared memory, and a few special units